



Setup Environment

If you are working on this assignment using Google Colab, please execute the codes below.

Alternatively, you can also do this assignment using a local anaconda environment (or a Python virtualenv). Please clone the GitHub repo by running `git clone https://github.com/Berkeley-CS182/cs182hw2.git` and refer to `README.md` for further details.

```
In [ ]: #@title Mount your Google Drive
```

```
import os
from google.colab import drive
drive.mount('/content/gdrive')
```

```
In [ ]: #@title Set up mount symlink
```

```
DRIVE_PATH = '/content/gdrive/My\ Drive/cs182hw2_sp25'
DRIVE_PYTHON_PATH = DRIVE_PATH.replace('\\', '/')
if not os.path.exists(DRIVE_PYTHON_PATH):
    %mkdir $DRIVE_PATH

## the space in `My Drive` causes some issues,
## make a symlink to avoid this
SYM_PATH = '/content/cs182hw2'
if not os.path.exists(SYM_PATH):
    !ln -s $DRIVE_PATH $SYM_PATH
```

```
In [ ]: #@title Install dependencies
```

```
!pip install --no-deps --upgrade IPython
import matplotlib
%matplotlib inline
%load_ext autoreload
%autoreload 2
```

```
In [ ]: #@title Clone homework repo
```

```
%cd $SYM_PATH
!git clone https://github.com/Berkeley-CS182/cs182sp25_public.git
%cd cs182sp25_public/hw02/q_coding_optimizer_init/code
```

```
In [ ]: #@title Download datasets
```

```
%cd deeplearning/datasets/
!bash ./get_datasets_curl.sh
%cd ../../
```

In []: `#@title Configure Jupyter Notebook`

Optimization Methods and Initialization

Until now, you've always used Gradient Descent to update the parameters and minimize the cost. In this notebook, you will learn more advanced optimization methods that can speed up learning and perhaps even get you to a better final value for the cost function. Having a good optimization algorithm can be the difference between waiting days vs. just a few hours to get a good result.

Gradient descent goes "downhill" on a cost function J . Think of it as trying to do this:

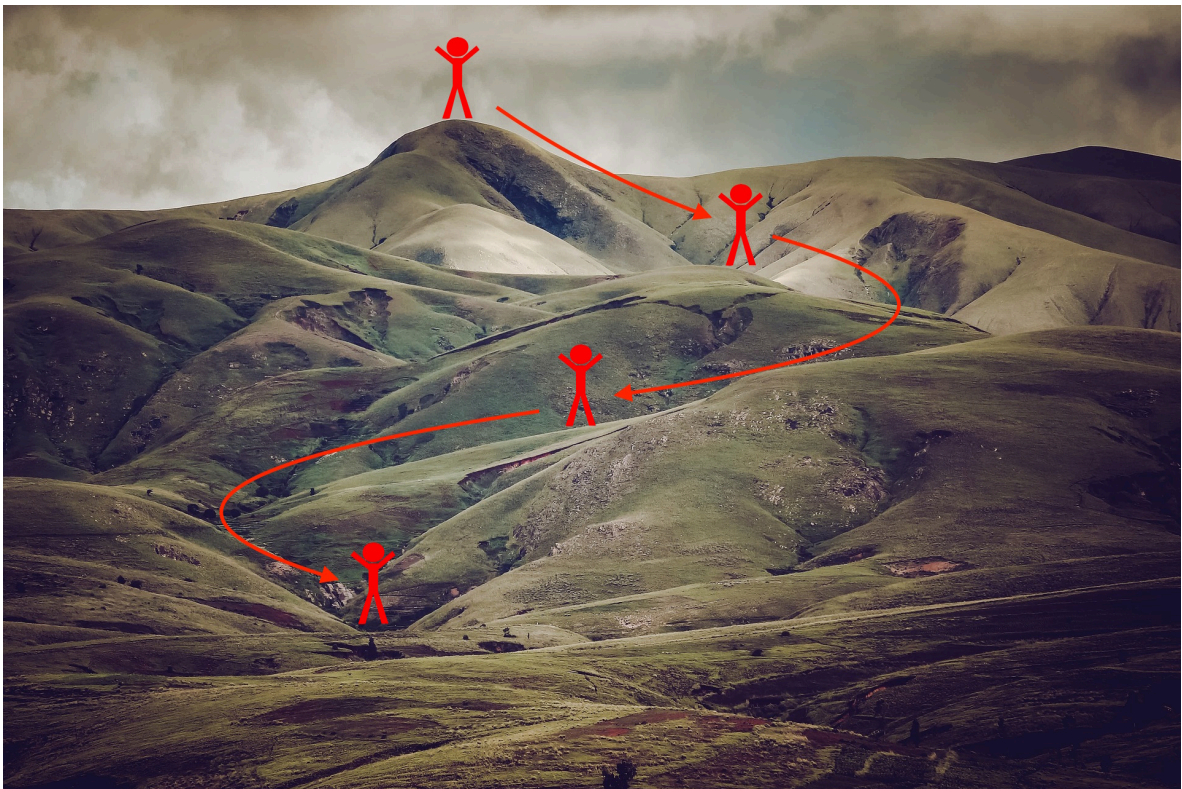


Figure 1: Minimizing the cost is like finding the lowest point in a hilly landscape

At each step of the training, you update your parameters following a certain direction to try to get to the lowest possible point.

```
In [1]: # As usual, a bit of setup

import json
import time
import numpy as np
import matplotlib.pyplot as plt
```

```

from deeplearning.classifiers.fc_net import *
from deeplearning.data_utils import get_CIFAR10_data
from deeplearning.gradient_check import eval_numerical_gradient, eval_numerical_gradient_jacobian
from deeplearning.solver import Solver
import random
import torch
seed = 7
torch.manual_seed(seed)
random.seed(seed)
np.random.seed(seed)

plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipython
%autoreload 2

def rel_error(x, y):
    """ returns relative error """
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))

```

In [2]: # Load the (preprocessed) CIFAR10 data.

```

data = get_CIFAR10_data()
for k, v in data.items():
    print('%s: ' % k, v.shape)

```

```

deeplearning/datasets/cifar-10-batches-py\data_batch_1
deeplearning/datasets/cifar-10-batches-py\data_batch_2
deeplearning/datasets/cifar-10-batches-py\data_batch_3
deeplearning/datasets/cifar-10-batches-py\data_batch_4
deeplearning/datasets/cifar-10-batches-py\data_batch_5
deeplearning/datasets/cifar-10-batches-py\test_batch
X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
X_test: (1000, 3, 32, 32)
y_test: (1000,)

```

1 - Stochastic Gradient Descent

A simple optimization method in machine learning is gradient descent (GD). When you take gradient steps with respect to all m examples on each step, it is also called Batch Gradient Descent.

A variant of this is Stochastic Gradient Descent (SGD), which is equivalent to mini-batch gradient descent where each mini-batch has just 1 example. The update rule that you have just implemented does not change. What changes is that you would

be computing gradients on just one training example at a time, rather than on the whole training set. The code examples below illustrate the difference between stochastic gradient descent and (batch) gradient descent.

In Stochastic Gradient Descent, you use only 1 training example before updating the gradients. When the training set is large, SGD can be faster. But the parameters will "oscillate" toward the minimum rather than converge smoothly. Here is an illustration of this:

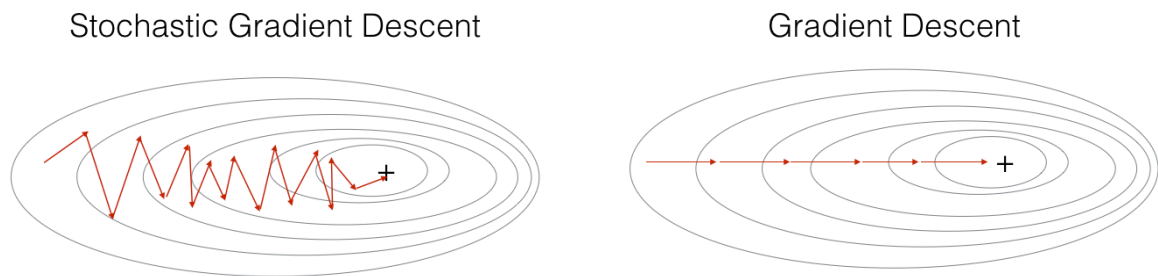


Figure 1: SGD vs GD

"+" denotes a minimum of the cost. SGD leads to many oscillations to reach convergence. But each step is a lot faster to compute for SGD than for GD, as it uses only one training example (vs. the whole batch for GD).

In the following code snippet, we will use SGD to optimize a five-layer FullyConnectedNet so that it overfits to 50 training examples.

```
In [3]: ## Use a five-layer Net to overfit 50 training examples.

num_train = 50
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}

weight_scale = 1e-1
learning_rate = 1e-3
model = FullyConnectedNet([100, 100, 100, 100],
                           weight_scale=weight_scale, dtype=np.float64)

solver = Solver(model, small_data,
                 print_every=10, num_epochs=20, batch_size=25,
                 update_rule='sgd',
                 optim_config={
                     'learning_rate': learning_rate,
                 })
solver.train()
```

```

plt.subplot(3, 1, 1)
plt.plot(solver.loss_history, 'o')
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')

plt.subplot(3, 1, 2)
plt.plot(solver.train_acc_history, 'o')
plt.title('Training Accuracy history')
plt.xlabel('Iteration')
plt.ylabel('Training Accuracy')

plt.subplot(3, 1, 3)
plt.plot(solver.val_acc_history, 'o')
plt.title('Validation Accuracy history')
plt.xlabel('Iteration')
plt.ylabel('Validation Accuracy')
plt.gcf().set_size_inches(15, 15)

plt.show()

```

```

(Iteration 1 / 40) loss: 94.233583
(Epoch 0 / 20) train acc: 0.140000; val_acc: 0.123000
(Epoch 1 / 20) train acc: 0.120000; val_acc: 0.102000
(Epoch 2 / 20) train acc: 0.320000; val_acc: 0.118000
(Epoch 3 / 20) train acc: 0.480000; val_acc: 0.133000

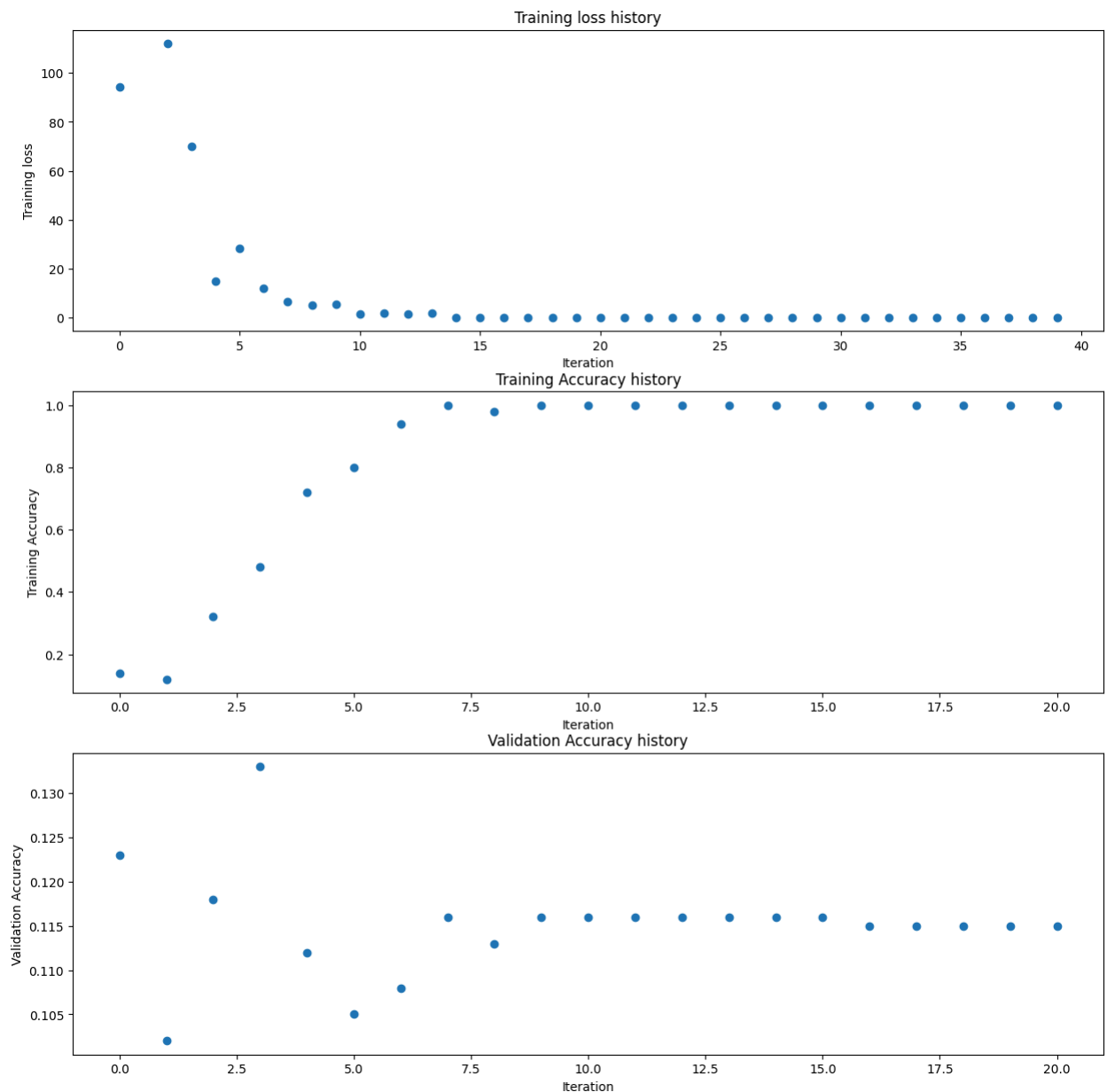
```

C:\Users\NLP\Desktop\homework\DL\QWen4HW2\deeplearning\layers.py:153: RuntimeWarning: divide by zero encountered in log

```

    loss = -np.sum(np.log(probs[np.arange(N), y])) / N
(Epoch 4 / 20) train acc: 0.720000; val_acc: 0.112000
(Epoch 5 / 20) train acc: 0.800000; val_acc: 0.105000
(Iteration 11 / 40) loss: 1.410664
(Epoch 6 / 20) train acc: 0.940000; val_acc: 0.108000
(Epoch 7 / 20) train acc: 1.000000; val_acc: 0.116000
(Epoch 8 / 20) train acc: 0.980000; val_acc: 0.113000
(Epoch 9 / 20) train acc: 1.000000; val_acc: 0.116000
(Epoch 10 / 20) train acc: 1.000000; val_acc: 0.116000
(Iteration 21 / 40) loss: 0.000076
(Epoch 11 / 20) train acc: 1.000000; val_acc: 0.116000
(Epoch 12 / 20) train acc: 1.000000; val_acc: 0.116000
(Epoch 13 / 20) train acc: 1.000000; val_acc: 0.116000
(Epoch 14 / 20) train acc: 1.000000; val_acc: 0.116000
(Epoch 15 / 20) train acc: 1.000000; val_acc: 0.116000
(Iteration 31 / 40) loss: 0.000327
(Epoch 16 / 20) train acc: 1.000000; val_acc: 0.115000
(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.115000
(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.115000
(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.115000
(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.115000

```



2 - Momentum

Because mini-batch gradient descent makes a parameter update after seeing just a subset of examples, the direction of the update has some variance, and so the path taken by mini-batch gradient descent will "oscillate" toward convergence. Using momentum can reduce these oscillations.

Momentum takes into account the past gradients to smooth out the update. We will store the 'direction' of the previous gradients in the variable v . Formally, this will be the exponentially weighted average of the gradient on previous steps. You can also think of v as the "velocity" of a ball rolling downhill, building up speed (and momentum) according to the direction of the gradient/slope of the hill.

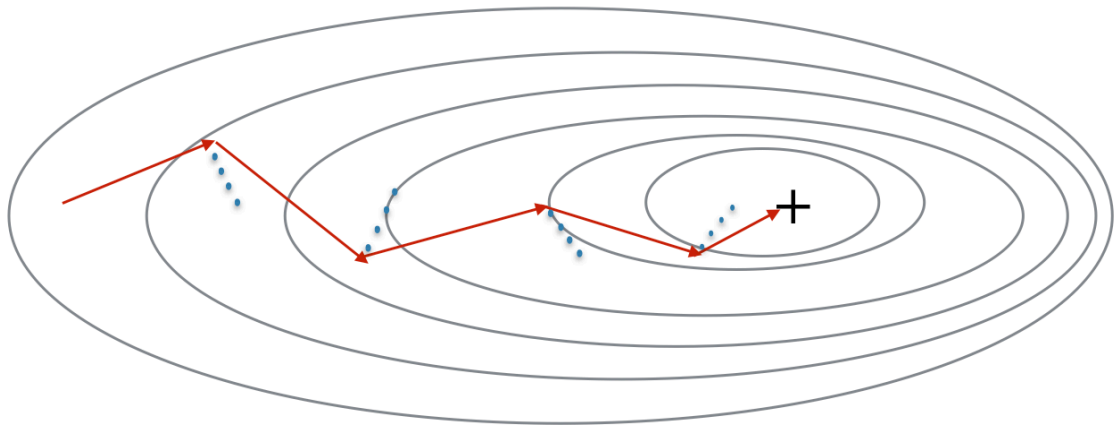


Figure 3: The red arrows shows the direction taken by one step of mini-batch gradient descent with momentum. The blue points show the direction of the gradient (with respect to the current mini-batch) on each step. Rather than just following the gradient, we let the gradient influence v (velocity) and then take a step in the direction of v .

The momentum update rule for a weight matrix w is:

$$\begin{cases} v_{dw}^t = m * v_{dw}^{(t-1)} + dw \\ \{w\} = w - \alpha v_{dw}^t \end{cases}$$

where m is the momentum and α is the learning rate. Note that the iterator t starts at 1.

Stochastic gradient descent with momentum is a widely used update rule that tends to make deep networks converge faster than vanilla stochastic gradient descent, it can be viewed conceptually a larger "effective batch size" versus vanilla stochastic gradient descent.

Open the file `deeplearning/optim.py` and read the documentation at the top of the file to make sure you understand the API. **Implement the SGD+momentum update rule** in the function `sgd_momentum` and run the following to check your implementation. You should see errors less than $1e-7$.

```
In [4]: from deeplearning.optim import sgd_momentum

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
v = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-3, 'velocity': v}
next_w, _ = sgd_momentum(w, dw, config=config)

expected_next_w = np.asarray([
```

```

[-0.39994, -0.347375263, -0.294810526, -0.242245789, -0.189681053],
[-0.137116316, -0.084551579, -0.031986842, 0.020577895, 0.073142632],
[0.125707368, 0.178272105, 0.230836842, 0.283401579, 0.335966316],
[0.388531053, 0.441095789, 0.493660526, 0.546225263, 0.59879]]])
expected_velocity = np.asarray([
[-0.06, 0.006842105, 0.073684211, 0.140526316, 0.207368421],
[0.274210526, 0.341052632, 0.407894737, 0.474736842, 0.541578947],
[0.608421053, 0.675263158, 0.742105263, 0.808947368, 0.875789474],
[0.942631579, 1.009473684, 1.076315789, 1.143157895, 1.21]
])

print ('next_w error: ', rel_error(next_w, expected_next_w))
print ('velocity error: ', rel_error(expected_velocity, config['velocity']))

```

```

next_w error: 6.3941900171621575e-09
velocity error: 1.923077600561035e-08

```

Once you have done so, run the following to train a six-layer network with both SGD and SGD+momentum. You should see the SGD+momentum update rule converge a bit faster.

```

In [5]: import os
num_train = 4000
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}

solvers = {}

for update_rule in ['sgd', 'sgd_momentum']:
    print ('running with ', update_rule)
    model = FullyConnectedNet([100, 100, 100, 100, 100], weight_scale=5e-2)

    solver = Solver(model, small_data,
                    num_epochs=5, batch_size=100,
                    update_rule=update_rule,
                    optim_config={
                        'learning_rate': 1e-2,
                    },
                    verbose=True)
    solvers[update_rule] = solver
    solver.train()
    os.makedirs("submission_logs", exist_ok=True)
    solver.record_histories_as_npz("submission_logs/optimizer_experiment_{}".f
    print

plt.subplot(3, 1, 1)
plt.title('Training loss')
plt.xlabel('Iteration')

```



```

plt.subplot(3, 1, 2)
plt.title('Training accuracy')
plt.xlabel('Epoch')

plt.subplot(3, 1, 3)
plt.title('Validation accuracy')
plt.xlabel('Epoch')

for update_rule, solver in solvers.items():
    plt.subplot(3, 1, 1)
    plt.plot(solver.loss_history, 'o', label=update_rule)

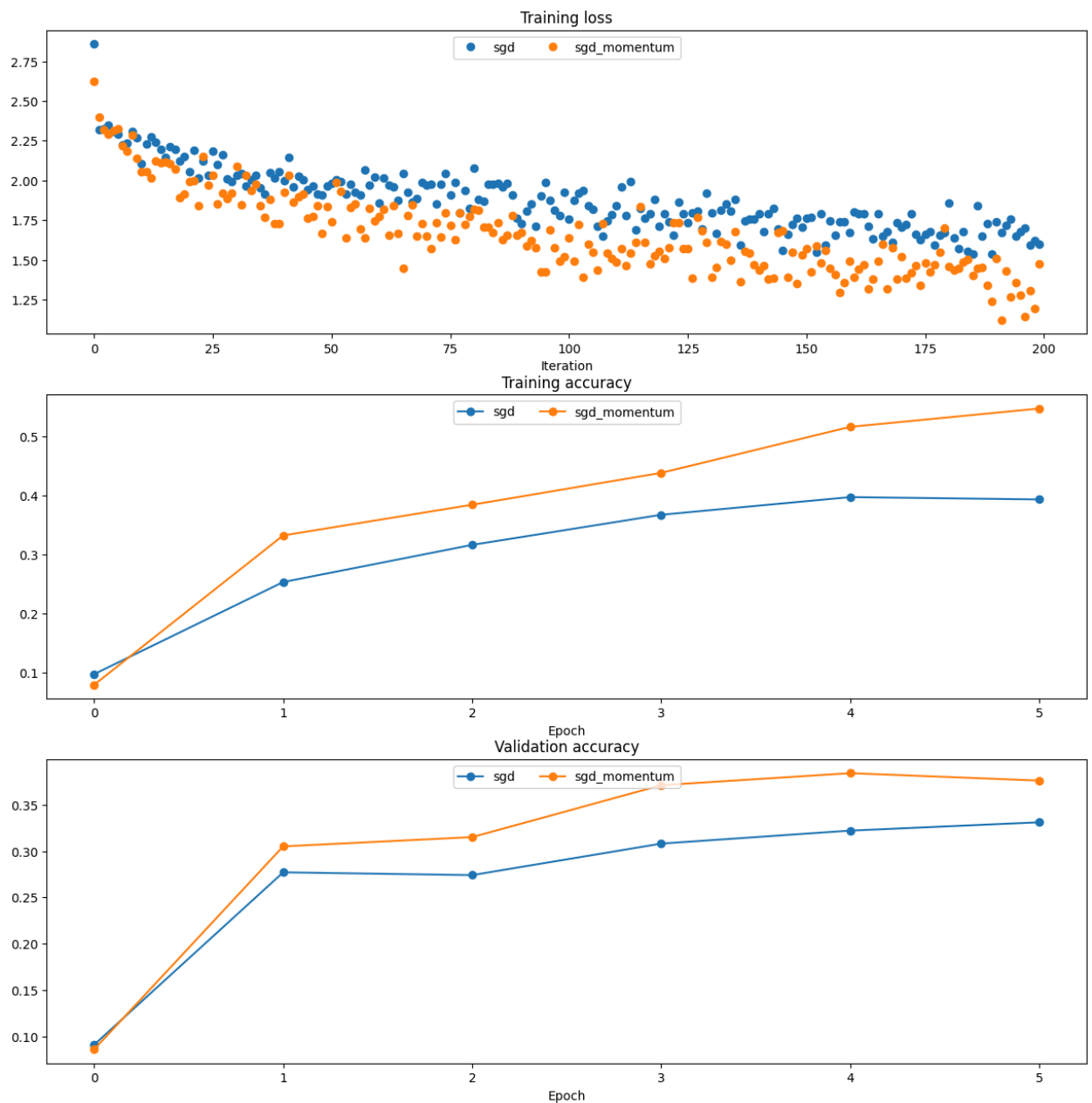
    plt.subplot(3, 1, 2)
    plt.plot(solver.train_acc_history, '-o', label=update_rule)

    plt.subplot(3, 1, 3)
    plt.plot(solver.val_acc_history, '-o', label=update_rule)

for i in [1, 2, 3]:
    plt.subplot(3, 1, i)
    plt.legend(loc='upper center', ncol=4)
plt.gcf().set_size_inches(15, 15)
plt.show()

```

```
running with  sgd
(Iteration 1 / 200) loss: 2.859678
(Epoch 0 / 5) train acc: 0.097000; val_acc: 0.091000
(Iteration 11 / 200) loss: 2.108301
(Iteration 21 / 200) loss: 2.054191
(Iteration 31 / 200) loss: 2.032144
(Epoch 1 / 5) train acc: 0.253000; val_acc: 0.277000
(Iteration 41 / 200) loss: 2.001398
(Iteration 51 / 200) loss: 1.982336
(Iteration 61 / 200) loss: 1.859230
(Iteration 71 / 200) loss: 1.972533
(Epoch 2 / 5) train acc: 0.316000; val_acc: 0.274000
(Iteration 81 / 200) loss: 2.080138
(Iteration 91 / 200) loss: 1.725780
(Iteration 101 / 200) loss: 1.754550
(Iteration 111 / 200) loss: 1.843685
(Epoch 3 / 5) train acc: 0.367000; val_acc: 0.308000
(Iteration 121 / 200) loss: 1.789679
(Iteration 131 / 200) loss: 1.797600
(Iteration 141 / 200) loss: 1.792210
(Iteration 151 / 200) loss: 1.762869
(Epoch 4 / 5) train acc: 0.397000; val_acc: 0.322000
(Iteration 161 / 200) loss: 1.799905
(Iteration 171 / 200) loss: 1.708248
(Iteration 181 / 200) loss: 1.858177
(Iteration 191 / 200) loss: 1.741494
(Epoch 5 / 5) train acc: 0.393000; val_acc: 0.331000
running with  sgd_momentum
(Iteration 1 / 200) loss: 2.624924
(Epoch 0 / 5) train acc: 0.079000; val_acc: 0.086000
(Iteration 11 / 200) loss: 2.055534
(Iteration 21 / 200) loss: 1.990266
(Iteration 31 / 200) loss: 2.090398
(Epoch 1 / 5) train acc: 0.332000; val_acc: 0.305000
(Iteration 41 / 200) loss: 1.924605
(Iteration 51 / 200) loss: 1.738813
(Iteration 61 / 200) loss: 1.770769
(Iteration 71 / 200) loss: 1.647567
(Epoch 2 / 5) train acc: 0.384000; val_acc: 0.315000
(Iteration 81 / 200) loss: 1.820153
(Iteration 91 / 200) loss: 1.673285
(Iteration 101 / 200) loss: 1.640875
(Iteration 111 / 200) loss: 1.486664
(Epoch 3 / 5) train acc: 0.438000; val_acc: 0.371000
(Iteration 121 / 200) loss: 1.510025
(Iteration 131 / 200) loss: 1.387503
(Iteration 141 / 200) loss: 1.432835
(Iteration 151 / 200) loss: 1.568692
(Epoch 4 / 5) train acc: 0.516000; val_acc: 0.384000
(Iteration 161 / 200) loss: 1.391197
(Iteration 171 / 200) loss: 1.521790
(Iteration 181 / 200) loss: 1.460578
(Iteration 191 / 200) loss: 1.509308
(Epoch 5 / 5) train acc: 0.547000; val_acc: 0.376000
```



A further step: as we discussed above, we can see how SGD+Momentum is conceptually giving you a larger "effective batch size" by increase the batch size used in the SGD above. In this way, SGD+Momentum can significantly speed up training.

Tune the batch size for plain SGD so that the training accuracy is similar to that of SGD with momentum. The average accuracy difference between them should be less than 0.04 . The accuracy is averaged over three different random seeds for better stability.

```
In [6]: #####
# TODO: Tune the batch size for the SGD below until you observe #
# similar end of iteration training performance. #
```

```

# It means rel_error(train_acc) < 0.04 #
#####
batch_sizes = {
    'sgd_momentum': 100,
    'sgd': 500, # try 400, 500, 600 - 500 often works
}

num_train = 6000
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}

solvers = {}
total_acc = {}

labels = {
    'sgd_momentum': 'sgd_momentum',
    'sgd': 'sgd_large_bsz',
}

for update_rule in ['sgd', 'sgd_momentum']:
    print('running with', update_rule, ' ; seed =', seed)
    # set the epochs so that we have the same number of steps for both rules
    training_epochs = 5 * int(batch_sizes[update_rule]/100)
    solvers[update_rule] = {}
    total_acc[update_rule] = 0

    for seed in [100, 200, 300]:
        torch.manual_seed(seed)
        np.random.seed(seed)
        model = FullyConnectedNet([100, 100, 100, 100, 100], weight_scale=5e-2

        solver = Solver(
            model, small_data,
            num_epochs=training_epochs,
            batch_size=batch_sizes[update_rule],
            update_rule=update_rule,
            optim_config={
                'learning_rate': 1e-2, # please do not change the learning ra
            },
            verbose=True,
            log_acc_iteration=True)

        solvers[update_rule][seed] = solver
        solver.train()
        solver.record_histories_as_npz(
            "submission_logs/sgd_momentum_compare_{}_{}".format(update_rule, seed)
        )

```

```

        total_acc[update_rule] += solvers[update_rule][seed].train_acc_history

print('Average Training Acc for sgd:', total_acc['sgd'] / 3)
print('Average Training Acc for sgd_momentum:', total_acc['sgd_momentum'] / 3)
print('Train Acc Difference: ',
      rel_error(total_acc['sgd'] / 3,
                total_acc['sgd_momentum'] / 3))

def plot_solver_seeds(solver_s, x_field, y_field, seeds, label):
    a = np.array([getattr(solver_s[seed], y_field) for seed in seeds])
    if x_field is None:
        plt_x = np.arange(a.shape[1]) + 1
    else:
        plt_x = getattr(solver_s[seeds[0]], x_field)
    plt.plot(plt_x, a.mean(axis=0), label=label)
    plt.fill_between(plt_x, a.min(axis=0), a.max(axis=0), alpha=0.4)

plt.subplot(3, 1, 1)
plt.title('Training loss')
plt.xlabel('Iteration')

plt.subplot(3, 1, 2)
plt.title('Training accuracy')
plt.xlabel('Iteration')

plt.subplot(3, 1, 3)
plt.title('Validation accuracy')
plt.xlabel('Iteration')

for update_rule, solver_s in solvers.items():
    plt.subplot(3, 1, 1)
    # plt.plot(solver.loss_history, 'o', label=labels[update_rule])
    plot_solver_seeds(solver_s, None, 'loss_history',
                      [100, 200, 300], labels[update_rule])

    plt.subplot(3, 1, 2)
    # plt.plot(solver.log_acc_iteration_history, solver.train_acc_history, '-o',
    plot_solver_seeds(solver_s, 'log_acc_iteration_history', 'train_acc_history',
                      [100, 200, 300], labels[update_rule])

    plt.subplot(3, 1, 3)
    # plt.plot(solver.log_acc_iteration_history, solver.val_acc_history, '-o',
    plot_solver_seeds(solver_s, 'log_acc_iteration_history', 'val_acc_history',
                      [100, 200, 300], labels[update_rule])

for i in [1, 2, 3]:
    plt.subplot(3, 1, i)
    plt.legend(loc='upper center', ncol=4)
plt.gcf().set_size_inches(15, 15)
plt.show()

```

```
running with sgd ; seed = 7
(Iteration 1 / 300) loss: 2.580742
(Epoch 0 / 25) train acc: 0.103000; val_acc: 0.094000
(Iteration 11 / 300) loss: 2.206216
(Epoch 1 / 25) train acc: 0.196000; val_acc: 0.185000
(Iteration 21 / 300) loss: 2.132044
(Epoch 2 / 25) train acc: 0.219000; val_acc: 0.228000
(Iteration 31 / 300) loss: 2.039230
(Epoch 3 / 25) train acc: 0.265000; val_acc: 0.254000
(Iteration 41 / 300) loss: 2.014709
(Epoch 4 / 25) train acc: 0.304000; val_acc: 0.289000
(Iteration 51 / 300) loss: 1.988363
(Epoch 5 / 25) train acc: 0.316000; val_acc: 0.283000
(Iteration 61 / 300) loss: 1.935177
(Iteration 71 / 300) loss: 1.849781
(Epoch 6 / 25) train acc: 0.333000; val_acc: 0.300000
(Iteration 81 / 300) loss: 1.888813
(Epoch 7 / 25) train acc: 0.314000; val_acc: 0.310000
(Iteration 91 / 300) loss: 1.844999
(Epoch 8 / 25) train acc: 0.345000; val_acc: 0.317000
(Iteration 101 / 300) loss: 1.809510
(Epoch 9 / 25) train acc: 0.375000; val_acc: 0.314000
(Iteration 111 / 300) loss: 1.811080
(Epoch 10 / 25) train acc: 0.359000; val_acc: 0.323000
(Iteration 121 / 300) loss: 1.771826
(Iteration 131 / 300) loss: 1.747219
(Epoch 11 / 25) train acc: 0.330000; val_acc: 0.326000
(Iteration 141 / 300) loss: 1.771499
(Epoch 12 / 25) train acc: 0.390000; val_acc: 0.319000
(Iteration 151 / 300) loss: 1.772557
(Epoch 13 / 25) train acc: 0.402000; val_acc: 0.329000
(Iteration 161 / 300) loss: 1.701948
(Epoch 14 / 25) train acc: 0.367000; val_acc: 0.321000
(Iteration 171 / 300) loss: 1.751552
(Epoch 15 / 25) train acc: 0.350000; val_acc: 0.344000
(Iteration 181 / 300) loss: 1.651430
(Iteration 191 / 300) loss: 1.607058
(Epoch 16 / 25) train acc: 0.416000; val_acc: 0.339000
(Iteration 201 / 300) loss: 1.688633
(Epoch 17 / 25) train acc: 0.442000; val_acc: 0.336000
(Iteration 211 / 300) loss: 1.701959
(Epoch 18 / 25) train acc: 0.405000; val_acc: 0.357000
(Iteration 221 / 300) loss: 1.679740
(Epoch 19 / 25) train acc: 0.446000; val_acc: 0.335000
(Iteration 231 / 300) loss: 1.616269
(Epoch 20 / 25) train acc: 0.433000; val_acc: 0.349000
(Iteration 241 / 300) loss: 1.570428
(Iteration 251 / 300) loss: 1.560517
(Epoch 21 / 25) train acc: 0.408000; val_acc: 0.332000
(Iteration 261 / 300) loss: 1.608994
(Epoch 22 / 25) train acc: 0.461000; val_acc: 0.346000
(Iteration 271 / 300) loss: 1.523120
(Epoch 23 / 25) train acc: 0.438000; val_acc: 0.351000
(Iteration 281 / 300) loss: 1.557269
```

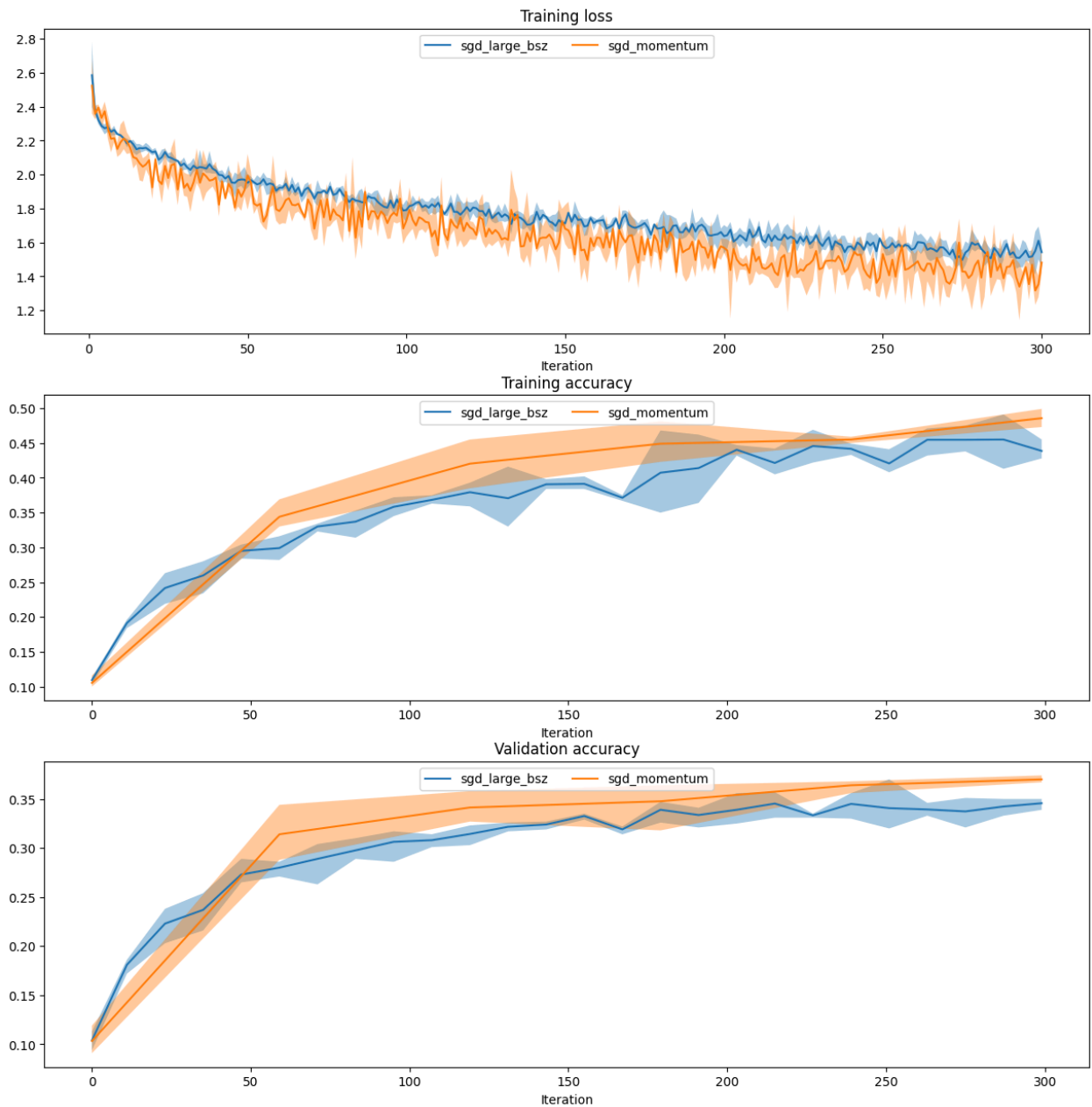
(Epoch 24 / 25) train acc: 0.491000; val_acc: 0.344000
(Iteration 291 / 300) loss: 1.511508
(Epoch 25 / 25) train acc: 0.455000; val_acc: 0.339000
(Iteration 1 / 300) loss: 2.397426
(Epoch 0 / 25) train acc: 0.112000; val_acc: 0.105000
(Iteration 11 / 300) loss: 2.215410
(Epoch 1 / 25) train acc: 0.194000; val_acc: 0.172000
(Iteration 21 / 300) loss: 2.155818
(Epoch 2 / 25) train acc: 0.243000; val_acc: 0.203000
(Iteration 31 / 300) loss: 2.029046
(Epoch 3 / 25) train acc: 0.234000; val_acc: 0.216000
(Iteration 41 / 300) loss: 2.019312
(Epoch 4 / 25) train acc: 0.284000; val_acc: 0.265000
(Iteration 51 / 300) loss: 1.966345
(Epoch 5 / 25) train acc: 0.282000; val_acc: 0.271000
(Iteration 61 / 300) loss: 1.928889
(Iteration 71 / 300) loss: 1.887423
(Epoch 6 / 25) train acc: 0.323000; val_acc: 0.263000
(Iteration 81 / 300) loss: 1.863041
(Epoch 7 / 25) train acc: 0.353000; val_acc: 0.289000
(Iteration 91 / 300) loss: 1.906766
(Epoch 8 / 25) train acc: 0.358000; val_acc: 0.286000
(Iteration 101 / 300) loss: 1.826982
(Epoch 9 / 25) train acc: 0.367000; val_acc: 0.301000
(Iteration 111 / 300) loss: 1.808799
(Epoch 10 / 25) train acc: 0.386000; val_acc: 0.317000
(Iteration 121 / 300) loss: 1.809198
(Iteration 131 / 300) loss: 1.829543
(Epoch 11 / 25) train acc: 0.366000; val_acc: 0.322000
(Iteration 141 / 300) loss: 1.835600
(Epoch 12 / 25) train acc: 0.384000; val_acc: 0.327000
(Iteration 151 / 300) loss: 1.773407
(Epoch 13 / 25) train acc: 0.388000; val_acc: 0.334000
(Iteration 161 / 300) loss: 1.735975
(Epoch 14 / 25) train acc: 0.372000; val_acc: 0.322000
(Iteration 171 / 300) loss: 1.692170
(Epoch 15 / 25) train acc: 0.404000; val_acc: 0.326000
(Iteration 181 / 300) loss: 1.788764
(Iteration 191 / 300) loss: 1.673321
(Epoch 16 / 25) train acc: 0.364000; val_acc: 0.321000
(Iteration 201 / 300) loss: 1.677029
(Epoch 17 / 25) train acc: 0.432000; val_acc: 0.325000
(Iteration 211 / 300) loss: 1.728901
(Epoch 18 / 25) train acc: 0.417000; val_acc: 0.331000
(Iteration 221 / 300) loss: 1.631646
(Epoch 19 / 25) train acc: 0.422000; val_acc: 0.334000
(Iteration 231 / 300) loss: 1.560325
(Epoch 20 / 25) train acc: 0.449000; val_acc: 0.330000
(Iteration 241 / 300) loss: 1.521566
(Iteration 251 / 300) loss: 1.616031
(Epoch 21 / 25) train acc: 0.413000; val_acc: 0.370000
(Iteration 261 / 300) loss: 1.641112
(Epoch 22 / 25) train acc: 0.432000; val_acc: 0.333000
(Iteration 271 / 300) loss: 1.551177

(Epoch 23 / 25) train acc: 0.475000; val_acc: 0.340000
(Iteration 281 / 300) loss: 1.554744
(Epoch 24 / 25) train acc: 0.413000; val_acc: 0.350000
(Iteration 291 / 300) loss: 1.609617
(Epoch 25 / 25) train acc: 0.428000; val_acc: 0.348000
(Iteration 1 / 300) loss: 2.778562
(Epoch 0 / 25) train acc: 0.113000; val_acc: 0.113000
(Iteration 11 / 300) loss: 2.219198
(Epoch 1 / 25) train acc: 0.184000; val_acc: 0.186000
(Iteration 21 / 300) loss: 2.120570
(Epoch 2 / 25) train acc: 0.263000; val_acc: 0.238000
(Iteration 31 / 300) loss: 2.058660
(Epoch 3 / 25) train acc: 0.280000; val_acc: 0.241000
(Iteration 41 / 300) loss: 1.960846
(Epoch 4 / 25) train acc: 0.297000; val_acc: 0.265000
(Iteration 51 / 300) loss: 1.947277
(Epoch 5 / 25) train acc: 0.299000; val_acc: 0.286000
(Iteration 61 / 300) loss: 1.903772
(Iteration 71 / 300) loss: 1.838015
(Epoch 6 / 25) train acc: 0.334000; val_acc: 0.304000
(Iteration 81 / 300) loss: 1.837380
(Epoch 7 / 25) train acc: 0.344000; val_acc: 0.294000
(Iteration 91 / 300) loss: 1.765100
(Epoch 8 / 25) train acc: 0.372000; val_acc: 0.316000
(Iteration 101 / 300) loss: 1.808500
(Epoch 9 / 25) train acc: 0.363000; val_acc: 0.309000
(Iteration 111 / 300) loss: 1.765733
(Epoch 10 / 25) train acc: 0.393000; val_acc: 0.303000
(Iteration 121 / 300) loss: 1.836957
(Iteration 131 / 300) loss: 1.720728
(Epoch 11 / 25) train acc: 0.416000; val_acc: 0.317000
(Iteration 141 / 300) loss: 1.735691
(Epoch 12 / 25) train acc: 0.398000; val_acc: 0.326000
(Iteration 151 / 300) loss: 1.779996
(Epoch 13 / 25) train acc: 0.384000; val_acc: 0.335000
(Iteration 161 / 300) loss: 1.748731
(Epoch 14 / 25) train acc: 0.375000; val_acc: 0.314000
(Iteration 171 / 300) loss: 1.632448
(Epoch 15 / 25) train acc: 0.468000; val_acc: 0.347000
(Iteration 181 / 300) loss: 1.632546
(Iteration 191 / 300) loss: 1.713144
(Epoch 16 / 25) train acc: 0.462000; val_acc: 0.341000
(Iteration 201 / 300) loss: 1.563140
(Epoch 17 / 25) train acc: 0.447000; val_acc: 0.356000
(Iteration 211 / 300) loss: 1.639268
(Epoch 18 / 25) train acc: 0.442000; val_acc: 0.348000
(Iteration 221 / 300) loss: 1.592751
(Epoch 19 / 25) train acc: 0.469000; val_acc: 0.331000
(Iteration 231 / 300) loss: 1.559653
(Epoch 20 / 25) train acc: 0.443000; val_acc: 0.356000
(Iteration 241 / 300) loss: 1.578864
(Iteration 251 / 300) loss: 1.521393
(Epoch 21 / 25) train acc: 0.441000; val_acc: 0.320000
(Iteration 261 / 300) loss: 1.556028

(Epoch 22 / 25) train acc: 0.471000; val_acc: 0.339000
(Iteration 271 / 300) loss: 1.511194
(Epoch 23 / 25) train acc: 0.451000; val_acc: 0.321000
(Iteration 281 / 300) loss: 1.573304
(Epoch 24 / 25) train acc: 0.461000; val_acc: 0.333000
(Iteration 291 / 300) loss: 1.507518
(Epoch 25 / 25) train acc: 0.433000; val_acc: 0.350000
running with sgd_momentum ; seed = 300
(Iteration 1 / 300) loss: 2.530025
(Epoch 0 / 5) train acc: 0.100000; val_acc: 0.091000
(Iteration 11 / 300) loss: 2.170274
(Iteration 21 / 300) loss: 2.046772
(Iteration 31 / 300) loss: 1.868999
(Iteration 41 / 300) loss: 1.753842
(Iteration 51 / 300) loss: 1.966552
(Epoch 1 / 5) train acc: 0.333000; val_acc: 0.310000
(Iteration 61 / 300) loss: 1.713833
(Iteration 71 / 300) loss: 1.611947
(Iteration 81 / 300) loss: 1.843096
(Iteration 91 / 300) loss: 1.893088
(Iteration 101 / 300) loss: 1.797488
(Iteration 111 / 300) loss: 1.983616
(Epoch 2 / 5) train acc: 0.385000; val_acc: 0.339000
(Iteration 121 / 300) loss: 1.720042
(Iteration 131 / 300) loss: 1.620181
(Iteration 141 / 300) loss: 1.560870
(Iteration 151 / 300) loss: 1.839286
(Iteration 161 / 300) loss: 1.566691
(Iteration 171 / 300) loss: 1.741478
(Epoch 3 / 5) train acc: 0.443000; val_acc: 0.361000
(Iteration 181 / 300) loss: 1.725085
(Iteration 191 / 300) loss: 1.530318
(Iteration 201 / 300) loss: 1.502148
(Iteration 211 / 300) loss: 1.526136
(Iteration 221 / 300) loss: 1.410825
(Iteration 231 / 300) loss: 1.466046
(Epoch 4 / 5) train acc: 0.459000; val_acc: 0.368000
(Iteration 241 / 300) loss: 1.569517
(Iteration 251 / 300) loss: 1.544676
(Iteration 261 / 300) loss: 1.476752
(Iteration 271 / 300) loss: 1.273675
(Iteration 281 / 300) loss: 1.536827
(Iteration 291 / 300) loss: 1.448791
(Epoch 5 / 5) train acc: 0.485000; val_acc: 0.374000
(Iteration 1 / 300) loss: 2.360223
(Epoch 0 / 5) train acc: 0.100000; val_acc: 0.100000
(Iteration 11 / 300) loss: 2.318004
(Iteration 21 / 300) loss: 2.106337
(Iteration 31 / 300) loss: 2.002348
(Iteration 41 / 300) loss: 1.954424
(Iteration 51 / 300) loss: 1.877097
(Epoch 1 / 5) train acc: 0.330000; val_acc: 0.344000
(Iteration 61 / 300) loss: 1.856482
(Iteration 71 / 300) loss: 1.676337

(Iteration 81 / 300) loss: 1.841227
(Iteration 91 / 300) loss: 1.686908
(Iteration 101 / 300) loss: 1.659393
(Iteration 111 / 300) loss: 1.748931
(Epoch 2 / 5) train acc: 0.421000; val_acc: 0.327000
(Iteration 121 / 300) loss: 1.616202
(Iteration 131 / 300) loss: 1.657402
(Iteration 141 / 300) loss: 1.594207
(Iteration 151 / 300) loss: 1.612730
(Iteration 161 / 300) loss: 1.724300
(Iteration 171 / 300) loss: 1.754347
(Epoch 3 / 5) train acc: 0.423000; val_acc: 0.318000
(Iteration 181 / 300) loss: 1.771232
(Iteration 191 / 300) loss: 1.394462
(Iteration 201 / 300) loss: 1.617990
(Iteration 211 / 300) loss: 1.408855
(Iteration 221 / 300) loss: 1.437453
(Iteration 231 / 300) loss: 1.403420
(Epoch 4 / 5) train acc: 0.450000; val_acc: 0.356000
(Iteration 241 / 300) loss: 1.642311
(Iteration 251 / 300) loss: 1.596960
(Iteration 261 / 300) loss: 1.435245
(Iteration 271 / 300) loss: 1.463460
(Iteration 281 / 300) loss: 1.203762
(Iteration 291 / 300) loss: 1.656364
(Epoch 5 / 5) train acc: 0.473000; val_acc: 0.367000
(Iteration 1 / 300) loss: 2.680606
(Epoch 0 / 5) train acc: 0.116000; val_acc: 0.119000
(Iteration 11 / 300) loss: 2.149887
(Iteration 21 / 300) loss: 2.115102
(Iteration 31 / 300) loss: 1.966216
(Iteration 41 / 300) loss: 1.760692
(Iteration 51 / 300) loss: 2.021934
(Epoch 1 / 5) train acc: 0.369000; val_acc: 0.288000
(Iteration 61 / 300) loss: 1.861866
(Iteration 71 / 300) loss: 1.755643
(Iteration 81 / 300) loss: 2.009469
(Iteration 91 / 300) loss: 1.634651
(Iteration 101 / 300) loss: 1.923615
(Iteration 111 / 300) loss: 1.721605
(Epoch 2 / 5) train acc: 0.455000; val_acc: 0.358000
(Iteration 121 / 300) loss: 1.696066
(Iteration 131 / 300) loss: 1.600500
(Iteration 141 / 300) loss: 1.540929
(Iteration 151 / 300) loss: 1.562100
(Iteration 161 / 300) loss: 1.588500
(Iteration 171 / 300) loss: 1.433436
(Epoch 3 / 5) train acc: 0.481000; val_acc: 0.364000
(Iteration 181 / 300) loss: 1.552393
(Iteration 191 / 300) loss: 1.564595
(Iteration 201 / 300) loss: 1.539341
(Iteration 211 / 300) loss: 1.611809
(Iteration 221 / 300) loss: 1.543027
(Iteration 231 / 300) loss: 1.479987

(Epoch 4 / 5) train acc: 0.456000; val_acc: 0.368000
(Iteration 241 / 300) loss: 1.423035
(Iteration 251 / 300) loss: 1.260210
(Iteration 261 / 300) loss: 1.556388
(Iteration 271 / 300) loss: 1.332439
(Iteration 281 / 300) loss: 1.438555
(Iteration 291 / 300) loss: 1.384180
(Epoch 5 / 5) train acc: 0.499000; val_acc: 0.369000
Average Training Acc for sgd: 0.4386666666666667
Average Training Acc for sgd_momentum: 0.48566666666666664
Train Acc Difference: 0.05084745762711857



3 - RMSProp (Optional)

As we saw with the "Implicit Regularization" example in lecture, using one fixed learning rate will cause the optimizer to converge more slowly in some parameters than in others. This may not be ideal since we need to set the learning rate small so that parameters that change quickly don't diverge, but parameters that change more slowly may need larger learning rates to converge quickly. RMSProp is an **adaptive** algorithm, which adapts the learning rate of each parameter individually in order to help all parameters converge at similar speed to each other.

RMSProp works by keeping an exponentially weighted moving average of the squared partial derivatives of the loss with respect each parameter, which serves as an estimate to how quickly that parameter is changing. RMSProp then scales the gradient of each component by the square root of this moving average, such that the gradients with respect to each parameter are (approximately) equal.

The RMSProp update rule is as follows:

$$\begin{cases} r = \rho r + (1 - \rho) (\frac{\partial \mathcal{J}}{\partial W})^2 \\ w = w - \alpha \frac{1}{\sqrt{r} + \epsilon} \frac{\partial \mathcal{J}}{\partial W} \end{cases} \text{ where:}$$

- ρ is the "decay rate" that controls the exponentially weighted average.
- α is the learning rate.
- ϵ is a very small number to avoid dividing by zero.

In the file `deeplearning/optim.py`, implement the RMSProp update rule in the `rmsprop` function, and check your implementations using the tests below.

[1] Tijmen Tieleman and Geoffrey Hinton. "Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude." COURSE: Neural Networks for Machine Learning 4 (2012).

```
In [7]: # Test RMSProp implementation; you should see errors less than 1e-7.
from deeplearning.optim import rmsprop

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
cache = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'cache': cache}
next_w, _ = rmsprop(w, dw, config=config)
```

```

expected_next_w = np.asarray([
    [-0.39223849, -0.34037513, -0.28849239, -0.23659121, -0.18467247],
    [-0.132737, -0.08078555, -0.02881884, 0.02316247, 0.07515774],
    [ 0.12716641, 0.17918792, 0.23122175, 0.28326742, 0.33532447],
    [ 0.38739248, 0.43947102, 0.49155973, 0.54365823, 0.59576619]])
expected_cache = np.asarray([
    [ 0.5976, 0.6126277, 0.6277108, 0.64284931, 0.65804321],
    [ 0.67329252, 0.68859723, 0.70395734, 0.71937285, 0.73484377],
    [ 0.75037008, 0.7659518, 0.78158892, 0.79728144, 0.81302936],
    [ 0.82883269, 0.84469141, 0.86060554, 0.87657507, 0.8926 ]])

print ('next_w error: ', rel_error(expected_next_w, next_w))
print ('cache error: ', rel_error(expected_cache, config['cache']))

```

```

next_w error: 9.524687511038133e-08
cache error: 2.6477955807156126e-09

```

4 - Adam

Adam is one of the most effective optimization algorithms for training neural networks. It combines ideas from RMSProp and Momentum.

How does Adam work?

1. It calculates an exponentially weighted average of past gradients, and stores it in variables m (before bias correction) and $m^{\text{corrected}}$ (with bias correction).
2. It calculates an exponentially weighted average of the squares of the past gradients, and stores it in variables v (before bias correction) and $v^{\text{corrected}}$ (with bias correction).
3. It updates parameters in a direction based on combining information from both $m^{\text{corrected}}$ and $v^{\text{corrected}}$.

$$m_{dw} = \beta_1 m_{dw} + (1 - \beta_1) \frac{\partial}{\partial W}$$

$$m^{\text{corrected}}_{dw} = \frac{m_{dw}}{1 - (\beta_1)^t}$$

$$v_{dw} = \beta_2 v_{dw} + (1 - \beta_2) \left(\frac{\partial}{\partial W} \right)^2$$

$$v^{\text{corrected}}_{dw} = \frac{v_{dw}}{1 - (\beta_2)^t}$$

$$w = w - \alpha \frac{m^{\text{corrected}}_{dw}}{\sqrt{v^{\text{corrected}}_{dw}}} + \epsilon$$
 where:

- t counts the number of steps taken of Adam
- β_1 and β_2 are hyperparameters that control the two exponentially weighted averages
- α is the learning rate
- ϵ is a very small number to avoid dividing by zero

In the file `deeplearning/optim.py`, implement the Adam update rule in the `adam` function, and check your implementations using the tests below.

[1] Diederik Kingma and Jimmy Ba, "Adam: A Method for Stochastic Optimization", ICLR 2015.

In [8]: *# Test Adam implementation; you should see errors around 1e-7 or less.*

```
from deeplearning.optim import adam

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
m = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)
v = np.linspace(0.7, 0.5, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'm': m, 'v': v, 't': 5}
next_w, _ = adam(w, dw, config=config)

expected_next_w = np.asarray([
    [-0.40094747, -0.34836187, -0.29577703, -0.24319299, -0.19060977],
    [-0.1380274, -0.08544591, -0.03286534, 0.01971428, 0.0722929],
    [ 0.1248705,  0.17744702,  0.23002243,  0.28259667,  0.33516969],
    [ 0.38774145,  0.44031188,  0.49288093,  0.54544852,  0.59801459]])
expected_v = np.asarray([
    [ 0.69966,    0.68908382,  0.67851319,  0.66794809,  0.65738853],
    [ 0.64683452,  0.63628604,  0.6257431,   0.61520571,  0.60467385],
    [ 0.59414753,  0.58362676,  0.57311152,  0.56260183,  0.55209767],
    [ 0.54159906,  0.53110598,  0.52061845,  0.51013645,  0.49966   ]])
expected_m = np.asarray([
    [ 0.48,    0.49947368,  0.51894737,  0.53842105,  0.55789474],
    [ 0.57736842,  0.59684211,  0.61631579,  0.63578947,  0.65526316],
    [ 0.67473684,  0.69421053,  0.71368421,  0.73315789,  0.75263158],
    [ 0.77210526,  0.79157895,  0.81105263,  0.83052632,  0.85   ]])
expected_t = 6

print ('next_w error: ', rel_error(expected_next_w, next_w))
print ('v error: ', rel_error(expected_v, config['v']))
print ('m error: ', rel_error(expected_m, config['m']))
print ('t error: ', rel_error(expected_t, config['t']))
```

```
next_w error:  1.1395691798535431e-07
v error:  4.208314038113071e-09
m error:  4.214963193114416e-09
t error:  0.0
```

Once you have debugged your RMSProp and Adam implementations, run the following to train a pair of deep networks using these new update rules. As a sanity check, you should see that RMSProp and Adam typically obtain at least 45% training accuracy within 5 epochs.

In [9]: `num_train = 4000`
`small_data = {`

```

'X_train': data['X_train'][:num_train],
'y_train': data['y_train'][:num_train],
'X_val': data['X_val'],
'y_val': data['y_val'],
}

learning_rates = {'rmsprop': 1e-4, 'adam': 1e-3, 'sgd': 1e-2, 'sgd_momentum':
for update_rule in ['sgd', 'sgd_momentum', 'adam', 'rmsprop']:
    print ('running with ', update_rule)

    torch.manual_seed(0)
    np.random.seed(0)

    model = FullyConnectedNet([100, 100, 100, 100, 100], weight_scale=5e-2)

    solver = Solver(model, small_data,
                    num_epochs=5, batch_size=100,
                    update_rule=update_rule,
                    optim_config={
                        'learning_rate': learning_rates[update_rule]
                    },
                    verbose=True,)
    solvers[update_rule] = solver
    solver.train()
    solver.record_histories_as_npz("submission_logs/optimizer_experiment_{}".f
    print

plt.subplot(3, 1, 1)
plt.title('Training loss')
plt.xlabel('Iteration')

plt.subplot(3, 1, 2)
plt.title('Training accuracy')
plt.xlabel('Epoch')

plt.subplot(3, 1, 3)
plt.title('Validation accuracy')
plt.xlabel('Epoch')

for update_rule, solver in solvers.items():
    plt.subplot(3, 1, 1)
    plt.plot(solver.loss_history, label=update_rule)

    plt.subplot(3, 1, 2)
    plt.plot(solver.train_acc_history, '-o', label=update_rule)

    plt.subplot(3, 1, 3)
    plt.plot(solver.val_acc_history, '-o', label=update_rule)

for i in [1, 2, 3]:
    plt.subplot(3, 1, i)
    plt.legend(loc='upper center', ncol=4)
plt.gcf().set_size_inches(15, 15)

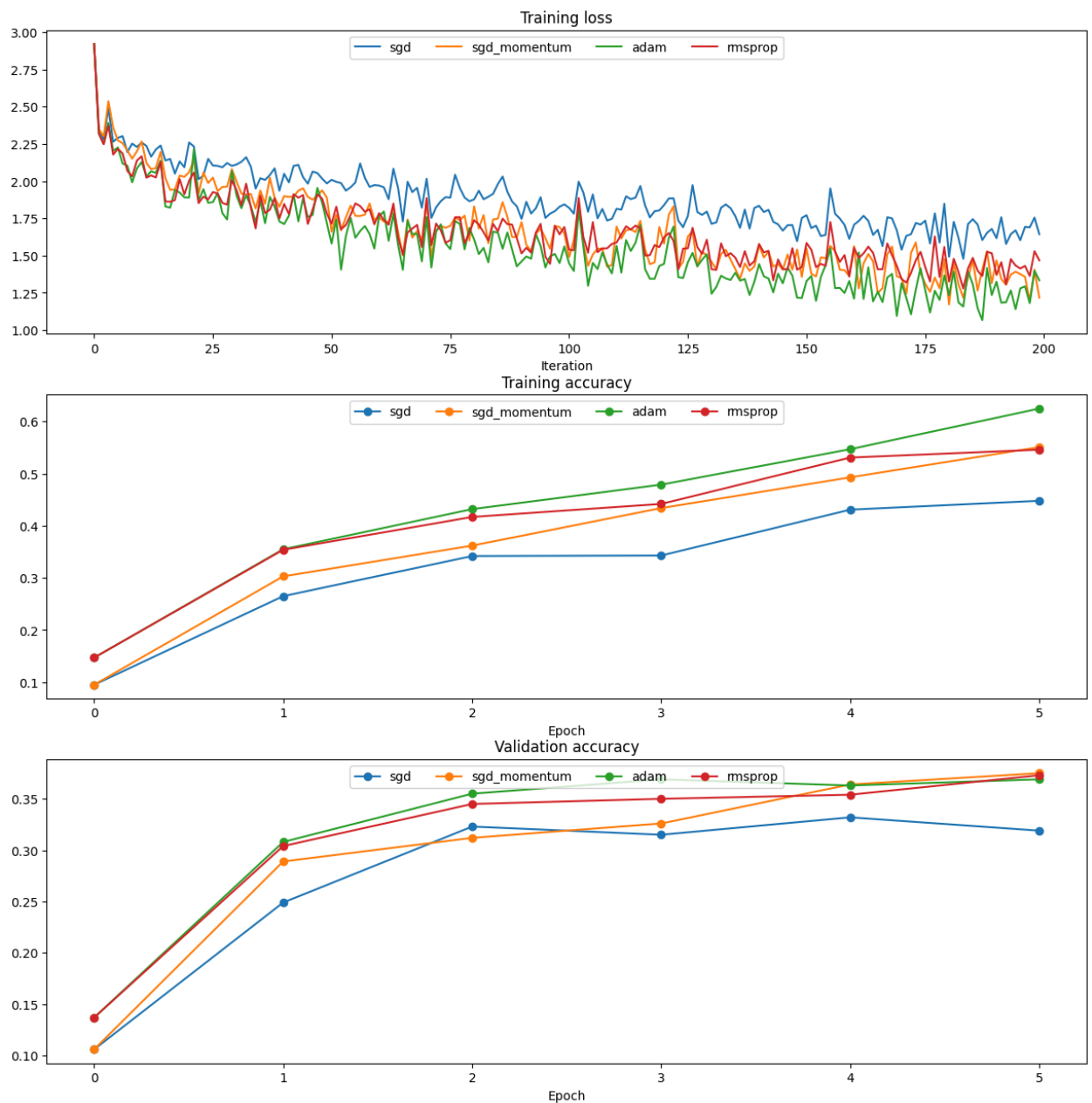
```



```
plt.show()
```

```
running with  sgd
(Iteration 1 / 200) loss: 2.920601
(Epoch 0 / 5) train acc: 0.095000; val_acc: 0.106000
(Iteration 11 / 200) loss: 2.259420
(Iteration 21 / 200) loss: 2.259472
(Iteration 31 / 200) loss: 2.111666
(Epoch 1 / 5) train acc: 0.265000; val_acc: 0.249000
(Iteration 41 / 200) loss: 2.049079
(Iteration 51 / 200) loss: 2.008793
(Iteration 61 / 200) loss: 1.969929
(Iteration 71 / 200) loss: 2.015482
(Epoch 2 / 5) train acc: 0.342000; val_acc: 0.323000
(Iteration 81 / 200) loss: 1.877204
(Iteration 91 / 200) loss: 1.746421
(Iteration 101 / 200) loss: 1.820583
(Iteration 111 / 200) loss: 1.814485
(Epoch 3 / 5) train acc: 0.343000; val_acc: 0.315000
(Iteration 121 / 200) loss: 1.839073
(Iteration 131 / 200) loss: 1.712923
(Iteration 141 / 200) loss: 1.834413
(Iteration 151 / 200) loss: 1.771377
(Epoch 4 / 5) train acc: 0.431000; val_acc: 0.332000
(Iteration 161 / 200) loss: 1.707295
(Iteration 171 / 200) loss: 1.539645
(Iteration 181 / 200) loss: 1.492332
(Iteration 191 / 200) loss: 1.616897
(Epoch 5 / 5) train acc: 0.448000; val_acc: 0.319000
running with  sgd_momentum
(Iteration 1 / 200) loss: 2.920601
(Epoch 0 / 5) train acc: 0.095000; val_acc: 0.106000
(Iteration 11 / 200) loss: 2.265393
(Iteration 21 / 200) loss: 2.059928
(Iteration 31 / 200) loss: 1.995965
(Epoch 1 / 5) train acc: 0.303000; val_acc: 0.289000
(Iteration 41 / 200) loss: 1.899152
(Iteration 51 / 200) loss: 1.659711
(Iteration 61 / 200) loss: 1.756672
(Iteration 71 / 200) loss: 1.873106
(Epoch 2 / 5) train acc: 0.362000; val_acc: 0.312000
(Iteration 81 / 200) loss: 1.829993
(Iteration 91 / 200) loss: 1.724056
(Iteration 101 / 200) loss: 1.489399
(Iteration 111 / 200) loss: 1.695339
(Epoch 3 / 5) train acc: 0.434000; val_acc: 0.326000
(Iteration 121 / 200) loss: 1.580010
(Iteration 131 / 200) loss: 1.457715
(Iteration 141 / 200) loss: 1.570430
(Iteration 151 / 200) loss: 1.548434
(Epoch 4 / 5) train acc: 0.493000; val_acc: 0.364000
(Iteration 161 / 200) loss: 1.563106
(Iteration 171 / 200) loss: 1.350203
(Iteration 181 / 200) loss: 1.172008
(Iteration 191 / 200) loss: 1.467957
(Epoch 5 / 5) train acc: 0.551000; val_acc: 0.375000
```

```
running with adam
(Iteration 1 / 200) loss: 2.920601
(Epoch 0 / 5) train acc: 0.147000; val_acc: 0.137000
(Iteration 11 / 200) loss: 2.128094
(Iteration 21 / 200) loss: 1.889504
(Iteration 31 / 200) loss: 1.901563
(Epoch 1 / 5) train acc: 0.355000; val_acc: 0.308000
(Iteration 41 / 200) loss: 1.711346
(Iteration 51 / 200) loss: 1.580716
(Iteration 61 / 200) loss: 1.762455
(Iteration 71 / 200) loss: 1.758541
(Epoch 2 / 5) train acc: 0.432000; val_acc: 0.355000
(Iteration 81 / 200) loss: 1.608817
(Iteration 91 / 200) loss: 1.461546
(Iteration 101 / 200) loss: 1.447599
(Iteration 111 / 200) loss: 1.565091
(Epoch 3 / 5) train acc: 0.479000; val_acc: 0.369000
(Iteration 121 / 200) loss: 1.444845
(Iteration 131 / 200) loss: 1.243555
(Iteration 141 / 200) loss: 1.443196
(Iteration 151 / 200) loss: 1.329008
(Epoch 4 / 5) train acc: 0.547000; val_acc: 0.363000
(Iteration 161 / 200) loss: 1.210604
(Iteration 171 / 200) loss: 1.314864
(Iteration 181 / 200) loss: 1.234458
(Iteration 191 / 200) loss: 1.324541
(Epoch 5 / 5) train acc: 0.625000; val_acc: 0.369000
running with rmsprop
(Iteration 1 / 200) loss: 2.920601
(Epoch 0 / 5) train acc: 0.147000; val_acc: 0.137000
(Iteration 11 / 200) loss: 2.166484
(Iteration 21 / 200) loss: 2.002731
(Iteration 31 / 200) loss: 1.937274
(Epoch 1 / 5) train acc: 0.354000; val_acc: 0.304000
(Iteration 41 / 200) loss: 1.846939
(Iteration 51 / 200) loss: 1.714023
(Iteration 61 / 200) loss: 1.785220
(Iteration 71 / 200) loss: 1.885689
(Epoch 2 / 5) train acc: 0.417000; val_acc: 0.345000
(Iteration 81 / 200) loss: 1.737758
(Iteration 91 / 200) loss: 1.514806
(Iteration 101 / 200) loss: 1.539760
(Iteration 111 / 200) loss: 1.587685
(Epoch 3 / 5) train acc: 0.442000; val_acc: 0.350000
(Iteration 121 / 200) loss: 1.622331
(Iteration 131 / 200) loss: 1.408595
(Iteration 141 / 200) loss: 1.578290
(Iteration 151 / 200) loss: 1.584815
(Epoch 4 / 5) train acc: 0.531000; val_acc: 0.354000
(Iteration 161 / 200) loss: 1.552534
(Iteration 171 / 200) loss: 1.342006
(Iteration 181 / 200) loss: 1.325363
(Iteration 191 / 200) loss: 1.370476
(Epoch 5 / 5) train acc: 0.546000; val_acc: 0.373000
```



Initialization

Training your neural network requires specifying an initial value of the weights. A well chosen initialization method will help learning.

A well chosen initialization can:

- Speed up the convergence of gradient descent
- Increase the odds of gradient descent converging to a lower training (and generalization) error

We will use three different initialization methods to illustrate this concept.

- Zero Initialization:

This initializes the weights to 0.

- Random Initialization:

This initializes the weights drawn from a distribution with *manually* specified scales. In this homework, **we use normal distribution with the `weight_scale` argument in `fc_net.py` as its std.**

- He/Xavier/Glorot Initialization:

This is a special case for random initialization, where the scaling factor is set so that the std of each parameter is `gain / sqrt(fan_mode)`. `gain` is determined by the activation function. For example, linear activation has `gain = 1` and ReLU activation has `gain = sqrt(2)`. There are three types of fan mode:

- Fan in: `fan_mode = in_dim`, i.e., the width of the preceding layer, preserving the magnitude in forward pass. **This is what you need to implement below** and also the default in PyTorch.
- Fan out: `fan_mode = out_dim`, i.e., the width of the succeeding layer, preserving the magnitude in backpropagation.
- Average: `fan_mode = (in_dim + out_dim) / 2`.

When the std is determined, another choice is between normal distribution or uniform distribution. In this homework **we use normal distribution for initialization.**

```
In [10]: #####
# TODO:
num_epochs = 15
batch_size = 100
lr = 1e-3
lr_decay = 0.95
# 1. implement the last two initialization schemes in
#    deeplearning/classifiers/fc_net.py
# 2. record the mean of l2 norm of the gradients
#    in the deeplearning/solver.py
#####

learning_rates = {'sgd': 1e-3}
update_rule = 'sgd'
solvers = dict()
```

```

num_train = 4000
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}

for initialization in ['he', 'random', 'zero']:
    print ('running with ', update_rule)

    model = FullyConnectedNet([50]*10, initialization=initialization)
    weight_stds = [float(model.params["W" + str(i)].std()) for i in range(1, 10)]
    print("initialization scheme:", initialization)
    if initialization == "he":
        # It is fine if the rel_error is less than 0.03 due to randomness
        print("Layer 1, rel_error", rel_error(0.02551551815399, weight_stds[0]))
        print("Layer 2, rel_error", rel_error(0.2, weight_stds[1]))
    elif initialization == "random":
        # It is fine if the rel_error is less than 0.03 due to randomness
        print("Layer 1, rel_error", rel_error(0.01, weight_stds[0]))
        print("Layer 2, rel_error", rel_error(0.01, weight_stds[1]))
    with open("submission_logs/w_stds_{}.json".format(initialization), "w", encoding='utf-8') as f:
        json.dump(weight_stds, f)

    solver = Solver(model, small_data,
                    num_epochs=5, batch_size=100,
                    update_rule=update_rule,
                    optim_config={
                        'learning_rate': learning_rates[update_rule]
                    },
                    verbose=True)
    solvers[initialization] = solver
    solver.train()
    solver.record_histories_as_npz("submission_logs/initialization_experiment_{}.npz".format(initialization))
    print("{}: {}".format(initialization, solver.loss))

plt.subplot(4, 1, 1)
plt.title('Training loss')
plt.xlabel('Iteration')

plt.subplot(4, 1, 2)
plt.title('Training accuracy')
plt.xlabel('Epoch')

plt.subplot(4, 1, 3)
plt.title('Validation accuracy')
plt.xlabel('Epoch')

plt.subplot(4, 1, 4)
plt.title('Mean of the Gradient Norm')

```

```
plt.xlabel('Iteration')

for initialization, solver in solvers.items():
    plt.subplot(4, 1, 1)
    plt.plot(solver.loss_history, label=initialization)

    plt.subplot(4, 1, 2)
    plt.plot(solver.train_acc_history, '-o', label=initialization)

    plt.subplot(4, 1, 3)
    plt.plot(solver.val_acc_history, '-o', label=initialization)

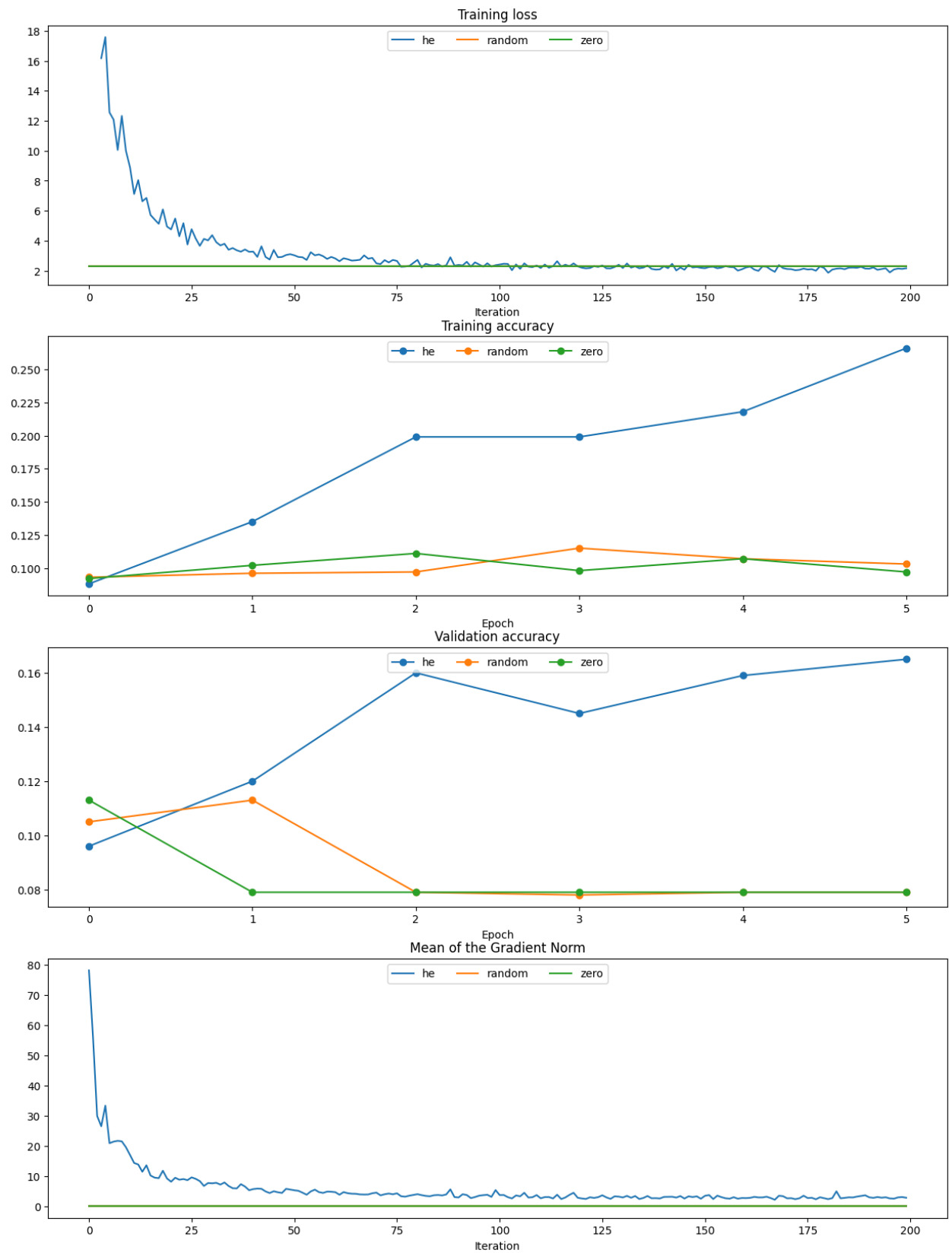
    plt.subplot(4, 1, 4)
    plt.plot(solver.log_grad_norm_history, label=initialization)

for i in [1, 2, 3, 4]:
    plt.subplot(4, 1, i)
    plt.legend(loc='upper center', ncol=4)
plt.gcf().set_size_inches(15, 20)

plt.show()
```

```
running with  sgd
initialization scheme: he
Layer 1, rel_error 1.903445941569443e-05
Layer 2, rel_error 0.001928263065996156
(Iteration 1 / 200) loss: inf
(Epoch 0 / 5) train acc: 0.088000; val_acc: 0.096000
(Iteration 11 / 200) loss: 8.892769
(Iteration 21 / 200) loss: 4.755562
(Iteration 31 / 200) loss: 4.370598
(Epoch 1 / 5) train acc: 0.135000; val_acc: 0.120000
(Iteration 41 / 200) loss: 3.277878
(Iteration 51 / 200) loss: 3.031024
(Iteration 61 / 200) loss: 2.815829
(Iteration 71 / 200) loss: 2.491996
(Epoch 2 / 5) train acc: 0.199000; val_acc: 0.160000
(Iteration 81 / 200) loss: 2.726078
(Iteration 91 / 200) loss: 2.389881
(Iteration 101 / 200) loss: 2.414062
(Iteration 111 / 200) loss: 2.183079
(Epoch 3 / 5) train acc: 0.199000; val_acc: 0.145000
(Iteration 121 / 200) loss: 2.200212
(Iteration 131 / 200) loss: 2.181910
(Iteration 141 / 200) loss: 2.298311
(Iteration 151 / 200) loss: 2.166447
(Epoch 4 / 5) train acc: 0.218000; val_acc: 0.159000
(Iteration 161 / 200) loss: 2.218004
(Iteration 171 / 200) loss: 2.114251
(Iteration 181 / 200) loss: 1.863833
(Iteration 191 / 200) loss: 2.132788
(Epoch 5 / 5) train acc: 0.266000; val_acc: 0.165000
running with  sgd
initialization scheme: random
Layer 1, rel_error 0.00016643502911157485
Layer 2, rel_error 0.004246082830986831
(Iteration 1 / 200) loss: 2.302585
(Epoch 0 / 5) train acc: 0.093000; val_acc: 0.105000
(Iteration 11 / 200) loss: 2.302581
(Iteration 21 / 200) loss: 2.302574
(Iteration 31 / 200) loss: 2.302552
(Epoch 1 / 5) train acc: 0.096000; val_acc: 0.113000
(Iteration 41 / 200) loss: 2.302572
(Iteration 51 / 200) loss: 2.302567
(Iteration 61 / 200) loss: 2.302600
(Iteration 71 / 200) loss: 2.302548
(Epoch 2 / 5) train acc: 0.097000; val_acc: 0.079000
(Iteration 81 / 200) loss: 2.302480
(Iteration 91 / 200) loss: 2.302642
(Iteration 101 / 200) loss: 2.302568
(Iteration 111 / 200) loss: 2.302578
(Epoch 3 / 5) train acc: 0.115000; val_acc: 0.078000
(Iteration 121 / 200) loss: 2.302731
(Iteration 131 / 200) loss: 2.302442
(Iteration 141 / 200) loss: 2.302414
(Iteration 151 / 200) loss: 2.302506
```


(Epoch 4 / 5) train acc: 0.107000; val_acc: 0.079000
(Iteration 161 / 200) loss: 2.302597
(Iteration 171 / 200) loss: 2.302611
(Iteration 181 / 200) loss: 2.302584
(Iteration 191 / 200) loss: 2.302443
(Epoch 5 / 5) train acc: 0.103000; val_acc: 0.079000
running with sgd
initialization scheme: zero
(Iteration 1 / 200) loss: 2.302585
(Epoch 0 / 5) train acc: 0.092000; val_acc: 0.113000
(Iteration 11 / 200) loss: 2.302593
(Iteration 21 / 200) loss: 2.302588
(Iteration 31 / 200) loss: 2.302596
(Epoch 1 / 5) train acc: 0.102000; val_acc: 0.079000
(Iteration 41 / 200) loss: 2.302576
(Iteration 51 / 200) loss: 2.302601
(Iteration 61 / 200) loss: 2.302580
(Iteration 71 / 200) loss: 2.302578
(Epoch 2 / 5) train acc: 0.111000; val_acc: 0.079000
(Iteration 81 / 200) loss: 2.302585
(Iteration 91 / 200) loss: 2.302554
(Iteration 101 / 200) loss: 2.302543
(Iteration 111 / 200) loss: 2.302541
(Epoch 3 / 5) train acc: 0.098000; val_acc: 0.079000
(Iteration 121 / 200) loss: 2.302546
(Iteration 131 / 200) loss: 2.302615
(Iteration 141 / 200) loss: 2.302549
(Iteration 151 / 200) loss: 2.302553
(Epoch 4 / 5) train acc: 0.107000; val_acc: 0.079000
(Iteration 161 / 200) loss: 2.302677
(Iteration 171 / 200) loss: 2.302487
(Iteration 181 / 200) loss: 2.302551
(Iteration 191 / 200) loss: 2.302562
(Epoch 5 / 5) train acc: 0.097000; val_acc: 0.079000



Question:

What you observe in the mean of gradient norm plot above in the above

plots? Try to give an explanation. **Write your answer on the written assignment.**

Zero initialization: Gradients vanish quickly → gradient norm drops to near zero → no learning. Random (small scale): Gradients may vanish or explode depending on scale; often suboptimal. He initialization: Gradient norms stay stable across layers and iterations → enables effective backpropagation and faster convergence. This is because He initialization preserves the variance of activations and gradients in networks with ReLU, avoiding vanishing/exploding gradients.

Train a good model!

Train the best fully-connected model that you can on CIFAR-10, storing your best model in the `best_model` variable and the solver used in the `best_solver` variable. We require you to get at least 45% accuracy *on the validation set* using a fully-connected net.

If you are careful it should be possible to get accuracies above 55%, but we don't require it for this part and won't assign extra credit for doing so. Later in the assignment we will ask you to train the best convolutional network that you can on CIFAR-10, and we would prefer that you spend your effort working on convolutional nets rather than fully-connected nets.

```
In [11]: best_model = None
best_solver = None

width = 200 # please don't change this
n_layers = 10 # please don't change this

#####
# TODO: Train the best FullyConnectedNet that you can on CIFAR-10.
# Store your best model in the best_model variable
# and the solver used to train it in the best_solver variable
# Please use the He Initialization and adam.
# You could tune the following variables only below,
# it should achieve above 45% accuracy on the validation set.
#####
pass # TODO
#####
#                                     END OF YOUR CODE
#####

np.random.seed(2023) # please don't change this for reproducibility
torch.manual_seed(2023) # please don't change this for reproducibility
model = FullyConnectedNet([width] * n_layers,
```

```

                                initialization='he'
                                )
solver = Solver(model,
                data,
                num_epochs=num_epochs,
                batch_size=batch_size,
                update_rule=update_rule,
                optim_config={
                    'learning_rate': lr
                },
                lr_decay=lr_decay,
                verbose=True)
solver.train()
best_model = model
best_solver = solver

```

C:\Users\NLP\Desktop\homework\DL\Qwen4HW2\deeplearning\layers.py:153: RuntimeWarning: divide by zero encountered in log
 loss = -np.sum(np.log(probs[np.arange(N), y])) / N

```
(Iteration 1 / 7350) loss: inf
(Epoch 0 / 15) train acc: 0.110000; val_acc: 0.141000
(Iteration 11 / 7350) loss: 10.253752
(Iteration 21 / 7350) loss: 5.293920
(Iteration 31 / 7350) loss: 5.253567
(Iteration 41 / 7350) loss: 4.375992
(Iteration 51 / 7350) loss: 4.060196
(Iteration 61 / 7350) loss: 3.069574
(Iteration 71 / 7350) loss: 3.294896
(Iteration 81 / 7350) loss: 2.715113
(Iteration 91 / 7350) loss: 3.011237
(Iteration 101 / 7350) loss: 2.718027
(Iteration 111 / 7350) loss: 2.693106
(Iteration 121 / 7350) loss: 2.533947
(Iteration 131 / 7350) loss: 2.504399
(Iteration 141 / 7350) loss: 2.665716
(Iteration 151 / 7350) loss: 2.374033
(Iteration 161 / 7350) loss: 2.330561
(Iteration 171 / 7350) loss: 2.448994
(Iteration 181 / 7350) loss: 2.386706
(Iteration 191 / 7350) loss: 2.270184
(Iteration 201 / 7350) loss: 2.347964
(Iteration 211 / 7350) loss: 2.376978
(Iteration 221 / 7350) loss: 2.264696
(Iteration 231 / 7350) loss: 2.227189
(Iteration 241 / 7350) loss: 2.428056
(Iteration 251 / 7350) loss: 2.230962
(Iteration 261 / 7350) loss: 2.102770
(Iteration 271 / 7350) loss: 1.995658
(Iteration 281 / 7350) loss: 2.188453
(Iteration 291 / 7350) loss: 2.120989
(Iteration 301 / 7350) loss: 2.183363
(Iteration 311 / 7350) loss: 2.275199
(Iteration 321 / 7350) loss: 2.173472
(Iteration 331 / 7350) loss: 2.294279
(Iteration 341 / 7350) loss: 1.953754
(Iteration 351 / 7350) loss: 2.156109
(Iteration 361 / 7350) loss: 2.106407
(Iteration 371 / 7350) loss: 1.982086
(Iteration 381 / 7350) loss: 1.984692
(Iteration 391 / 7350) loss: 2.085773
(Iteration 401 / 7350) loss: 2.147653
(Iteration 411 / 7350) loss: 2.100070
(Iteration 421 / 7350) loss: 2.244721
(Iteration 431 / 7350) loss: 2.140698
(Iteration 441 / 7350) loss: 2.191444
(Iteration 451 / 7350) loss: 1.821623
(Iteration 461 / 7350) loss: 2.151850
(Iteration 471 / 7350) loss: 2.077577
(Iteration 481 / 7350) loss: 2.141273
(Epoch 1 / 15) train acc: 0.247000; val_acc: 0.242000
(Iteration 491 / 7350) loss: 2.024962
(Iteration 501 / 7350) loss: 2.186350
(Iteration 511 / 7350) loss: 1.904109
```

(Iteration 521 / 7350) loss: 2.253432
(Iteration 531 / 7350) loss: 2.202360
(Iteration 541 / 7350) loss: 1.934614
(Iteration 551 / 7350) loss: 1.957478
(Iteration 561 / 7350) loss: 2.047019
(Iteration 571 / 7350) loss: 2.117706
(Iteration 581 / 7350) loss: 2.035733
(Iteration 591 / 7350) loss: 2.152368
(Iteration 601 / 7350) loss: 2.013094
(Iteration 611 / 7350) loss: 1.966051
(Iteration 621 / 7350) loss: 2.055068
(Iteration 631 / 7350) loss: 2.042627
(Iteration 641 / 7350) loss: 1.978569
(Iteration 651 / 7350) loss: 2.161112
(Iteration 661 / 7350) loss: 2.148979
(Iteration 671 / 7350) loss: 1.958866
(Iteration 681 / 7350) loss: 2.036369
(Iteration 691 / 7350) loss: 2.111672
(Iteration 701 / 7350) loss: 2.045877
(Iteration 711 / 7350) loss: 1.976911
(Iteration 721 / 7350) loss: 2.053420
(Iteration 731 / 7350) loss: 2.036753
(Iteration 741 / 7350) loss: 2.042908
(Iteration 751 / 7350) loss: 1.960068
(Iteration 761 / 7350) loss: 1.815581
(Iteration 771 / 7350) loss: 1.983831
(Iteration 781 / 7350) loss: 1.919695
(Iteration 791 / 7350) loss: 1.947036
(Iteration 801 / 7350) loss: 2.012319
(Iteration 811 / 7350) loss: 2.183578
(Iteration 821 / 7350) loss: 2.111491
(Iteration 831 / 7350) loss: 1.817918
(Iteration 841 / 7350) loss: 2.035508
(Iteration 851 / 7350) loss: 1.930783
(Iteration 861 / 7350) loss: 1.865972
(Iteration 871 / 7350) loss: 2.000549
(Iteration 881 / 7350) loss: 1.886447
(Iteration 891 / 7350) loss: 2.041518
(Iteration 901 / 7350) loss: 2.024795
(Iteration 911 / 7350) loss: 1.860164
(Iteration 921 / 7350) loss: 1.839698
(Iteration 931 / 7350) loss: 2.102753
(Iteration 941 / 7350) loss: 1.996007
(Iteration 951 / 7350) loss: 1.916295
(Iteration 961 / 7350) loss: 2.074734
(Iteration 971 / 7350) loss: 1.984980
(Epoch 2 / 15) train acc: 0.304000; val_acc: 0.270000
(Iteration 981 / 7350) loss: 2.024352
(Iteration 991 / 7350) loss: 1.885286
(Iteration 1001 / 7350) loss: 1.862939
(Iteration 1011 / 7350) loss: 1.931579
(Iteration 1021 / 7350) loss: 1.946861
(Iteration 1031 / 7350) loss: 1.834945
(Iteration 1041 / 7350) loss: 1.847509

(Iteration 1051 / 7350) loss: 1.929489
(Iteration 1061 / 7350) loss: 1.883307
(Iteration 1071 / 7350) loss: 1.891236
(Iteration 1081 / 7350) loss: 1.860436
(Iteration 1091 / 7350) loss: 1.782051
(Iteration 1101 / 7350) loss: 1.965734
(Iteration 1111 / 7350) loss: 1.795438
(Iteration 1121 / 7350) loss: 1.803468
(Iteration 1131 / 7350) loss: 2.098014
(Iteration 1141 / 7350) loss: 1.832673
(Iteration 1151 / 7350) loss: 1.729952
(Iteration 1161 / 7350) loss: 1.761833
(Iteration 1171 / 7350) loss: 1.824499
(Iteration 1181 / 7350) loss: 2.000541
(Iteration 1191 / 7350) loss: 1.943167
(Iteration 1201 / 7350) loss: 1.870388
(Iteration 1211 / 7350) loss: 1.811310
(Iteration 1221 / 7350) loss: 1.941578
(Iteration 1231 / 7350) loss: 1.780726
(Iteration 1241 / 7350) loss: 2.008674
(Iteration 1251 / 7350) loss: 1.968501
(Iteration 1261 / 7350) loss: 1.822184
(Iteration 1271 / 7350) loss: 1.731272
(Iteration 1281 / 7350) loss: 1.801391
(Iteration 1291 / 7350) loss: 1.891346
(Iteration 1301 / 7350) loss: 1.887134
(Iteration 1311 / 7350) loss: 2.003284
(Iteration 1321 / 7350) loss: 1.836089
(Iteration 1331 / 7350) loss: 1.984785
(Iteration 1341 / 7350) loss: 1.826028
(Iteration 1351 / 7350) loss: 1.717167
(Iteration 1361 / 7350) loss: 1.862341
(Iteration 1371 / 7350) loss: 1.869890
(Iteration 1381 / 7350) loss: 1.898455
(Iteration 1391 / 7350) loss: 1.798099
(Iteration 1401 / 7350) loss: 1.938001
(Iteration 1411 / 7350) loss: 1.864457
(Iteration 1421 / 7350) loss: 1.843293
(Iteration 1431 / 7350) loss: 1.877556
(Iteration 1441 / 7350) loss: 1.747849
(Iteration 1451 / 7350) loss: 1.910857
(Iteration 1461 / 7350) loss: 2.038835
(Epoch 3 / 15) train acc: 0.351000; val_acc: 0.330000
(Iteration 1471 / 7350) loss: 1.853380
(Iteration 1481 / 7350) loss: 1.823502
(Iteration 1491 / 7350) loss: 1.886596
(Iteration 1501 / 7350) loss: 1.735948
(Iteration 1511 / 7350) loss: 1.735311
(Iteration 1521 / 7350) loss: 1.836874
(Iteration 1531 / 7350) loss: 1.883438
(Iteration 1541 / 7350) loss: 1.752184
(Iteration 1551 / 7350) loss: 1.915800
(Iteration 1561 / 7350) loss: 1.752196
(Iteration 1571 / 7350) loss: 1.915965

(Iteration 1581 / 7350) loss: 1.809046
(Iteration 1591 / 7350) loss: 1.877393
(Iteration 1601 / 7350) loss: 1.962269
(Iteration 1611 / 7350) loss: 1.769499
(Iteration 1621 / 7350) loss: 1.756540
(Iteration 1631 / 7350) loss: 1.760058
(Iteration 1641 / 7350) loss: 1.795139
(Iteration 1651 / 7350) loss: 1.595430
(Iteration 1661 / 7350) loss: 1.808421
(Iteration 1671 / 7350) loss: 1.747934
(Iteration 1681 / 7350) loss: 1.853193
(Iteration 1691 / 7350) loss: 1.726883
(Iteration 1701 / 7350) loss: 1.856720
(Iteration 1711 / 7350) loss: 1.810165
(Iteration 1721 / 7350) loss: 1.892134
(Iteration 1731 / 7350) loss: 1.866010
(Iteration 1741 / 7350) loss: 1.856017
(Iteration 1751 / 7350) loss: 1.761352
(Iteration 1761 / 7350) loss: 1.618328
(Iteration 1771 / 7350) loss: 1.641649
(Iteration 1781 / 7350) loss: 1.721931
(Iteration 1791 / 7350) loss: 1.668448
(Iteration 1801 / 7350) loss: 1.778059
(Iteration 1811 / 7350) loss: 1.777037
(Iteration 1821 / 7350) loss: 1.809153
(Iteration 1831 / 7350) loss: 1.811160
(Iteration 1841 / 7350) loss: 1.820527
(Iteration 1851 / 7350) loss: 1.692573
(Iteration 1861 / 7350) loss: 1.647390
(Iteration 1871 / 7350) loss: 1.755611
(Iteration 1881 / 7350) loss: 1.769723
(Iteration 1891 / 7350) loss: 1.965016
(Iteration 1901 / 7350) loss: 1.768505
(Iteration 1911 / 7350) loss: 1.824217
(Iteration 1921 / 7350) loss: 1.747474
(Iteration 1931 / 7350) loss: 1.753206
(Iteration 1941 / 7350) loss: 1.572269
(Iteration 1951 / 7350) loss: 1.799763
(Epoch 4 / 15) train acc: 0.347000; val_acc: 0.317000
(Iteration 1961 / 7350) loss: 1.927438
(Iteration 1971 / 7350) loss: 1.816836
(Iteration 1981 / 7350) loss: 1.946886
(Iteration 1991 / 7350) loss: 1.526314
(Iteration 2001 / 7350) loss: 1.776179
(Iteration 2011 / 7350) loss: 1.841724
(Iteration 2021 / 7350) loss: 1.614056
(Iteration 2031 / 7350) loss: 1.793993
(Iteration 2041 / 7350) loss: 1.689486
(Iteration 2051 / 7350) loss: 1.637134
(Iteration 2061 / 7350) loss: 1.617715
(Iteration 2071 / 7350) loss: 1.610736
(Iteration 2081 / 7350) loss: 1.641036
(Iteration 2091 / 7350) loss: 1.569375
(Iteration 2101 / 7350) loss: 1.578428

(Iteration 2111 / 7350) loss: 1.740137
(Iteration 2121 / 7350) loss: 1.806998
(Iteration 2131 / 7350) loss: 1.784331
(Iteration 2141 / 7350) loss: 1.679218
(Iteration 2151 / 7350) loss: 1.877663
(Iteration 2161 / 7350) loss: 1.906501
(Iteration 2171 / 7350) loss: 1.737500
(Iteration 2181 / 7350) loss: 1.804416
(Iteration 2191 / 7350) loss: 1.796891
(Iteration 2201 / 7350) loss: 1.716767
(Iteration 2211 / 7350) loss: 1.614698
(Iteration 2221 / 7350) loss: 1.715787
(Iteration 2231 / 7350) loss: 1.711263
(Iteration 2241 / 7350) loss: 1.652341
(Iteration 2251 / 7350) loss: 1.749527
(Iteration 2261 / 7350) loss: 1.622154
(Iteration 2271 / 7350) loss: 1.630387
(Iteration 2281 / 7350) loss: 1.676162
(Iteration 2291 / 7350) loss: 1.787181
(Iteration 2301 / 7350) loss: 1.678322
(Iteration 2311 / 7350) loss: 1.672856
(Iteration 2321 / 7350) loss: 1.898375
(Iteration 2331 / 7350) loss: 1.647891
(Iteration 2341 / 7350) loss: 1.712308
(Iteration 2351 / 7350) loss: 1.699647
(Iteration 2361 / 7350) loss: 1.679179
(Iteration 2371 / 7350) loss: 1.607802
(Iteration 2381 / 7350) loss: 1.849558
(Iteration 2391 / 7350) loss: 1.827078
(Iteration 2401 / 7350) loss: 1.640499
(Iteration 2411 / 7350) loss: 1.703768
(Iteration 2421 / 7350) loss: 1.694267
(Iteration 2431 / 7350) loss: 1.730981
(Iteration 2441 / 7350) loss: 1.509716
(Epoch 5 / 15) train acc: 0.384000; val_acc: 0.343000
(Iteration 2451 / 7350) loss: 1.757138
(Iteration 2461 / 7350) loss: 1.621885
(Iteration 2471 / 7350) loss: 1.787332
(Iteration 2481 / 7350) loss: 1.657088
(Iteration 2491 / 7350) loss: 1.723526
(Iteration 2501 / 7350) loss: 1.626739
(Iteration 2511 / 7350) loss: 1.626434
(Iteration 2521 / 7350) loss: 1.788653
(Iteration 2531 / 7350) loss: 1.792722
(Iteration 2541 / 7350) loss: 1.715520
(Iteration 2551 / 7350) loss: 1.715441
(Iteration 2561 / 7350) loss: 1.768384
(Iteration 2571 / 7350) loss: 1.723809
(Iteration 2581 / 7350) loss: 1.786046
(Iteration 2591 / 7350) loss: 1.713307
(Iteration 2601 / 7350) loss: 1.732312
(Iteration 2611 / 7350) loss: 1.766575
(Iteration 2621 / 7350) loss: 1.718021
(Iteration 2631 / 7350) loss: 1.614539

(Iteration 2641 / 7350) loss: 1.736054
(Iteration 2651 / 7350) loss: 1.871364
(Iteration 2661 / 7350) loss: 1.743687
(Iteration 2671 / 7350) loss: 1.487849
(Iteration 2681 / 7350) loss: 1.593164
(Iteration 2691 / 7350) loss: 1.773078
(Iteration 2701 / 7350) loss: 1.778151
(Iteration 2711 / 7350) loss: 1.693882
(Iteration 2721 / 7350) loss: 1.759364
(Iteration 2731 / 7350) loss: 1.546474
(Iteration 2741 / 7350) loss: 1.677588
(Iteration 2751 / 7350) loss: 1.704570
(Iteration 2761 / 7350) loss: 1.772723
(Iteration 2771 / 7350) loss: 1.721161
(Iteration 2781 / 7350) loss: 1.766111
(Iteration 2791 / 7350) loss: 1.749083
(Iteration 2801 / 7350) loss: 1.717012
(Iteration 2811 / 7350) loss: 1.674953
(Iteration 2821 / 7350) loss: 1.561665
(Iteration 2831 / 7350) loss: 1.606159
(Iteration 2841 / 7350) loss: 1.723880
(Iteration 2851 / 7350) loss: 1.712925
(Iteration 2861 / 7350) loss: 1.757216
(Iteration 2871 / 7350) loss: 1.616173
(Iteration 2881 / 7350) loss: 1.896086
(Iteration 2891 / 7350) loss: 1.839060
(Iteration 2901 / 7350) loss: 1.677402
(Iteration 2911 / 7350) loss: 1.906292
(Iteration 2921 / 7350) loss: 1.730796
(Iteration 2931 / 7350) loss: 1.563092
(Epoch 6 / 15) train acc: 0.390000; val_acc: 0.340000
(Iteration 2941 / 7350) loss: 1.573685
(Iteration 2951 / 7350) loss: 1.752312
(Iteration 2961 / 7350) loss: 1.757302
(Iteration 2971 / 7350) loss: 1.618949
(Iteration 2981 / 7350) loss: 1.704945
(Iteration 2991 / 7350) loss: 1.655889
(Iteration 3001 / 7350) loss: 1.681134
(Iteration 3011 / 7350) loss: 1.705733
(Iteration 3021 / 7350) loss: 1.637981
(Iteration 3031 / 7350) loss: 1.941449
(Iteration 3041 / 7350) loss: 1.443501
(Iteration 3051 / 7350) loss: 1.592239
(Iteration 3061 / 7350) loss: 1.680915
(Iteration 3071 / 7350) loss: 1.744233
(Iteration 3081 / 7350) loss: 1.746512
(Iteration 3091 / 7350) loss: 1.629330
(Iteration 3101 / 7350) loss: 1.580118
(Iteration 3111 / 7350) loss: 1.669169
(Iteration 3121 / 7350) loss: 1.662932
(Iteration 3131 / 7350) loss: 1.718648
(Iteration 3141 / 7350) loss: 1.709824
(Iteration 3151 / 7350) loss: 1.735231
(Iteration 3161 / 7350) loss: 1.680724

(Iteration 3171 / 7350) loss: 1.579450
(Iteration 3181 / 7350) loss: 1.559007
(Iteration 3191 / 7350) loss: 1.587867
(Iteration 3201 / 7350) loss: 1.703295
(Iteration 3211 / 7350) loss: 1.636710
(Iteration 3221 / 7350) loss: 1.784556
(Iteration 3231 / 7350) loss: 1.732344
(Iteration 3241 / 7350) loss: 1.708678
(Iteration 3251 / 7350) loss: 1.547014
(Iteration 3261 / 7350) loss: 1.818969
(Iteration 3271 / 7350) loss: 1.654319
(Iteration 3281 / 7350) loss: 1.736920
(Iteration 3291 / 7350) loss: 1.547641
(Iteration 3301 / 7350) loss: 1.690699
(Iteration 3311 / 7350) loss: 1.708776
(Iteration 3321 / 7350) loss: 1.717904
(Iteration 3331 / 7350) loss: 1.697409
(Iteration 3341 / 7350) loss: 1.883383
(Iteration 3351 / 7350) loss: 1.707395
(Iteration 3361 / 7350) loss: 1.514776
(Iteration 3371 / 7350) loss: 1.527355
(Iteration 3381 / 7350) loss: 1.706768
(Iteration 3391 / 7350) loss: 1.787125
(Iteration 3401 / 7350) loss: 1.562759
(Iteration 3411 / 7350) loss: 1.607900
(Iteration 3421 / 7350) loss: 1.720338
(Epoch 7 / 15) train acc: 0.422000; val_acc: 0.337000
(Iteration 3431 / 7350) loss: 1.627940
(Iteration 3441 / 7350) loss: 1.609412
(Iteration 3451 / 7350) loss: 1.819164
(Iteration 3461 / 7350) loss: 1.685932
(Iteration 3471 / 7350) loss: 1.597829
(Iteration 3481 / 7350) loss: 1.604729
(Iteration 3491 / 7350) loss: 1.636478
(Iteration 3501 / 7350) loss: 1.544490
(Iteration 3511 / 7350) loss: 1.635816
(Iteration 3521 / 7350) loss: 1.672957
(Iteration 3531 / 7350) loss: 1.561501
(Iteration 3541 / 7350) loss: 1.773350
(Iteration 3551 / 7350) loss: 1.473546
(Iteration 3561 / 7350) loss: 1.594542
(Iteration 3571 / 7350) loss: 1.562101
(Iteration 3581 / 7350) loss: 1.673133
(Iteration 3591 / 7350) loss: 1.763798
(Iteration 3601 / 7350) loss: 1.635322
(Iteration 3611 / 7350) loss: 1.646769
(Iteration 3621 / 7350) loss: 1.578119
(Iteration 3631 / 7350) loss: 1.511472
(Iteration 3641 / 7350) loss: 1.642761
(Iteration 3651 / 7350) loss: 1.678872
(Iteration 3661 / 7350) loss: 1.626059
(Iteration 3671 / 7350) loss: 1.525433
(Iteration 3681 / 7350) loss: 1.530810
(Iteration 3691 / 7350) loss: 1.670099

(Iteration 3701 / 7350) loss: 1.631819
(Iteration 3711 / 7350) loss: 1.727195
(Iteration 3721 / 7350) loss: 1.719988
(Iteration 3731 / 7350) loss: 1.491564
(Iteration 3741 / 7350) loss: 1.770479
(Iteration 3751 / 7350) loss: 1.663405
(Iteration 3761 / 7350) loss: 1.705459
(Iteration 3771 / 7350) loss: 1.748615
(Iteration 3781 / 7350) loss: 1.582296
(Iteration 3791 / 7350) loss: 1.620096
(Iteration 3801 / 7350) loss: 1.767531
(Iteration 3811 / 7350) loss: 1.496059
(Iteration 3821 / 7350) loss: 1.641890
(Iteration 3831 / 7350) loss: 1.736558
(Iteration 3841 / 7350) loss: 1.596326
(Iteration 3851 / 7350) loss: 1.544076
(Iteration 3861 / 7350) loss: 1.663333
(Iteration 3871 / 7350) loss: 1.749368
(Iteration 3881 / 7350) loss: 1.559552
(Iteration 3891 / 7350) loss: 1.740926
(Iteration 3901 / 7350) loss: 1.638734
(Iteration 3911 / 7350) loss: 1.642497
(Epoch 8 / 15) train acc: 0.416000; val_acc: 0.349000
(Iteration 3921 / 7350) loss: 1.736288
(Iteration 3931 / 7350) loss: 1.763661
(Iteration 3941 / 7350) loss: 1.695768
(Iteration 3951 / 7350) loss: 1.670315
(Iteration 3961 / 7350) loss: 1.688090
(Iteration 3971 / 7350) loss: 1.588383
(Iteration 3981 / 7350) loss: 1.645222
(Iteration 3991 / 7350) loss: 1.665863
(Iteration 4001 / 7350) loss: 1.603670
(Iteration 4011 / 7350) loss: 1.516547
(Iteration 4021 / 7350) loss: 1.707760
(Iteration 4031 / 7350) loss: 1.579160
(Iteration 4041 / 7350) loss: 1.613453
(Iteration 4051 / 7350) loss: 1.784897
(Iteration 4061 / 7350) loss: 1.678826
(Iteration 4071 / 7350) loss: 1.534573
(Iteration 4081 / 7350) loss: 1.743631
(Iteration 4091 / 7350) loss: 1.527484
(Iteration 4101 / 7350) loss: 1.665314
(Iteration 4111 / 7350) loss: 1.586576
(Iteration 4121 / 7350) loss: 1.688172
(Iteration 4131 / 7350) loss: 1.517099
(Iteration 4141 / 7350) loss: 1.715220
(Iteration 4151 / 7350) loss: 1.619242
(Iteration 4161 / 7350) loss: 1.528406
(Iteration 4171 / 7350) loss: 1.617249
(Iteration 4181 / 7350) loss: 1.530325
(Iteration 4191 / 7350) loss: 1.550908
(Iteration 4201 / 7350) loss: 1.804154
(Iteration 4211 / 7350) loss: 1.632166
(Iteration 4221 / 7350) loss: 1.416915

(Iteration 4231 / 7350) loss: 1.592140
(Iteration 4241 / 7350) loss: 1.546158
(Iteration 4251 / 7350) loss: 1.648957
(Iteration 4261 / 7350) loss: 1.726808
(Iteration 4271 / 7350) loss: 1.607181
(Iteration 4281 / 7350) loss: 1.604156
(Iteration 4291 / 7350) loss: 1.626336
(Iteration 4301 / 7350) loss: 1.460919
(Iteration 4311 / 7350) loss: 1.645727
(Iteration 4321 / 7350) loss: 1.519661
(Iteration 4331 / 7350) loss: 1.696694
(Iteration 4341 / 7350) loss: 1.607997
(Iteration 4351 / 7350) loss: 1.613232
(Iteration 4361 / 7350) loss: 1.606322
(Iteration 4371 / 7350) loss: 1.636105
(Iteration 4381 / 7350) loss: 1.531007
(Iteration 4391 / 7350) loss: 1.513185
(Iteration 4401 / 7350) loss: 1.620198
(Epoch 9 / 15) train acc: 0.452000; val_acc: 0.365000
(Iteration 4411 / 7350) loss: 1.399637
(Iteration 4421 / 7350) loss: 1.550688
(Iteration 4431 / 7350) loss: 1.857136
(Iteration 4441 / 7350) loss: 1.591826
(Iteration 4451 / 7350) loss: 1.613282
(Iteration 4461 / 7350) loss: 1.635039
(Iteration 4471 / 7350) loss: 1.762930
(Iteration 4481 / 7350) loss: 1.559752
(Iteration 4491 / 7350) loss: 1.333174
(Iteration 4501 / 7350) loss: 1.797943
(Iteration 4511 / 7350) loss: 1.450168
(Iteration 4521 / 7350) loss: 1.552367
(Iteration 4531 / 7350) loss: 1.539169
(Iteration 4541 / 7350) loss: 1.547512
(Iteration 4551 / 7350) loss: 1.556792
(Iteration 4561 / 7350) loss: 1.554707
(Iteration 4571 / 7350) loss: 1.745754
(Iteration 4581 / 7350) loss: 1.593257
(Iteration 4591 / 7350) loss: 1.587000
(Iteration 4601 / 7350) loss: 1.497356
(Iteration 4611 / 7350) loss: 1.654980
(Iteration 4621 / 7350) loss: 1.581804
(Iteration 4631 / 7350) loss: 1.531778
(Iteration 4641 / 7350) loss: 1.373508
(Iteration 4651 / 7350) loss: 1.598676
(Iteration 4661 / 7350) loss: 1.536860
(Iteration 4671 / 7350) loss: 1.582486
(Iteration 4681 / 7350) loss: 1.546960
(Iteration 4691 / 7350) loss: 1.533469
(Iteration 4701 / 7350) loss: 1.467071
(Iteration 4711 / 7350) loss: 1.585996
(Iteration 4721 / 7350) loss: 1.637584
(Iteration 4731 / 7350) loss: 1.572569
(Iteration 4741 / 7350) loss: 1.706105
(Iteration 4751 / 7350) loss: 1.377429

(Iteration 4761 / 7350) loss: 1.569135
(Iteration 4771 / 7350) loss: 1.531828
(Iteration 4781 / 7350) loss: 1.692779
(Iteration 4791 / 7350) loss: 1.446657
(Iteration 4801 / 7350) loss: 1.471519
(Iteration 4811 / 7350) loss: 1.538597
(Iteration 4821 / 7350) loss: 1.708885
(Iteration 4831 / 7350) loss: 1.460866
(Iteration 4841 / 7350) loss: 1.559790
(Iteration 4851 / 7350) loss: 1.668201
(Iteration 4861 / 7350) loss: 1.424656
(Iteration 4871 / 7350) loss: 1.529562
(Iteration 4881 / 7350) loss: 1.528592
(Iteration 4891 / 7350) loss: 1.545803
(Epoch 10 / 15) train acc: 0.464000; val_acc: 0.359000
(Iteration 4901 / 7350) loss: 1.640426
(Iteration 4911 / 7350) loss: 1.649938
(Iteration 4921 / 7350) loss: 1.645410
(Iteration 4931 / 7350) loss: 1.606691
(Iteration 4941 / 7350) loss: 1.539555
(Iteration 4951 / 7350) loss: 1.569702
(Iteration 4961 / 7350) loss: 1.614459
(Iteration 4971 / 7350) loss: 1.586536
(Iteration 4981 / 7350) loss: 1.653891
(Iteration 4991 / 7350) loss: 1.574034
(Iteration 5001 / 7350) loss: 1.512305
(Iteration 5011 / 7350) loss: 1.578489
(Iteration 5021 / 7350) loss: 1.567282
(Iteration 5031 / 7350) loss: 1.507464
(Iteration 5041 / 7350) loss: 1.450799
(Iteration 5051 / 7350) loss: 1.578315
(Iteration 5061 / 7350) loss: 1.626389
(Iteration 5071 / 7350) loss: 1.300891
(Iteration 5081 / 7350) loss: 1.632202
(Iteration 5091 / 7350) loss: 1.525532
(Iteration 5101 / 7350) loss: 1.442806
(Iteration 5111 / 7350) loss: 1.475776
(Iteration 5121 / 7350) loss: 1.677707
(Iteration 5131 / 7350) loss: 1.638915
(Iteration 5141 / 7350) loss: 1.464244
(Iteration 5151 / 7350) loss: 1.434529
(Iteration 5161 / 7350) loss: 1.661768
(Iteration 5171 / 7350) loss: 1.471631
(Iteration 5181 / 7350) loss: 1.447519
(Iteration 5191 / 7350) loss: 1.519500
(Iteration 5201 / 7350) loss: 1.569190
(Iteration 5211 / 7350) loss: 1.546380
(Iteration 5221 / 7350) loss: 1.424012
(Iteration 5231 / 7350) loss: 1.766391
(Iteration 5241 / 7350) loss: 1.489593
(Iteration 5251 / 7350) loss: 1.647411
(Iteration 5261 / 7350) loss: 1.854420
(Iteration 5271 / 7350) loss: 1.622475
(Iteration 5281 / 7350) loss: 1.508118

(Iteration 5291 / 7350) loss: 1.451864
(Iteration 5301 / 7350) loss: 1.530497
(Iteration 5311 / 7350) loss: 1.487017
(Iteration 5321 / 7350) loss: 1.602176
(Iteration 5331 / 7350) loss: 1.597854
(Iteration 5341 / 7350) loss: 1.475751
(Iteration 5351 / 7350) loss: 1.679535
(Iteration 5361 / 7350) loss: 1.639779
(Iteration 5371 / 7350) loss: 1.459090
(Iteration 5381 / 7350) loss: 1.500519
(Epoch 11 / 15) train acc: 0.432000; val_acc: 0.345000
(Iteration 5391 / 7350) loss: 1.497567
(Iteration 5401 / 7350) loss: 1.526861
(Iteration 5411 / 7350) loss: 1.461011
(Iteration 5421 / 7350) loss: 1.607199
(Iteration 5431 / 7350) loss: 1.532030
(Iteration 5441 / 7350) loss: 1.620995
(Iteration 5451 / 7350) loss: 1.422959
(Iteration 5461 / 7350) loss: 1.604468
(Iteration 5471 / 7350) loss: 1.533340
(Iteration 5481 / 7350) loss: 1.549359
(Iteration 5491 / 7350) loss: 1.604627
(Iteration 5501 / 7350) loss: 1.408310
(Iteration 5511 / 7350) loss: 1.566339
(Iteration 5521 / 7350) loss: 1.520908
(Iteration 5531 / 7350) loss: 1.422619
(Iteration 5541 / 7350) loss: 1.485161
(Iteration 5551 / 7350) loss: 1.736367
(Iteration 5561 / 7350) loss: 1.291956
(Iteration 5571 / 7350) loss: 1.530924
(Iteration 5581 / 7350) loss: 1.421009
(Iteration 5591 / 7350) loss: 1.336724
(Iteration 5601 / 7350) loss: 1.546154
(Iteration 5611 / 7350) loss: 1.675825
(Iteration 5621 / 7350) loss: 1.423157
(Iteration 5631 / 7350) loss: 1.500024
(Iteration 5641 / 7350) loss: 1.480500
(Iteration 5651 / 7350) loss: 1.423486
(Iteration 5661 / 7350) loss: 1.512968
(Iteration 5671 / 7350) loss: 1.519859
(Iteration 5681 / 7350) loss: 1.554901
(Iteration 5691 / 7350) loss: 1.367742
(Iteration 5701 / 7350) loss: 1.714640
(Iteration 5711 / 7350) loss: 1.526329
(Iteration 5721 / 7350) loss: 1.504936
(Iteration 5731 / 7350) loss: 1.417141
(Iteration 5741 / 7350) loss: 1.629880
(Iteration 5751 / 7350) loss: 1.608638
(Iteration 5761 / 7350) loss: 1.630223
(Iteration 5771 / 7350) loss: 1.464205
(Iteration 5781 / 7350) loss: 1.406808
(Iteration 5791 / 7350) loss: 1.523562
(Iteration 5801 / 7350) loss: 1.456771
(Iteration 5811 / 7350) loss: 1.357193

(Iteration 5821 / 7350) loss: 1.513581
(Iteration 5831 / 7350) loss: 1.550456
(Iteration 5841 / 7350) loss: 1.418116
(Iteration 5851 / 7350) loss: 1.458601
(Iteration 5861 / 7350) loss: 1.434243
(Iteration 5871 / 7350) loss: 1.438173
(Epoch 12 / 15) train acc: 0.456000; val_acc: 0.365000
(Iteration 5881 / 7350) loss: 1.373173
(Iteration 5891 / 7350) loss: 1.610624
(Iteration 5901 / 7350) loss: 1.562413
(Iteration 5911 / 7350) loss: 1.483557
(Iteration 5921 / 7350) loss: 1.581828
(Iteration 5931 / 7350) loss: 1.538396
(Iteration 5941 / 7350) loss: 1.550413
(Iteration 5951 / 7350) loss: 1.413893
(Iteration 5961 / 7350) loss: 1.527498
(Iteration 5971 / 7350) loss: 1.486133
(Iteration 5981 / 7350) loss: 1.618425
(Iteration 5991 / 7350) loss: 1.370815
(Iteration 6001 / 7350) loss: 1.334680
(Iteration 6011 / 7350) loss: 1.598767
(Iteration 6021 / 7350) loss: 1.488774
(Iteration 6031 / 7350) loss: 1.567060
(Iteration 6041 / 7350) loss: 1.518217
(Iteration 6051 / 7350) loss: 1.451558
(Iteration 6061 / 7350) loss: 1.472381
(Iteration 6071 / 7350) loss: 1.521604
(Iteration 6081 / 7350) loss: 1.410892
(Iteration 6091 / 7350) loss: 1.509011
(Iteration 6101 / 7350) loss: 1.347182
(Iteration 6111 / 7350) loss: 1.577205
(Iteration 6121 / 7350) loss: 1.499918
(Iteration 6131 / 7350) loss: 1.720949
(Iteration 6141 / 7350) loss: 1.543195
(Iteration 6151 / 7350) loss: 1.536866
(Iteration 6161 / 7350) loss: 1.656547
(Iteration 6171 / 7350) loss: 1.335533
(Iteration 6181 / 7350) loss: 1.410430
(Iteration 6191 / 7350) loss: 1.568277
(Iteration 6201 / 7350) loss: 1.505142
(Iteration 6211 / 7350) loss: 1.488476
(Iteration 6221 / 7350) loss: 1.732659
(Iteration 6231 / 7350) loss: 1.404679
(Iteration 6241 / 7350) loss: 1.522541
(Iteration 6251 / 7350) loss: 1.472715
(Iteration 6261 / 7350) loss: 1.606227
(Iteration 6271 / 7350) loss: 1.450694
(Iteration 6281 / 7350) loss: 1.525869
(Iteration 6291 / 7350) loss: 1.277868
(Iteration 6301 / 7350) loss: 1.586787
(Iteration 6311 / 7350) loss: 1.289741
(Iteration 6321 / 7350) loss: 1.581915
(Iteration 6331 / 7350) loss: 1.496498
(Iteration 6341 / 7350) loss: 1.578500

(Iteration 6351 / 7350) loss: 1.588855
(Iteration 6361 / 7350) loss: 1.434273
(Epoch 13 / 15) train acc: 0.491000; val_acc: 0.357000
(Iteration 6371 / 7350) loss: 1.494416
(Iteration 6381 / 7350) loss: 1.410093
(Iteration 6391 / 7350) loss: 1.439927
(Iteration 6401 / 7350) loss: 1.603835
(Iteration 6411 / 7350) loss: 1.474359
(Iteration 6421 / 7350) loss: 1.481388
(Iteration 6431 / 7350) loss: 1.435388
(Iteration 6441 / 7350) loss: 1.511647
(Iteration 6451 / 7350) loss: 1.505457
(Iteration 6461 / 7350) loss: 1.287160
(Iteration 6471 / 7350) loss: 1.550198
(Iteration 6481 / 7350) loss: 1.500644
(Iteration 6491 / 7350) loss: 1.636622
(Iteration 6501 / 7350) loss: 1.452750
(Iteration 6511 / 7350) loss: 1.538059
(Iteration 6521 / 7350) loss: 1.398171
(Iteration 6531 / 7350) loss: 1.684813
(Iteration 6541 / 7350) loss: 1.379698
(Iteration 6551 / 7350) loss: 1.589687
(Iteration 6561 / 7350) loss: 1.354870
(Iteration 6571 / 7350) loss: 1.551334
(Iteration 6581 / 7350) loss: 1.558222
(Iteration 6591 / 7350) loss: 1.520080
(Iteration 6601 / 7350) loss: 1.369413
(Iteration 6611 / 7350) loss: 1.515885
(Iteration 6621 / 7350) loss: 1.390640
(Iteration 6631 / 7350) loss: 1.568769
(Iteration 6641 / 7350) loss: 1.520538
(Iteration 6651 / 7350) loss: 1.581415
(Iteration 6661 / 7350) loss: 1.580938
(Iteration 6671 / 7350) loss: 1.480574
(Iteration 6681 / 7350) loss: 1.455587
(Iteration 6691 / 7350) loss: 1.324207
(Iteration 6701 / 7350) loss: 1.429388
(Iteration 6711 / 7350) loss: 1.577855
(Iteration 6721 / 7350) loss: 1.316812
(Iteration 6731 / 7350) loss: 1.411847
(Iteration 6741 / 7350) loss: 1.512113
(Iteration 6751 / 7350) loss: 1.409080
(Iteration 6761 / 7350) loss: 1.667095
(Iteration 6771 / 7350) loss: 1.340542
(Iteration 6781 / 7350) loss: 1.410708
(Iteration 6791 / 7350) loss: 1.419889
(Iteration 6801 / 7350) loss: 1.444630
(Iteration 6811 / 7350) loss: 1.306867
(Iteration 6821 / 7350) loss: 1.415932
(Iteration 6831 / 7350) loss: 1.382849
(Iteration 6841 / 7350) loss: 1.427792
(Iteration 6851 / 7350) loss: 1.401860
(Epoch 14 / 15) train acc: 0.507000; val_acc: 0.368000
(Iteration 6861 / 7350) loss: 1.652642

(Iteration 6871 / 7350) loss: 1.469034
(Iteration 6881 / 7350) loss: 1.580691
(Iteration 6891 / 7350) loss: 1.466055
(Iteration 6901 / 7350) loss: 1.317152
(Iteration 6911 / 7350) loss: 1.479717
(Iteration 6921 / 7350) loss: 1.651964
(Iteration 6931 / 7350) loss: 1.513250
(Iteration 6941 / 7350) loss: 1.517727
(Iteration 6951 / 7350) loss: 1.622452
(Iteration 6961 / 7350) loss: 1.287146
(Iteration 6971 / 7350) loss: 1.354483
(Iteration 6981 / 7350) loss: 1.470998
(Iteration 6991 / 7350) loss: 1.560420
(Iteration 7001 / 7350) loss: 1.452498
(Iteration 7011 / 7350) loss: 1.352650
(Iteration 7021 / 7350) loss: 1.487309
(Iteration 7031 / 7350) loss: 1.461487
(Iteration 7041 / 7350) loss: 1.500072
(Iteration 7051 / 7350) loss: 1.375355
(Iteration 7061 / 7350) loss: 1.489775
(Iteration 7071 / 7350) loss: 1.470765
(Iteration 7081 / 7350) loss: 1.556940
(Iteration 7091 / 7350) loss: 1.500007
(Iteration 7101 / 7350) loss: 1.368693
(Iteration 7111 / 7350) loss: 1.540051
(Iteration 7121 / 7350) loss: 1.391096
(Iteration 7131 / 7350) loss: 1.440499
(Iteration 7141 / 7350) loss: 1.360743
(Iteration 7151 / 7350) loss: 1.374358
(Iteration 7161 / 7350) loss: 1.604628
(Iteration 7171 / 7350) loss: 1.671852
(Iteration 7181 / 7350) loss: 1.588542
(Iteration 7191 / 7350) loss: 1.469882
(Iteration 7201 / 7350) loss: 1.496700
(Iteration 7211 / 7350) loss: 1.291842
(Iteration 7221 / 7350) loss: 1.391199
(Iteration 7231 / 7350) loss: 1.464823
(Iteration 7241 / 7350) loss: 1.457575
(Iteration 7251 / 7350) loss: 1.347855
(Iteration 7261 / 7350) loss: 1.563089
(Iteration 7271 / 7350) loss: 1.509206
(Iteration 7281 / 7350) loss: 1.618334
(Iteration 7291 / 7350) loss: 1.403822
(Iteration 7301 / 7350) loss: 1.453261
(Iteration 7311 / 7350) loss: 1.507360
(Iteration 7321 / 7350) loss: 1.535640
(Iteration 7331 / 7350) loss: 1.547574
(Iteration 7341 / 7350) loss: 1.565458
(Epoch 15 / 15) train acc: 0.490000; val_acc: 0.352000

Test your model

Run your best model on the validation and test sets and record the training logs of the best solver. You should achieve above 45% accuracy on the validation set.

```
In [12]: y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
val_acc = (y_val_pred == data['y_val']).mean()
test_acc = (y_test_pred == data['y_test']).mean()
print ('Validation set accuracy: ', val_acc)
print ('Test set accuracy: ', test_acc)
best_solver.record_histories_as_npz('submission_logs/best_fc_model.npz')
import json
with open("submission_logs/results.json", "w", encoding="utf-8") as f:
    json.dump(dict(
        val_acc = val_acc,
        test_acc = test_acc,
        lr = lr,
        num_epochs = num_epochs,
        batch_size = batch_size,
        lr_decay = lr_decay,
        update_rule = update_rule
    ), f)
```

Validation set accuracy: 0.368

Test set accuracy: 0.366

Collect your submissions

On Colab, after running the following cell, you can download your submissions from the **Files** tab, which can be opened by clicking the file icon on the left hand side of the screen.

```
In [ ]: !rm -f cs182hw2_submission.zip
!zip -r cs182hw2_submission.zip . -x "*.git*" "*deeplearning/datasets*" "*.ipy"
```

Visualizing features from local linearization of neural nets

```
!pip install ipyml torchviz
!pip install torch

import torch
import torch.nn as nn
import matplotlib.pyplot as plt
import numpy as np
import copy
import time
from torchvision.models.feature_extraction import
create_feature_extractor
from ipywidgets import fixed, interactive, widgets
%matplotlib inline

def to_torch(x):
    return torch.from_numpy(x).float()

def to_numpy(x):
    return x.detach().cpu().numpy()

def plot_data(X, y, X_test, y_test):
    clip_bound = 2.5
    plt.xlim(0, 1)
    plt.ylim(-clip_bound, clip_bound)
    plt.scatter(X[:, 0], y, c='darkorange', s=40.0, label='training
data points')
    plt.plot(X_test, y_test, '--', color='royalblue', linewidth=2.0,
label='Ground truth')

def plot_relu(bias, slope):
    plt.scatter([-bias / slope], 0, c='darkgrey', s=40.0)
    if slope > 0 and bias < 0:
        plt.plot([0, -bias / slope, 1], [0, 0, slope * (1 - bias)],
':')
    elif slope < 0 and bias > 0:
        plt.plot([0, -bias / slope, 1], [-bias * slope, 0, 0], ':')

def plot_relus(params):
    slopes = to_numpy(params[0]).ravel()
    biases = to_numpy(params[1])
```

```

    for relu in range(biases.size):
        plot_relu(biases[relu], slopes[relu])

def plot_function(X_test, net):
    y_pred = net(to_torch(X_test))
    plt.plot(X_test, to_numpy(y_pred), '-', color='forestgreen',
label='prediction')

def plot_update(X, y, X_test, y_test, net, state=None):
    if state is not None:
        net.load_state_dict(state)
    plt.figure(figsize=(10, 7))
    plot_relus(list(net.parameters()))
    plot_function(X_test, net)
    plot_data(X, y, X_test, y_test)
    plt.legend()
    plt.show();

def train_network(X, y, X_test, y_test, net, optim, n_steps,
save_every, device="cpu", initial_weights=None, verbose=False):
    loss = torch.nn.MSELoss()
    y_train = to_torch(y.reshape(-1, 1)).to(device=device)
    X_train = to_torch(X).to(device=device)

    y_test = to_torch(y_test.reshape(-1, 1)).to(device=device)
    X_test = to_torch(X_test).to(device=device)
    if initial_weights is not None:
        net.load_state_dict(initial_weights)
    history = {}
    for s in range(n_steps):
        perm = torch.randperm(y.size, device=device)
        subsample = perm[:y.size // 5]
        step_loss = loss(y_train[subsample],
net(X_train[subsample, :]))
        optim.zero_grad()
        step_loss.backward()
        optim.step()
        if (s + 1) % save_every == 0 or s == 0:
            history[s + 1] = {}
            history[s + 1]['state'] = copy.deepcopy(net.state_dict())
            with torch.no_grad():
                test_loss = loss(y_test, net(X_test))
            history[s + 1]['train_error'] = to_numpy(step_loss).item()
            history[s + 1]['test_error'] = to_numpy(test_loss).item()
            if verbose:
                print("SGD Iteration %d" % (s + 1))
                print("\tTrain Loss: %.3f" %

```

```

to_numpy(step_loss).item())
    print("\tTest Loss: %.3f" %
to_numpy(test_loss).item())
    else:
        # Print update every 10th save point
        if (s + 1) % (save_every * 10) == 0:
            print("SGD Iteration %d" % (s + 1))

    return history

def plot_test_train_errors(history):
    sample_points = np.array(list(history.keys()))
    etrain = [history[s]['train_error'] for s in history]
    etest = [history[s]['test_error'] for s in history]
    plt.plot(sample_points / 1e3, etrain, label='Train Error')
    plt.plot(sample_points / 1e3, etest, label='Test Error')
    plt.xlabel("Iterations (1000's)")
    plt.ylabel("MSE")
    plt.yscale('log')
    plt.legend()
    plt.show();

def make_iter_slider(iters):
    return widgets.SelectionSlider(
        options=iters,
        value=1,
        description='SGD Iterations: ',
        disabled=False
    )

def history_interactive(history, idx, X, y, X_test, y_test, net):
    plot_update(X, y, X_test, y_test, net, state=history[idx]
['state'])
    plt.show()
    print("Train Error: %.3f" % history[idx]['train_error'])
    print("Test Error: %.3f" % history[idx]['test_error'])

def make_history_interactive(history, X, y, X_test, y_test, net):
    sample_points = list(history.keys())
    return interactive(history_interactive,
                        history=fixed(history),
                        idx=make_iter_slider(sample_points),
                        X=fixed(X),
                        y=fixed(y),
                        X_test=fixed(X_test),
                        y_test=fixed(y_test),

```

```
net=fixed(net))
```

```
%matplotlib inline
```

Generate Training and Test Data

We are using piecewise linear function. Our training data has added noise $y=f(x)+\epsilon, \epsilon \sim N(0, \sigma^2)$. The test data is noise free.

Once you have gone through the discussion once you may wish to adjust the number of training samples and noise variance to see how gradient descent behaves under the new conditions.

```
f_type = 'piecewise_linear'

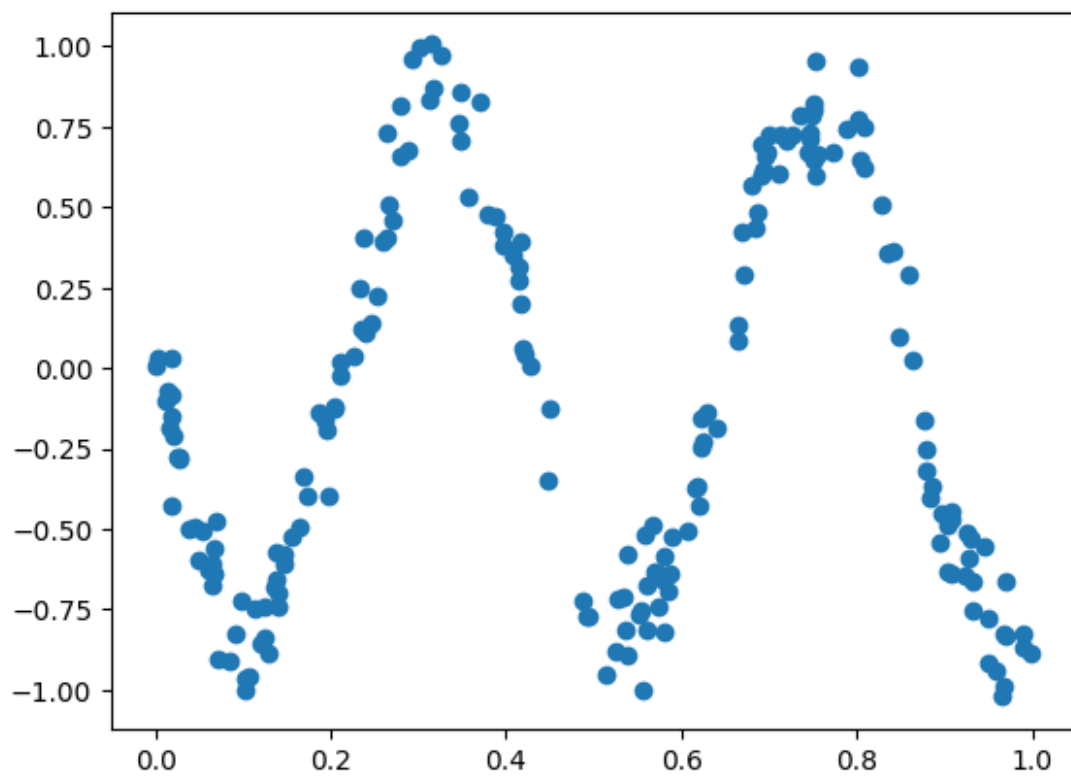
def f_true(X, f_type):
    if f_type == 'sin(20x)':
        return np.sin(20 * X[:,0])
    else:
        TenX = 10 * X[:,0]
        _ = 12345
        return (TenX - np.floor(TenX)) * np.sin(_ * np.ceil(TenX)) -
            (TenX - np.ceil(TenX)) * np.sin(_ * np.floor(TenX))

n_features = 1
n_samples = 200
sigma = 0.1
rng = np.random.RandomState(1)

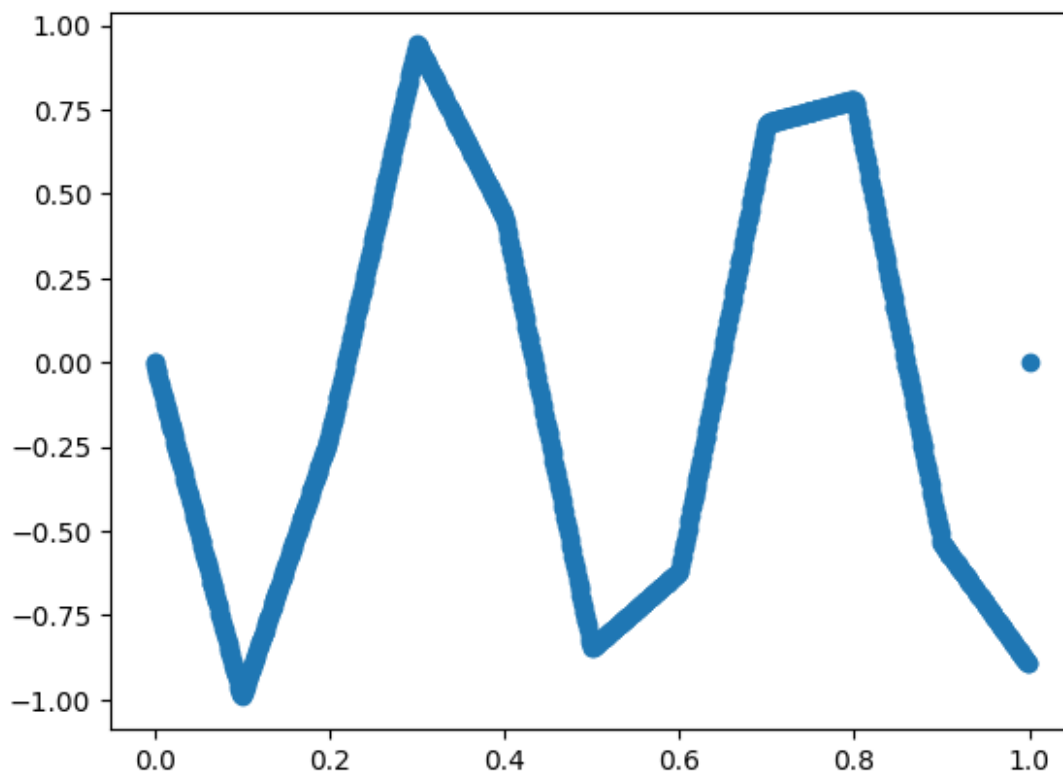
# Generate train data
X = np.sort(rng.rand(n_samples, n_features), axis=0)
y = f_true(X, f_type) + rng.randn(n_samples) * sigma

# Generate NOISELESS test data
X_test = np.concatenate([X.copy(), np.expand_dims(np.linspace(0., 1.,
1000), axis=1)])
X_test = np.sort(X_test, axis=0)
y_test = f_true(X_test, f_type)

plt.scatter(X, y)
plt.show()
```



```
plt.scatter(X_test, y_test)  
plt.show()
```

Define the Neural Networks

We will learn the piecewise linear target function using a simple 1-hidden layer neural network with ReLU non-linearity, defined by

$$\hat{y} = W^{(2)} \Phi(W^{(1)} x + b^{(1)}) + b^{(2)}$$

where $\Phi(x) = \text{ReLU}(x)$ and superscripts refer to indices, not the power operator.

We will also create two SGD optimizers to allow us to choose whether to train all parameters or only the linear output layer's parameters. Note that we use separate learning rates for the two version of training. There is too much variance in the gradients when training all layers to use a large learning rate, so we have to decrease it.

We will modify the default initialization of the biases so that the ReLU elbows are all inside the region we are interested in.

We create several versions of this network with varying widths to explore how hidden layer width impacts learning performance.

Once you have gone through the discussion once you may wish to train networks with even larger widths to see how they behave under the three different training paradigms in this notebook.

```

# Don't rerun this cell after training or you will lose all your work
nets_by_size = {}

widths = [10, 20, 40]
for width in widths:
    # Define a 1-hidden layer ReLU nonlinearity network
    net = nn.Sequential(nn.Linear(1, width),
                        nn.ReLU(),
                        nn.Linear(width, 1))

    loss = nn.MSELoss()
    # Get trainable parameters
    weights_all = list(net.parameters())
    # Get the output weights alone
    weights_out = weights_all[2:]
    # Adjust initial biases so elbows are in [0,1]
    elbows = np.sort(np.random.rand(width))
    new_biases = -elbows * to_numpy(weights_all[0]).ravel()
    weights_all[1].data = to_torch(new_biases)
    # Create SGD optimizers for outputs alone and for all weights
    lr_out = 0.2
    lr_all = 0.02
    opt_all = torch.optim.SGD(params=weights_all, lr=lr_all)
    opt_out = torch.optim.SGD(params=weights_out, lr=lr_out)
    # Save initial state for comparisons
    initial_weights = copy.deepcopy(net.state_dict())
    # print("Initial Weights", initial_weights)
    nets_by_size[width] = {'net': net, 'opt_all': opt_all,
                          'opt_out': opt_out, 'init':
initial_weights}

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
for width, net in nets_by_size.items():
    net['net'].to(device=device)

```

Train the neural networks

```

n_steps = 150000
save_every = 1000
t0 = time.time()
for w in widths:
    print("-"*40)
    print("Width", w)
    new_net = nn.Sequential(nn.Linear(1, w),
                          nn.ReLU(),
                          nn.Linear(w, 1))
    new_net.load_state_dict(nets_by_size[w]

```

```

['net'].state_dict().copy())
    new_net.to(device=device)
    opt_all = torch.optim.SGD(params=new_net.parameters(), lr=lr_all)
    initial_weights = nets_by_size[w]['init']
    history_all = train_network(X, y, X_test, y_test,
                               new_net, optim=opt_all,
                               n_steps=n_steps, save_every=save_every,
                               initial_weights=initial_weights,
                               verbose=False, device=device)

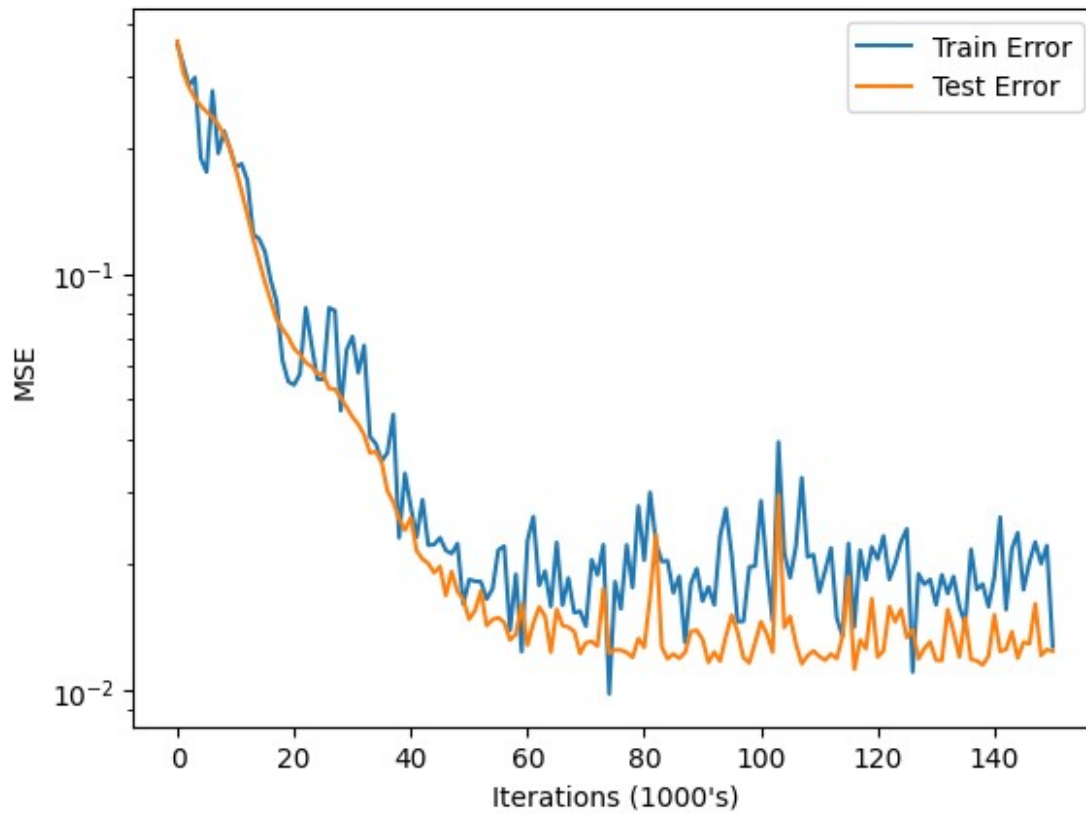
    nets_by_size[w]['trained_net'] = new_net
    nets_by_size[w]['hist_all'] = history_all
    print("Width", w)
    plot_test_train_errors(history_all)
t1 = time.time()
print("-"*40)
print("Trained all layers in %.1f minutes" % ((t1 - t0) / 60))

```

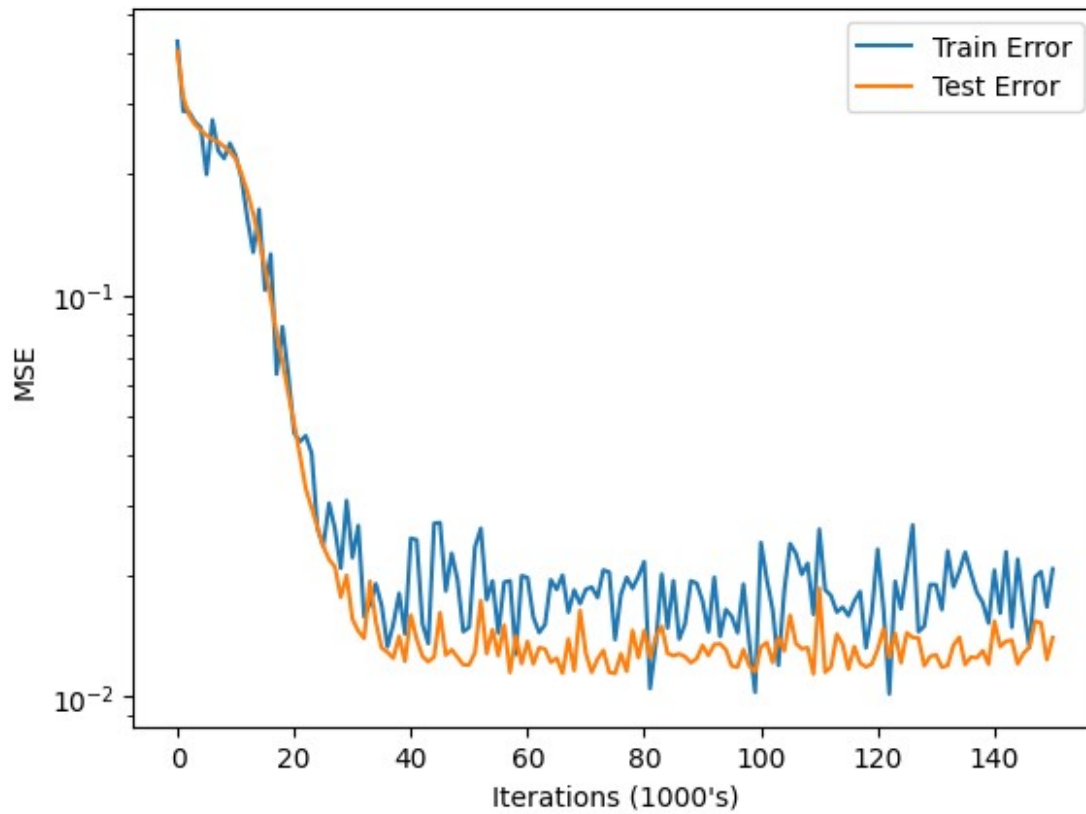
```

-----
Width 10
SGD Iteration 10000
SGD Iteration 20000
SGD Iteration 30000
SGD Iteration 40000
SGD Iteration 50000
SGD Iteration 60000
SGD Iteration 70000
SGD Iteration 80000
SGD Iteration 90000
SGD Iteration 100000
SGD Iteration 110000
SGD Iteration 120000
SGD Iteration 130000
SGD Iteration 140000
SGD Iteration 150000
Width 10

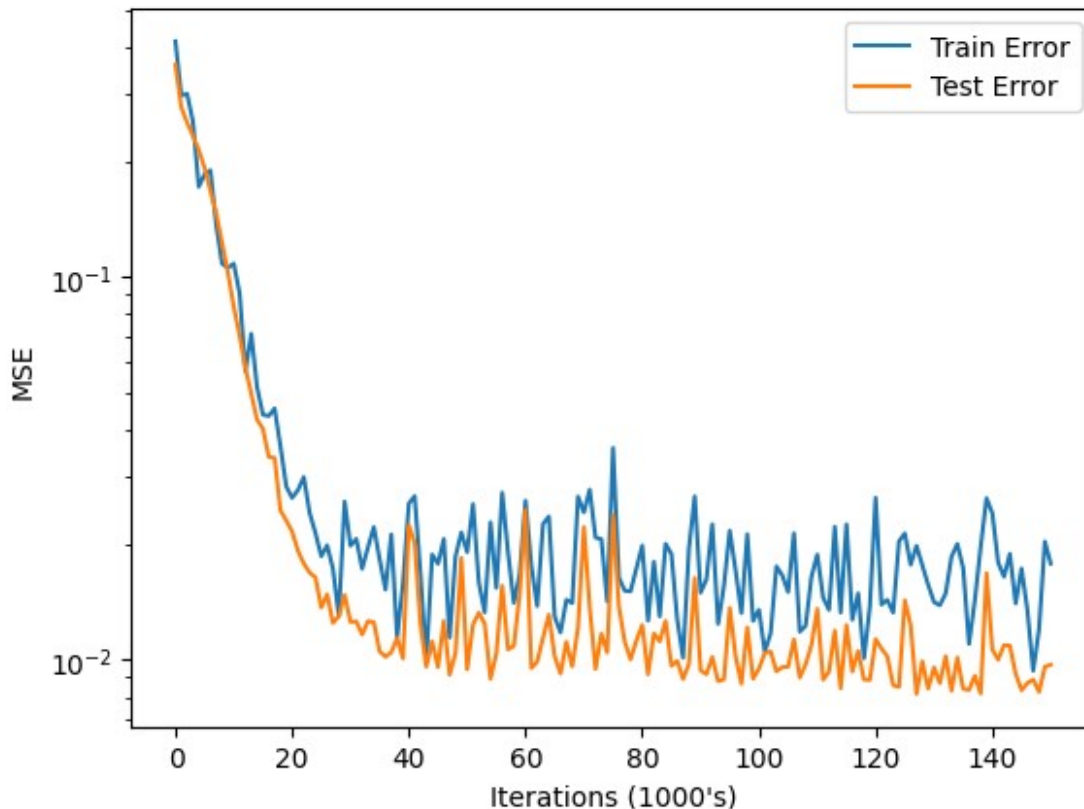
```



```
-----  
Width 20  
SGD Iteration 10000  
SGD Iteration 20000  
SGD Iteration 30000  
SGD Iteration 40000  
SGD Iteration 50000  
SGD Iteration 60000  
SGD Iteration 70000  
SGD Iteration 80000  
SGD Iteration 90000  
SGD Iteration 100000  
SGD Iteration 110000  
SGD Iteration 120000  
SGD Iteration 130000  
SGD Iteration 140000  
SGD Iteration 150000  
Width 20
```



```
-----  
Width 40  
SGD Iteration 10000  
SGD Iteration 20000  
SGD Iteration 30000  
SGD Iteration 40000  
SGD Iteration 50000  
SGD Iteration 60000  
SGD Iteration 70000  
SGD Iteration 80000  
SGD Iteration 90000  
SGD Iteration 100000  
SGD Iteration 110000  
SGD Iteration 120000  
SGD Iteration 130000  
SGD Iteration 140000  
SGD Iteration 150000  
Width 40
```



Trained all layers in 14.0 minutes

(a) Visualize Gradients

Visualize the features corresponding to $\frac{\partial}{\partial w_i^{(1)}} y(x)$ and $\frac{\partial}{\partial b_i^{(1)}} y(x)$ where $w_i^{(1)}$ are the first hidden layer's weights and the $b_i^{(1)}$ are the first hidden layer's biases. These derivatives should be evaluated at at least both the random initialization and the final trained network. When visualizing these features, plot them as a function of the scalar input x , the same way that the notebook plots the constituent "elbow" features that are the outputs of the penultimate layer.

```
def backward_and_plot_grad(X, model, vis_name='all', title='',
    legend=False):
    """
    Run backpropagation on `model` using `X` as the input
    to compute the gradient w.r.t. parameters of `y`,
    and then visualize collected gradients according to `vis_name`
    """
    width = model[0].out_features # the width is the number of hidden
    units.
```

```

gradients = np.zeros((width, X.shape[0]))
num_pts = 0
gradient_collect, vis_collect = { }, { }
for x in X:
    y = model(to_torch(x).to(device=device))

#####
##
# TODO: Complete the following part to run backpropagation. (2
lines)
# Hint: Remember to set grad to zero before backpropagation
model.zero_grad()
y.backward()
#####
##
# (YOUR CODE HERE)

#####
##

# collect gradients from `p.grad.data`
for n, p in model.named_parameters():
    for w_idx, w_grad in enumerate( p.grad.data.reshape(-1) ):
        if f'{n}.{w_idx}' not in gradient_collect:
            gradient_collect[ f'{n}.{w_idx}' ] = {'x':[], 'y':
[]]

            if vis_name == 'all' or vis_name == n:
                if f'{n}.{w_idx}' not in vis_collect:
                    vis_collect[f'{n}.{w_idx}'] = True
                gradient_collect[ f'{n}.{w_idx}' ]
['y'].append( w_grad.item() )
            gradient_collect[ f'{n}.{w_idx}' ][ 'x' ].append( x )

for w_n in vis_collect:
    # we assume that X is sorted, so we use line plot
    plt.plot( X, gradient_collect[w_n]['y'], label=w_n )

plt.xlabel('Data Points (X)')
plt.ylabel(f'Gradient for {vis_name} of {width}-width Net')
if legend:
    plt.legend()
plt.title(title)
plt.show()

for width in nets_by_size:
    backward_and_plot_grad(X, nets_by_size[width]['net'], '0.weight',
'Random Init')
    backward_and_plot_grad(X, nets_by_size[width]['trained_net'],

```

```

'0.weight', 'Trained')
    backward_and_plot_grad(X, nets_by_size[width]['net'], '0.bias',
'Random Init')
    backward_and_plot_grad(X, nets_by_size[width]['trained_net'],
'0.bias', 'Trained')

```

(b) SVD for feature matrix

During training, we can imagine that we have a generalized linear model with a feature matrix corresponding to the linearized features corresponding to each learnable parameter. We know from our analysis of gradient descent, that the singular values and singular vectors corresponding to this feature matrix are important.

Use the SVD of this feature matrix to plot both the singular values and visualize the “principle features” that correspond to the d -dimensional singular vectors multiplied by all the features corresponding to the parameters

(HINT: Remember that the feature matrix whose SVD you are taking has n rows where each row corresponds to one training point and d columns where each column corresponds to each of the learnable features. Meanwhile, you are going to be plotting/visualizing the “principle features” as functions of x even at places where you don't have training points.)

```

def compute_svd_plot_features(X, y, X_test, y_test, model):
    width = model[0].out_features # the width is the number of hidden
units.
    gradients = np.zeros((width, X.shape[0]))
    num_pts = 0
    gradient_collect, vis_collect = { }, { }
    for x in X:
        y = model(to_torch(x).to(device=device))

#####
##
# TODO: Complete the following part to run backpropagation. (2
lines)
# Hint: The same as part (a)
model.zero_grad()
y.backward()
#####
##
# (YOUR CODE HERE)

#####
##

    for n, p in model.named_parameters():
        for w_idx, w_grad in enumerate( p.grad.view(-1).data ):

```



```

        if f'{n}.{w_idx}' not in gradient_collect:
            gradient_collect[ f'{n}.{w_idx}' ] = {'x':[], 'y':
[]}]
            gradient_collect[ f'{n}.{w_idx}' ]
['y'].append( w_grad.item() )
            gradient_collect[ f'{n}.{w_idx}' ][ 'x' ].append( x )

feature_matrix = []
for w_n in gradient_collect:
    feature_matrix.append( gradient_collect[w_n]['y'] )
feature_matrix = np.array( feature_matrix ).T

#####
#####
# TODO: Complete the following part to SVD-decompose the feature
matrix.
#         (1 line)
# Hint: the shape of u, s, vh should be [n, d], [d], and [d, d]
#         respectively

#####
#####
U, s, Vh = np.linalg.svd(feature_matrix, full_matrices=False)
#####
#####

plt.scatter(np.arange(s.shape[0]), s, c='darkorange', s=40.0,
label='singular values')
plt.legend()
plt.show()

# Construct more training matrix

#####
#####
# TODO: Complete the following part to compute the principal
feature
#         (1 line)
principle_feature = feature_matrix @ Vh.T
#####
#####
# (YOUR CODE HERE)

#####
#####

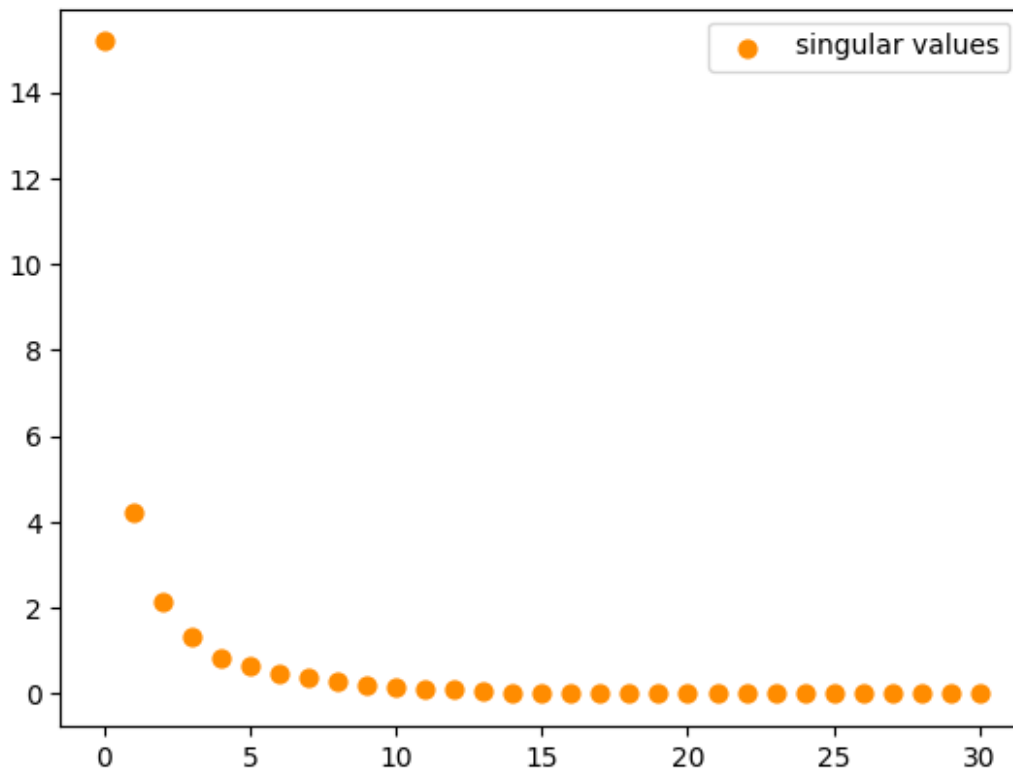
for w_idx in range(feature_matrix.shape[1]):
    plt.plot( X, principle_feature.T[w_idx] )

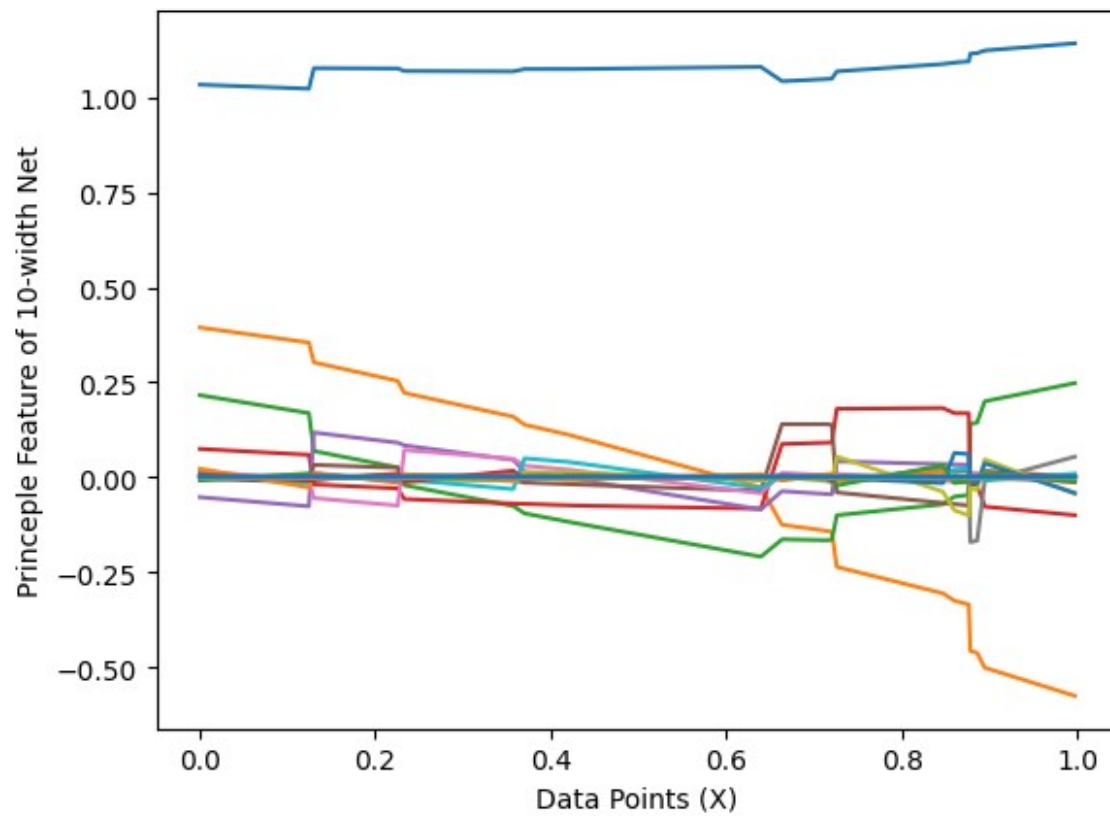
```

```
plt.xlabel('Data Points (X)')
plt.ylabel(f'Principle Feature of {width}-width Net')
plt.show()

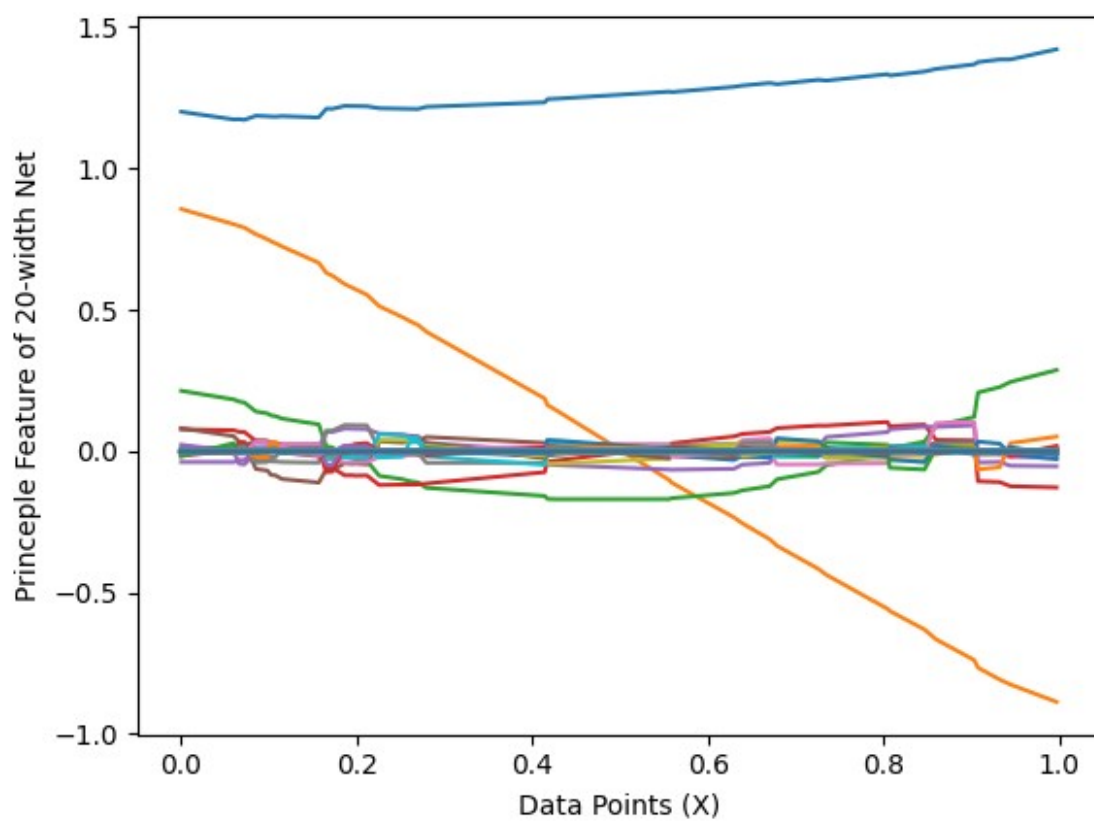
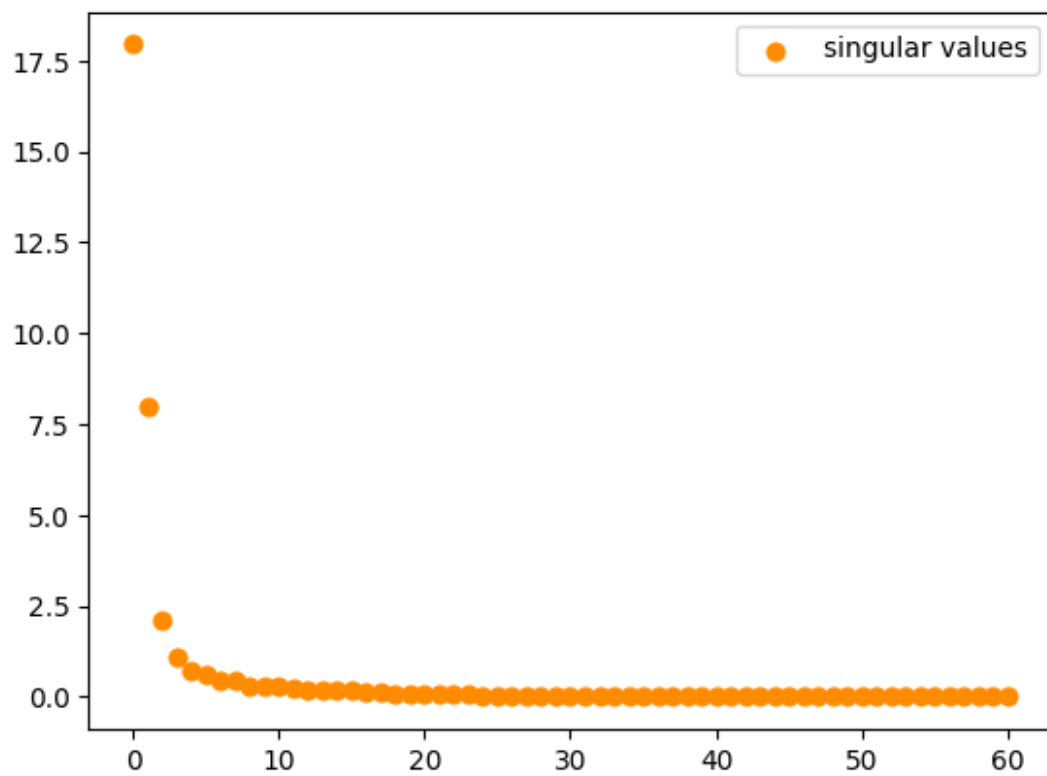
for w in widths:
    net = nets_by_size[w]['net']
    print("Width", w)
    compute_svd_plot_features(X, y, X_test, y_test, net)
```

Width 10

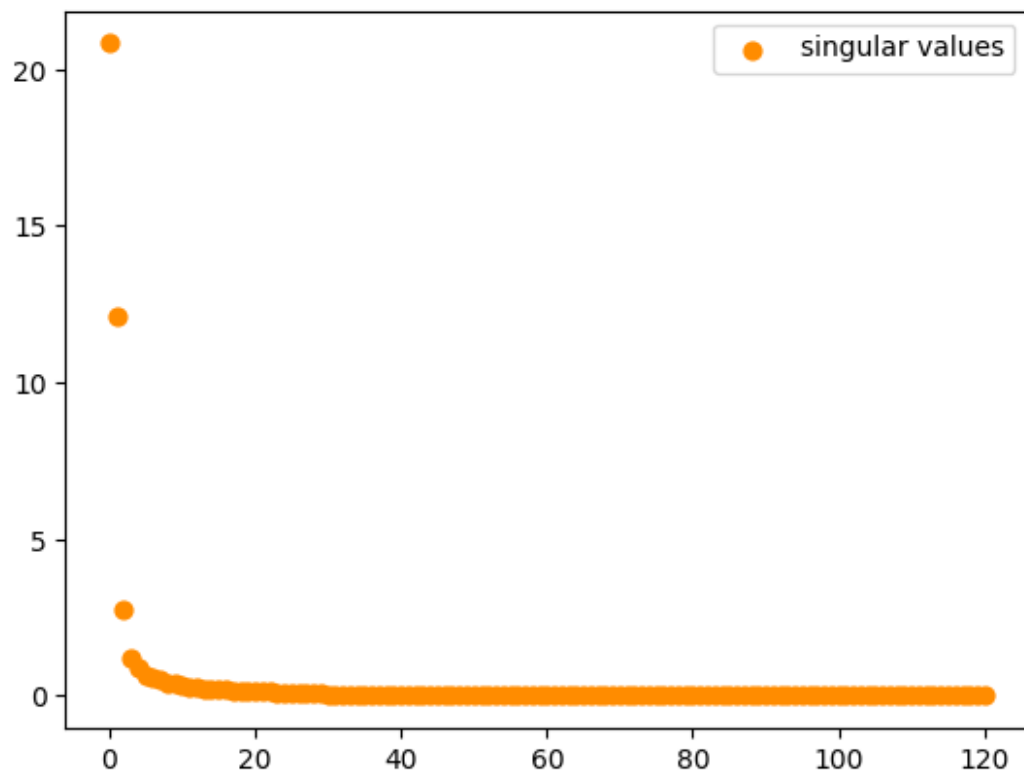


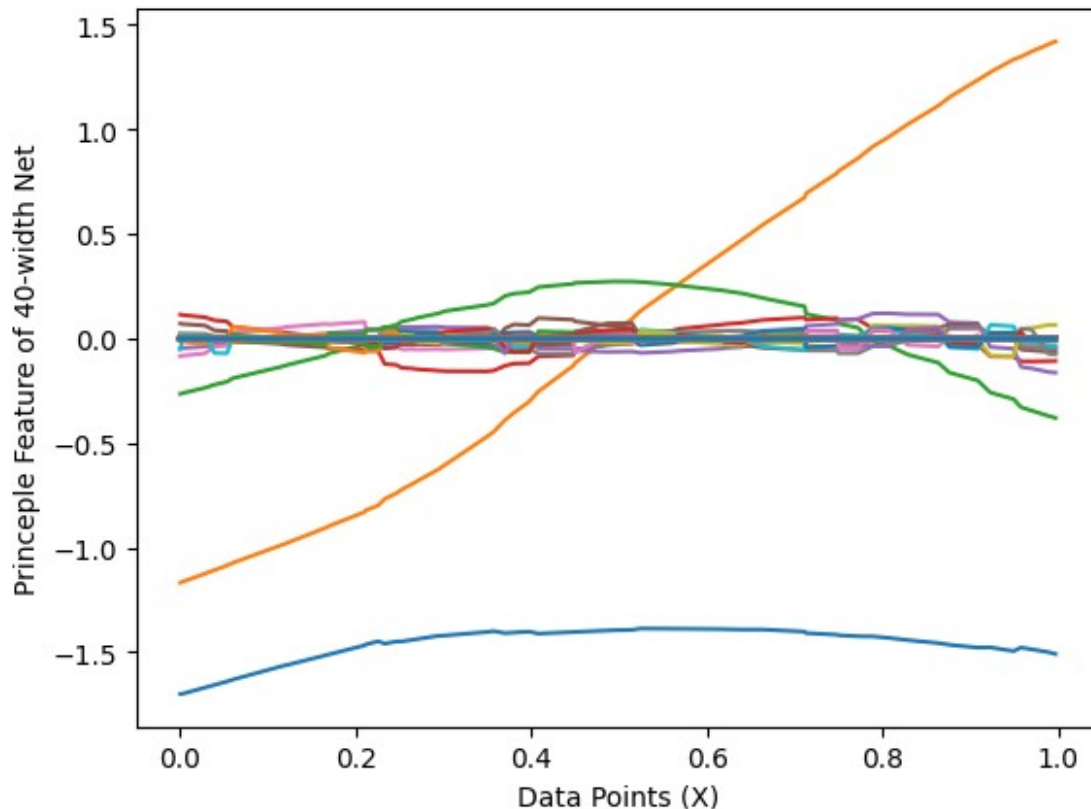


Width 20



Width 40





(c) Two-layer Network

Augment the jupyter notebook to add a second hidden layer of the same size as the first hidden layer, fully connected to the first hidden layer.

Allow the visualization of the features corresponding to the parameters in both hidden layers, as well as the "principle features" and the singular values.

```
#####
#####
# (YOUR CODE HERE)
# Hint: You can reuse most of the code from above. You should change
# the network
# definition and visualization function 'backward_and_plot_grad' such
# that an
# additional hidden layer is added.
#####
#####

widths = [10, 20, 40]
nets_by_size = {}

for width in widths:
```

```

# Two hidden layers
net = nn.Sequential(
    nn.Linear(1, width),
    nn.ReLU(),
    nn.Linear(width, width),
    nn.ReLU(),
    nn.Linear(width, 1)
)

# Adjust biases of first layer only (as before)
with torch.no_grad():
    elbows = np.sort(np.random.rand(width))
    net[0].bias.data = to_torch(-elbows *
to_numpy(net[0].weight).ravel())

opt_all = torch.optim.SGD(net.parameters(), lr=0.01) # smaller lr
for deeper net
initial_weights = copy.deepcopy(net.state_dict())
nets_by_size[width] = {
    'net': net,
    'opt_all': opt_all,
    'init': initial_weights
}

```

Optimization Techniques for "Bad" 😞 Objective Functions

In this homework, we will be learning a 2-parameter model $f(x, y)$ (x and y are the two parameters) using different optimization techniques. Formally, the problem we are trying to solve is

$$\arg \max_{x, y} f(x, y)$$

Our function has the following closed form:

$$f(x, y) = \sin(2.5x - 5y) + \cos(4x + 8y) - (x - 2)^2 - (y - 2)^2.$$

As you might have already guessed, this objective function has a lot of local optima (e.g., when $\nabla f(x, y) = 0$).

In this notebook, you will see how standard gradient descent (ascent here as we are maximizing the function) might run into issues optimizing such a function and learn about a couple of other optimization techniques that overcome the local optima issue.

To get started, let's first write the function in code and visualize it (brighter color = higher value).

```
import numpy as np

def f(x, y):
    return np.sin(2.5 * (x - 2 * y)) + np.cos(4.0 * (x + 2 * y)) - ((x - 2.0) ** 2 + (y - 2.0) ** 2)

import matplotlib
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from ipywidgets import interactive, widgets, Layout, interact

# add contour line
plt.rcParams['contour.negative_linestyle'] = 'solid'

class Visualizer:
    def __init__(self, func, x_lim=(-1, 3), y_lim=(-1, 3), res=100, n_contour_lines=15, quantized_values=None):
        xs = np.linspace(*x_lim, res)
        ys = np.linspace(*y_lim, res)
        self.xs, self.ys = np.meshgrid(xs, ys)
        self.zs = func(self.xs, self.ys)

        index = np.argmax(self.zs.reshape(-1))
        self.opt_x, self.opt_y = self.xs.reshape(-1)[index], self.ys.reshape(-1)[index]
```



```

self.x_lim, self.y_lim = x_lim, y_lim

self.n_contour_lines = n_contour_lines
self.quantized_values = quantized_values

def _plot_base_function(self, ax, no_bg=False, alpha=1.0,
new_zs=None, new_opt=None):
    if new_zs is not None:
        zs = new_zs
    else:
        zs = self.zs
    if not no_bg:
        ctf = ax.contourf(self.xs, self.ys, zs, 100, cmap='plasma')
        ax.contour(self.xs, self.ys, zs, self.n_contour_lines,
linewidths=1, colors="black", alpha=alpha)

    if self.quantized_values is not None:
        assert not no_bg
        patches = []
        for q in self.quantized_values:
            color = ctf.to_rgba(q)
            patches.append(mpatches.Patch(color=color, label=f'$f_q(x,
y)={q}$'))
        legend = plt.legend(handles=patches, loc='best', ncol=3,
fontsize="small")
        ax.add_artist(legend)
    else:
        if new_opt is None:
            ax.scatter(self.opt_x, self.opt_y, marker="*", s=50,
color="red", label="Global Optimum", zorder=4)
        else:
            ax.scatter(self.opt_x, self.opt_y, marker="*", s=50,
color="red", label="Global Optimum (Original)", zorder=4, alpha=0.75)
            ax.scatter(*new_opt, marker="*", s=50, color="darkgreen",
label="Global Optimum (Smoothed)", zorder=4, alpha=0.75)

def draw(self, ax, curves=None, no_bg=False, plot_base=True,
base_alpha=1.0,
new_zs=None, new_opt=None, legend=True):
    ax.clear()
    if plot_base:
        self._plot_base_function(ax, no_bg=no_bg, alpha=base_alpha,
new_zs=new_zs, new_opt=new_opt)
    if curves is not None:
        for label, curve in curves.items():
            xs = curve["x"]
            ys = curve["y"]

            lines = []
            alphas = np.linspace(0.25, 1., len(xs) - 1)

```

```

        for x1, x2, y1, y2 in zip(xs[:-1], xs[1:], ys[:-1], ys[1:]):
            lines.extend([(x1, x2), (y1, y2)])

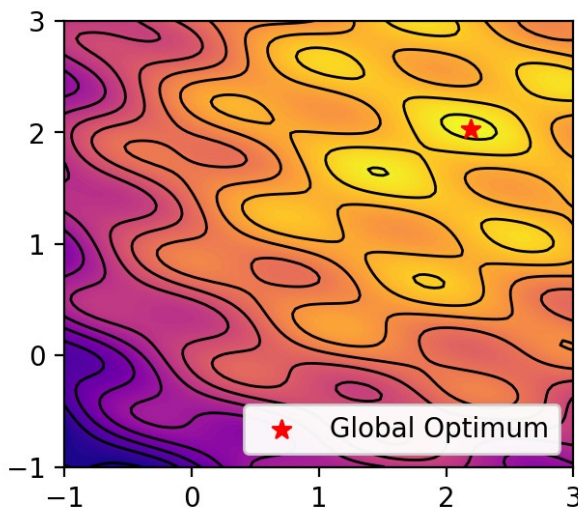
        ax.plot(*lines, color=curve["color"], linestyle="-",
                linewidth=2.0, alpha=0.25, zorder=4)
        ax.scatter(xs, ys, marker="o", facecolors='none',
                edgecolors=curve['color'], s=25, label=label, alpha=0.8, zorder=5)

        ax.set_xticks(np.linspace(-1., 3., 5))
        ax.set_yticks(np.linspace(-1., 3., 5))
        ax.set_xlim(*self.x_lim)
        ax.set_ylim(*self.y_lim)
        if legend:
            handles, labels = ax.get_legend_handles_labels()
            legend = plt.legend(handles=handles, labels=labels, loc='lower
right')
            ax.add_artist(legend)

visualizer = Visualizer(f)

fig, ax = plt.subplots(1, 1, figsize=(3.4, 3.0), dpi=200)
visualizer.draw(ax)

```



As you may have already noticed, there are a lot of local optima in this function. Would this be an issue for gradient descent method? Let's take a look!

Part 1. Gradient Ascent

In this part, we are going to implement gradient ascent and see how it does on this objective function. Your task is to fill in the `gd_step` function below. The function takes in the parameters x_t, y_t and the learning rate η and outputs a tuple (x_{t+1}, y_{t+1}) after one gradient step:

$$x_{t+1} \leftarrow x_t + \eta \frac{\partial f}{\partial x}(x_t, y_t),$$

$$y_{t+1} \leftarrow y_t + \eta \frac{\partial f}{\partial y}(x_t, y_t).$$

Your code should be able to take either scalar x and y or numpy arrays of x and y as input (in that case your output would be a tuple of array rather than a tuple of scalars).

Once you are done implementing the function, run the cell below and see how gradient ascent optimizes this objective function (you will see the trajectory of $\{(x_1, y_1), \dots, (x_T, y_T)\}$). You should feel free to change the initialization of x, y parameters and see how different initializations change the optimization trajectory (the line where it says **CHANGE ME!** below).

```
def grad_f(x, y):
    dx = 2.5 * np.cos(2.5 * (x - 2 * y)) - 4.0 * np.sin(4.0 * (x + 2 *
y)) - 2.0 * (x - 2.0)
    dy = -5.0 * np.cos(2.5 * (x - 2 * y)) - 8.0 * np.sin(4.0 * (x + 2 *
y)) - 2.0 * (y - 2.0)
    return dx, dy

num_its = 100

def gd_step(x, y, eta):
#####
#####
    # TODO: Implement gradient ascent step to change your parameters (x,
y)
    #
    # in the gradient direction (step size = eta).
    #
    # Hint: You may use the grad_f(x, y) that computes the gradient of
f.
    #
#####
#####
    dx, dy = grad_f(x, y)
    new_x = x + eta * dx
    new_y = y + eta * dy
#####
#####
    #
    # END OF YOUR CODE
    #
#####
#####
    return new_x, new_y

num_its = 30          # number of gradient steps
eta = 0.01           # learning rate
```

```

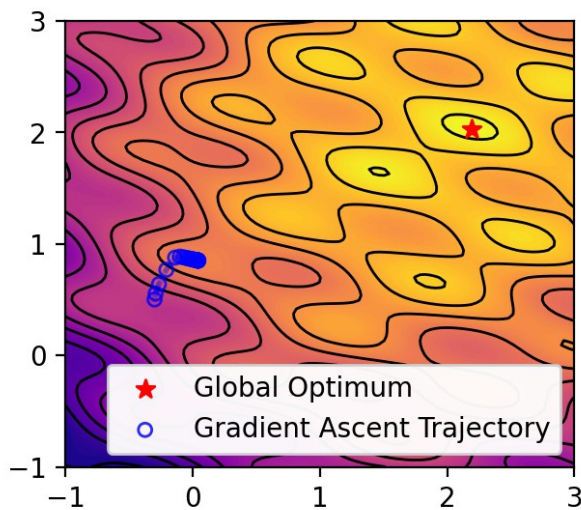
x, y = -0.3, 0.5 # initial parameter values (CHANGE ME!)

# history of parameter values
xs = [x]
ys = [y]

for i in range(num_its):
    x, y = gd_step(x, y, eta)
    xs.append(x)
    ys.append(y)

fig, ax = plt.subplots(1, 1, figsize=(3.4, 3.0), dpi=200)
visualizer.draw(ax, {"Gradient Ascent Trajectory": {"color": "blue",
"x": xs, "y": ys}})

```



(a). What do you observe from the optimization trajectory of your (x, y) parameters? Where do your parameters converge to? Try a few different initialization values and see how the convergence changes.

(your answer) GD gets stuck in local optima. -----

In the next part, we are going to cluster these initialization locations by their values at the end of their optimization trajectories.

```

# we put all points that are less than eps apart (Euclidean distance)
together.
def manual_clustering(t_xs, t_ys, max_num_clusters=23, eps=1e-2):
    labels = np.zeros_like(t_xs, dtype=int)

    codebook = []
    while True:

```

```

unlabeled_indx, unlabeled_indy = np.nonzero(labels == 0)
if len(unlabeled_indx) == 0 or len(codebook) > max_num_clusters:
break

first = (unlabeled_indx[0], unlabeled_indy[0])
codebook.append(np.array([t_xs[first], t_ys[first]]))

matched = (t_xs[first] - t_xs) ** 2 + (t_ys[first] - t_ys) ** 2 <=
eps * eps
labels[matched] = len(codebook)

# codebook stores an array of representative point in the cluster
# labels stores an array of index that represents which cluster each
example belongs to
return np.stack(codebook), labels

# generate a grid of initialization points
t_xs, t_ys = np.linspace(-1, 3, 100), np.linspace(-1, 3, 100)
t_xs, t_ys = np.meshgrid(t_xs, t_ys)
num_its = 1000

# run gradient ascent
t_xs_history, t_ys_history = [t_xs], [t_ys]
for i in range(num_its):
    t_xs, t_ys = gd_step(t_xs, t_ys, 0.01)
    t_xs_history.append(t_xs)
    t_ys_history.append(t_ys)

# cluster them based on the final parameter values
codebook, labels = manual_clustering(t_xs, t_ys)

```

Let's take a look at all the local maxima (the "x" symbols) that we obtained from clustering.

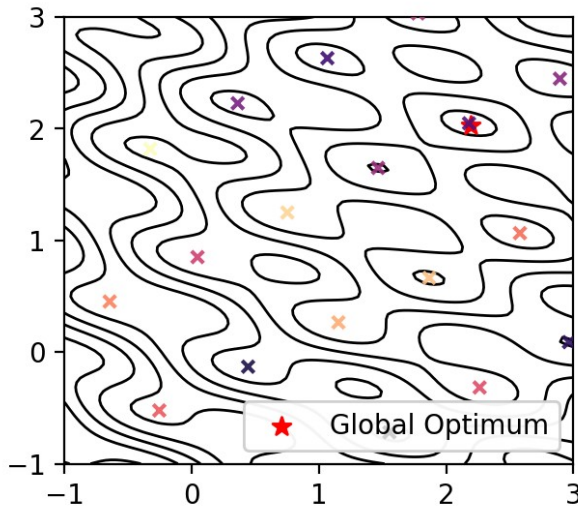
```

import random
cmap = matplotlib.colormaps['magma']
colors = [cmap(x) for x in np.linspace(0, 1.0, len(codebook) + 1)]
random.seed(2)
random.shuffle(colors)
colors = np.array(colors)

fig, ax = plt.subplots(1, 1, figsize=(3.4, 3), dpi=200)
visualizer.draw(ax, no_bg=True)
ax.scatter(codebook[:, 0], codebook[:, 1], color=colors[:-1],
alpha=0.9, s=20, marker="x", zorder=5)

<matplotlib.collections.PathCollection at 0x21849b1f130>

```



Now, we can visualize the trajectories of these clustered initialization points colored with their corresponding clustering labels via the following interaction widget. The slider corresponds to the gradient step. At step 0, we can see the [basins of attraction](#) towards these local optima. Try to drag the slider yourself to see how each initialization moves throughout the optimization process!

```
def update_images(i):
    fig, ax = plt.subplots(1, 1, figsize=(3.4, 3), dpi=200)
    ax.set_facecolor('lightgray')
    visualizer.draw(ax, no_bg=True)
    ax.scatter(t_xs_history[i].reshape(-1), t_ys_history[i].reshape(-1),
               facecolors='none', edgecolors=colors[labels.reshape(-1)], alpha=0.3,
               s=20, marker="o", zorder=5)
    plt.show()

slider = widgets.IntSlider(
    value=0,
    min=0,
    max=100,
    description="# of gd steps"
)
interact(update_images, i=slider)

{"model_id": "3538fb49cdbc4cf38606b58c0c9a3b7c", "version_major": 2, "version_minor": 0}

<function __main__.update_images(i)>
```

(b) What patterns do you observe from the basins of attraction visualization? How do they relate to the sine and cosine in our objective function?

(your answer) Basins of attraction align with sine/cosine frequency → many local maxima.

Next, we study how the optimization trajectory changes as the learning rate changes. Try to drag the slider to change the learning rate below and see how the trajectories from different initialization points change!

```
import matplotlib
from functools import partial

cmap = matplotlib.colormaps['ocean']
seed = 1437

rng = np.random.default_rng(seed=seed)
num_its = 15
num_inits = 3
init_xs, init_ys = rng.random(num_inits) * 0.9, rng.random(num_inits) * 1.2
colors = [cmap(x) for x in np.linspace(0, 0.6, num_inits)]

def generate_curves(eta, optimizer, num_its, num_inits):
    xs, ys = np.empty((num_its + 1, num_inits)), np.empty((num_its + 1, num_inits))
    xs[0], ys[0] = init_xs, init_ys
    for i in range(1, num_its + 1):
        xs[i], ys[i] = optimizer(xs[i-1], ys[i-1], eta)

    curves = {}
    for i in range(num_inits):
        curves[f"Trajectory (Init #{i+1})"] = {"color": colors[i], "x":
xs[:, i], "y": ys[:, i]}
    return curves

def get_interactive(visualizer, ax, gen_fn):

    def update_images(eta):
        fig, ax = plt.subplots(1, 1, figsize=(4.0, 4.0), dpi=200)
        curves = gen_fn(eta)
        visualizer.draw(ax, curves)

    slider = widgets.FloatLogSlider(
        value=0.001,
        base=10,
        min=-3, # min exponent of base
        max=-1, # max exponent of base
        step=0.2, # exponent step
        description='eta'
    )
    return interactive(update_images, eta=slider)
```

```
ip = get_interactive(visualizer, ax, partial(generate_curves,
optimizer=gd_step, num_its=num_its, num_inits=num_inits))
ip

{"model_id": "f73b99e438234890a28615ea7aa2abba", "version_major": 2, "version_minor": 0}
```

(c) What do you notice when the learning rate η changes? Is there a good learning rate that makes it more likely to find the global optimum?

(your answer) Too large $\eta \rightarrow$ diverge; too small $\rightarrow$ slow. Optimal η balances exploration/exploitation. -----

Part 2: Smoothing the optimization problem

We have seen that it is very difficult for gradient ascent to find the global optimum reliably unless the initialization is close to the global optimum. This is due to all the small "hills" and "puddles" that trap the optimization trajectories. Can we optimize a different problem that has a similar global optimum but a much nicer objective landscape?

In this part, we explore a smoothing technique that converts the original objective function into a more smoothed one. In particular, we convolve a 2-D Gaussian kernel over the function ([convolution](#), [Gaussian kernel](#)):

$$f_{\sigma}(x, y) = E_{\tilde{x} \sim N(x, \sigma^2), \tilde{y} \sim N(y, \sigma^2)}[f(\tilde{x}, \tilde{y})]$$

We can view this convolution operation as a continuous interpretation of the convolution layer (e.g., as the image approaches infinite resolution).

Let's first try to visualize the smoothed objective. Since we can't exactly compute the expectation, we are going to approximate it with a Monte-Carlo estimate:

$$\hat{f}_{\sigma}(x, y) = \frac{1}{N} \sum_i [f(\tilde{x}_i, \tilde{y}_i)]$$

with $\tilde{x}_1, \tilde{x}_2, \dots, \tilde{x}_N$ being independent samples from $N(x, \sigma^2)$ and $\tilde{y}_1, \tilde{y}_2, \dots, \tilde{y}_N$ being independent samples from $N(y, \sigma^2)$.

Your task is to implement the `smoothed_f` function below that returns a Monte-Carlo estimate of f_{σ} using parameters σ and N . The function should return a float scalar.

```
def smoothed_f(x, y, sigma, N):
#####
#####
    # TODO: Implement the Monte-Carlo estimate of the smoothed function
    f using #
```



```

#   N samples.
#

#####
#####
    tx = np.random.normal(x, sigma, size=N)
    ty = np.random.normal(y, sigma, size=N)
    return np.mean(f(tx, ty))
#####
#####
#                               END OF YOUR CODE
#

#####
#####

zs_dict = {}
opt_dict = {}
for sigma in [0.01, 0.1, 0.2, 0.3, 0.5]:
    zs = []
    xs, ys = visualizer.xs, visualizer.ys
    for x, y in zip(xs.reshape(-1), ys.reshape(-1)):
        zs.append(smoothed_f(x, y, sigma, N=10000))
    zs_dict[sigma] = np.array(zs).reshape(xs.shape)
    index = np.argmax(zs_dict[sigma].reshape(-1))
    opt_dict[sigma] = (xs.reshape(-1)[index], ys.reshape(-1)[index])

def update_images(sigma):
    fig, ax = plt.subplots(1, 1, figsize=(4.0, 4.0), dpi=200)
    xs, ys = visualizer.xs, visualizer.ys
    zs = zs_dict[sigma]
    ax.contourf(xs, ys, zs, 50, cmap='plasma')
    ax.contour(xs, ys, zs, 15, linewidths=1, colors="black")

    ax.scatter(visualizer.opt_x, visualizer.opt_y, marker="*", s=50,
color="red", label="Global Optimum (Original)", alpha=0.5, zorder=4)
    ax.scatter(*opt_dict[sigma], marker="*", s=50, color="darkgreen",
label="Global Optimum (Smoothed)", alpha=0.5, zorder=4)
    ax.legend()

slider = widgets.SelectionSlider(
    options=[0.01, 0.1, 0.2, 0.3, 0.5],
    value=0.01,
    description='sigma',
    disabled=False,
    continuous_update=False,
    orientation='horizontal',
    readout=True
)
interactive(update_images, sigma=slider)

```

```
{"model_id": "e7e7e0d7e74545768b3469d975798a1c", "version_major": 2, "version_minor": 0}
```

(d) Take a close look at the global optimum for the original function (red star) and the global optimum for the smoothed function (blue star). What do you observe as σ increases? Why? Can you give an example where the global optimum of the smoothed function is very far from the global optimum of the original function?

(your answer) As $\sigma \uparrow$, smoothed optimum moves toward broader peak (not necessarily global of original). -----

Our smoothed function successfully removes all the local optimum except the global optimum. However, we don't know how to optimize this smoothed function efficiently yet. The key challenge here is that we actually don't have a good idea on how to differentiate through f_σ or approximate ∇f_σ .

In the following part, we will be exploring three different optimization techniques to address this issue. We are going to use $\sigma = 0.3$

Part 3a: Finite-difference method

The first method that we are going to explore is the finite-difference method where we are going to approximate the gradient by trying to step in each dimension of θ and evaluate the difference in the objective function:

$$\frac{\partial f_\sigma(x, y)}{\partial x} \approx \frac{\hat{f}_\sigma(x + \delta, y) - \hat{f}_\sigma(x, y)}{\delta}$$
$$\frac{\partial f_\sigma(x, y)}{\partial y} \approx \frac{\hat{f}_\sigma(x, y + \delta) - \hat{f}_\sigma(x, y)}{\delta}$$

where $\hat{f}(x, y)$ is the Monte-Carlo estimate of $f(x, y)$ using random samples of x and y from the Gaussian distribution (same as the $\hat{f}$ in the previous part).

Your task is to implement the `finite_diff_grad_step` function below. In the function, you should first estimate the gradient of the smoothed function f_σ using the finite-difference method using δ , σ , and N . Then, you should use the gradient estimate to take a step with a step size of η and return the new parameter values.

```
def finite_diff_grad_step(x, y, delta, N, eta, sigma):
    tx = np.random.normal(x, sigma, size=(N,)) # tilde x samples
    ty = np.random.normal(y, sigma, size=(N,)) # tilde y samples

    #####
    #####
    # TODO: Implement the finite-difference gradient estimator of the
    smoothed #
```

```

# function f.
#
# Estimate f_sigma(x, y)
f_xy = np.mean([f(np.random.normal(x, sigma), np.random.normal(y,
sigma)) for _ in range(N)])
# Estimate f_sigma(x + delta, y)
f_xd = np.mean([f(np.random.normal(x + delta, sigma),
np.random.normal(y, sigma)) for _ in range(N)])
# Estimate f_sigma(x, y + delta)
f_yd = np.mean([f(np.random.normal(x, sigma), np.random.normal(y +
delta, sigma)) for _ in range(N)])

df_dx = (f_xd - f_xy) / delta
df_dy = (f_yd - f_xy) / delta

new_x = x + eta * df_dx
new_y = y + eta * df_dy
#####

#####

#####
#                                END OF YOUR CODE
#

#####
#####
return new_x, new_y

num_its = 30      # number of gradient steps
eta = 0.1         # learning rate
delta = 0.25      # finite difference magnitude
N = 20            # number of samples for MC estimation
sigma = 0.3       # the magnitude of the Gaussian noise.
x, y = -0.3, 0.5  # initial parameter values

# history of parameter values
xs = [x]
ys = [y]

for i in range(num_its):
    x, y = finite_diff_grad_step(x, y, delta, N, eta, sigma)
    xs.append(x)
    ys.append(y)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(7.2, 3.2), dpi=200)
visualizer.draw(ax1, {f"FD Trajectory": {"color": "blue", "x": xs,
"y": ys}}, new_opt=opt_dict[sigma], legend=False)
visualizer.draw(ax2, {f"FD Trajectory": {"color": "blue", "x": xs,
"y": ys}}, new_zs=zs_dict[sigma], new_opt=opt_dict[sigma],

```

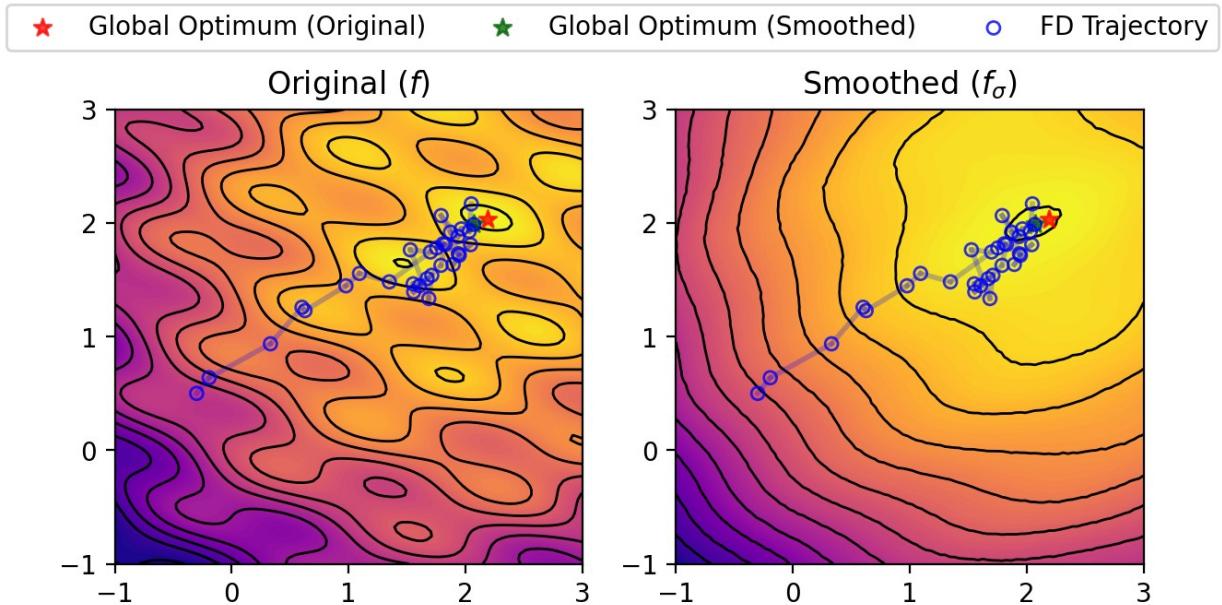
```

legend=False)

ax1.set_title("Original ($f$)")
ax2.set_title("Smoothed ($f_\sigma$)")
handles, labels = ax2.get_legend_handles_labels()
fig.legend(handles, labels, loc='upper center', ncol=3,
bbox_to_anchor=(0.5, 1.08))

<matplotlib.legend.Legend at 0x21853d24df0>

```



Now, we are going to look at how changing η , δ and N influences the learning trajectory. Run the following cell and drag the sliders to try it out yourself!

```

def get_interactive(grad_step_fn, label_name, init_x=-0.3, init_y=0.5,
num_its=30, has_delta=False):

    def generate_curves(*args, **kwargs):
        x, y = init_x, init_y
        xs, ys = [init_x], [init_y]
        for i in range(1, num_its + 1):
            x, y = grad_step_fn(x, y, *args, **kwargs)
            xs.append(x)
            ys.append(y)

        return {label_name: {"color": "blue", "x": xs, "y": ys}}

    def update_images(*args, **kwargs):
        sigma=0.3
        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(7.2, 3.2), dpi=200)
        curves = generate_curves(*args, **kwargs, sigma=sigma)

```

```

    visualizer.draw(ax1, curves, new_opt=opt_dict[sigma],
legend=False)
    visualizer.draw(ax2, curves, new_zs=zs_dict[sigma],
new_opt=opt_dict[sigma], legend=False)

    ax1.set_title("Original (%f%)")
    ax2.set_title("Smoothed (%f_\sigma%)")
    handles, labels = ax2.get_legend_handles_labels()
    fig.legend(handles, labels, loc='upper center', ncol=3,
bbox_to_anchor=(0.5, 1.08))
    plt.show()

eta_slider = widgets.FloatLogSlider(
    value=0.1,
    base=10,
    min=-3, # min exponent of base
    max=0, # max exponent of base
    step=0.2, # exponent step
    description='eta'
)

N_slider = widgets.SelectionSlider(
    options=[1, 2, 5, 10, 20, 50, 100],
    value=20,
    description='N',
    disabled=False,
    continuous_update=False,
    orientation='horizontal',
    readout=True
)

if has_delta:
    delta_slider = widgets.FloatLogSlider(
        value=0.3,
        base=10,
        min=-3, # min exponent of base
        max=0, # max exponent of base
        step=0.2, # exponent step
        description='delta'
    )

    return interactive(update_images, N=N_slider, eta=eta_slider,
delta=delta_slider)
    return interactive(update_images, N=N_slider, eta=eta_slider)

get_interactive(finite_diff_grad_step, "FD Trajectory",
has_delta=True)

{"model_id": "261f7380c84c420a87c6e4fe005efe3c", "version_major": 2, "version_minor": 0}

```

(e) What do you observe when you change δ , η (learning rate) and N (the sample size for Monte-Carlo estimate)?

(your answer) As $\sigma \uparrow$, smoothed optimum moves toward broader peak (not necessarily global of original). -----

Part 3b: Policy Gradient

Finite-difference method uses zero gradient information about the objective function. Can we come up with a technique that uses more information about our smoothed function?

In this part, you will learn about a technique that leverages the gradient information of the probability density function of the Gaussian kernel to better estimate the gradient of the smoothed function.

To start with, let's first write out the smoothed function in the integral form:

$$f_{\sigma}(x, y) = E_{\tilde{x}, \tilde{y} \sim N([x, y]^T, \sigma^2 I)}[f(\tilde{x}, \tilde{y})] = \int \int p(\tilde{x}, \tilde{y} | x, y) f(\tilde{x}, \tilde{y}) d\tilde{x} d\tilde{y}$$

$$\text{where } p(\tilde{x}, \tilde{y} | x, y) = \frac{1}{2\pi\sigma} \exp\left(-\frac{(\tilde{x} - x)^2 + (\tilde{y} - y)^2}{2\sigma^2}\right).$$

Now, we can take the derivative with respect to x :

$$\frac{\partial f_{\sigma}(x, y)}{\partial x} = \int \int \frac{\partial p(\tilde{x}, \tilde{y} | x, y)}{\partial x} f(\tilde{x}, \tilde{y}) d\tilde{x} d\tilde{y} = \int \int \frac{1}{2\pi\sigma} \exp\left(-\frac{(\tilde{x} - x)^2 + (\tilde{y} - y)^2}{2\sigma^2}\right) [(\tilde{x} - x)/\sigma^2] f(\tilde{x}, \tilde{y}) d\tilde{x} d\tilde{y}$$

While it is nice that we can represent it in the integral form, we don't know how to tractably compute it. So how should we proceed from here?

You might have already observed that there is a term in the operand of the integral that is same as $p(\tilde{x}, \tilde{y} | x, y)$. This means that we can actually put the result back to the expectation form!

$$\frac{\partial f_{\sigma}(x, y)}{\partial x} = \int \int p(\tilde{x}, \tilde{y} | x, y) (\tilde{x} - x) f(\tilde{x}, \tilde{y}) / \sigma^2 d\tilde{x} d\tilde{y} = E[(\tilde{x} - x) f(\tilde{x}, \tilde{y}) / \sigma^2]$$

By symmetry, we also have

$$\frac{\partial f_{\sigma}(x, y)}{\partial y} = E[(\tilde{y} - y) f(\tilde{x}, \tilde{y}) / \sigma^2]$$

With the gradient of the smoothed function in the expectation form, we can use it to approximate the gradient using Monte-Carlo estimation:

$$\frac{\partial f_{\sigma}(x, y)}{\partial x} \approx \frac{1}{N} \sum_i [(\tilde{x}_i - x) f(\tilde{x}_i, \tilde{y}_i) / \sigma^2]$$

$$\frac{\partial f_{\sigma}(x, y)}{\partial y} \approx \frac{1}{N} \sum_i [(\tilde{y}_i - y) f(\tilde{x}_i, \tilde{y}_i) / \sigma^2]$$

where again $\{\tilde{x}_i\}_{i=1}^N, \{\tilde{y}_i\}_{i=1}^N$ are independent samples of $N(x, \sigma^2)$ and $N(y, \sigma^2)$ respectively. The gradient estimator of such form is called the *policy gradient estimator* or *score function gradient estimator*.

(f) Can you provide an intuitive explanation of why the gradient estimator works? (Hint: which direction is the gradient estimator trying to push (x, y) into? By what magnitude?)

(your answer) Policy gradient pushes (x, y) toward regions with higher f . -----

Now comes the question -- What if $p(\tilde{x}, \tilde{y} \vee x, y)$ is not Gaussian? Is there still a nice way to rewrite the gradient of the smoothed function as an expectation of something? You will explore this more in the written part of your homework!

For now, your task is to implement the `pg_grad_step` function below. In the function, you should first estimate the gradient of the smoothed function f_{σ} using the policy gradient method using σ , and N . Then, you should use the gradient estimate to take a step with a step size of η and return the new parameter values.

```
def pg_grad_step(x, y, N, eta, sigma):
    tx = np.random.normal(x, sigma, size=(N,)) # tilde x samples
    ty = np.random.normal(y, sigma, size=(N,)) # tilde y samples

    #####
    #####
    # TODO: Implement the policy gradient estimator for the smoothed
    function f #

    f_vals = f(tx, ty)
    df_dx = np.mean((tx - x) * f_vals) / (sigma ** 2)
    df_dy = np.mean((ty - y) * f_vals) / (sigma ** 2)
    new_x = x + eta * df_dx
    new_y = y + eta * df_dy
    #####
    #####

    #####
    #####
    #
    #
    #
    #####
    #####
    return new_x, new_y

num_its = 30 # number of gradient steps
```



```

eta = 0.1           # learning rate
N = 20             # number of samples for MC estimation
sigma = 0.3        # the magnitude of the Gaussian noise
x, y = -0.3, 0.5   # initial parameter values

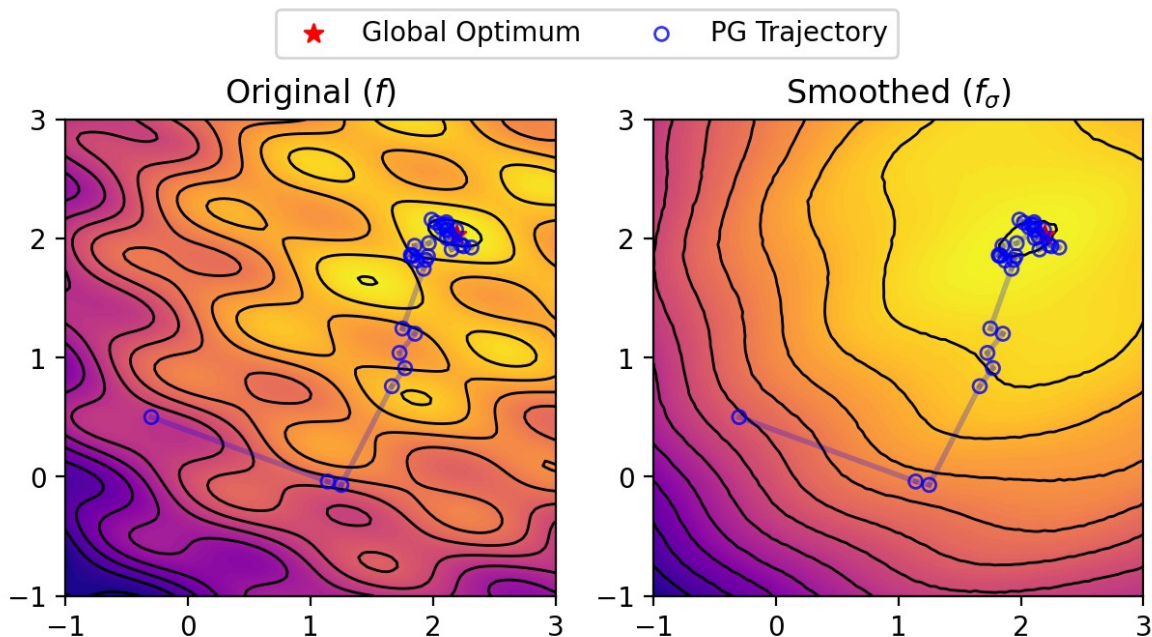
# history of parameter values
xs = [x]
ys = [y]

for i in range(num_its):
    x, y = pg_grad_step(x, y, N, eta, sigma)
    xs.append(x)
    ys.append(y)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(7.2, 3.2), dpi=200)
visualizer.draw(ax1, {f"PG Trajectory": {"color": "blue", "x": xs,
"y": ys}}, legend=False)
visualizer.draw(ax2, {f"PG Trajectory": {"color": "blue", "x": xs,
"y": ys}}, new_zs=zs_dict[sigma], legend=False)

ax1.set_title("Original ($f$)")
ax2.set_title("Smoothed ($f_{\sigma}$)")
handles, labels = ax2.get_legend_handles_labels()
fig.legend(handles, labels, loc='upper center', ncol=2,
bbox_to_anchor=(0.5, 1.08))
pass

```



Now, we are going to look at how changing η and N influences the learning trajectory. Run the following cell and drag the sliders to try it out yourself!


```
get_interactive(pg_grad_step, "PG Trajectory", has_delta=False)
{"model_id": "ac716a1658b24d11b3754e3394ce1d46", "version_major": 2, "version_minor": 0}
```

(g) What do you observe when you change η (learning rate) and N (the sample size for Monte-Carlo estimate)?

(your answer) Policy gradient pushes (x, y) toward regions with higher f . -----

Part 3c: Reparameterization Gradient

Policy gradient uses the information about the smoothing kernel to provide a better gradient estimation. Can we use any gradient information about $f(x)$? This might seem a bit unintuitive, as the function that has lots of local optima probably have poor gradient information locally. As we will show in this part, we can actually harness the poor gradient information to provide better gradient estimation of the smoothed function.

Let's go back to the definition of the smoothed function again (and take the gradient operator with respect to (x, y)):

$$\nabla f_{\sigma}(x, y) = \nabla E_{\tilde{x}, \tilde{y} \sim N((x, y), \sigma^2 I)}[f(\tilde{x}, \tilde{y})]$$

It would be so nice if we could move the gradient inside the expectation, but that is not possible immediately because the expectation distribution depends on x and y and the term inside the expectation does not directly depend on x and y . Can we somehow rewrite the expectation such that the distribution does not depend on x and y ? Or in another word, can we separate out the noise in $\tilde{x}$ and $\tilde{y}$ such that they can be rewritten as a deterministic function of x, y and some noise that is independent of x and y ?

Yes! Let z_x, z_y be independent samples from the Gaussian $N(0, \sigma^2)$. We can simply rewrite $\tilde{x}$ and $\tilde{y}$ as follows:

$$\tilde{x} = z_x + x,$$

$$\tilde{y} = z_y + y.$$

With such a reparameterization, we can now express the smoothed function using an expectation where its distribution is independent of x and y ! This would allow us to move the gradient operator inside:

$$\nabla f_{\sigma}(x, y) = \nabla E_{z_x, z_y \sim N(0, \sigma^2)}[f(x + z_x, y + z_y)] = E_{z_x, z_y \sim N(0, \sigma^2)}[\nabla f(x + z_x, y + z_y)]$$

Writing out the derivative for x and y separately:

$$\frac{\partial f_{\sigma}(x, y)}{\partial x} = E_{z_x, z_y \sim N(0, \sigma^2)}[f_x(x + z_x, y + z_y)]$$

$$\frac{\partial f_{\sigma}(x, y)}{\partial y} = E_{z_x, z_y \sim N(0, \sigma^2)}[f_y(x + z_x, y + z_y)]$$

where f_x is the partial derivative of f with respect to the first input (x) and f_y is the partial derivative of f with respect to the second input (y).

As you might have already guessed it, we can perform Monte-Carlo estimate of each of the derivative by drawing z_x and z_y samples. The gradient estimator of such form is called the *reparameterization gradient estimator*.

Your task is to implement the `reparam_grad_step` function below. In the function, you should first estimate the gradient of the smoothed function f_{σ} using the reparameterization gradient method using σ , and N . Then, you should use the gradient estimate to take a step with a step size of η and return the new parameter values.

```
def reparam_grad_step(x, y, N, eta, sigma):
    tx = np.random.normal(x, sigma, size=(N,)) # tilde x samples
    ty = np.random.normal(y, sigma, size=(N,)) # tilde y samples

    #####
    # TODO: Implement the reparameterization gradient estimator of the
    smoothed #
    # function f.
    #
    # Compute gradients of f at (tx, ty)
    df_dx, df_dy = grad_f(tx, ty)
    # Average
    avg_df_dx = np.mean(df_dx)
    avg_df_dy = np.mean(df_dy)
    new_x = x + eta * avg_df_dx
    new_y = y + eta * avg_df_dy
    #####

    #####
    #
    # END OF YOUR CODE
    #

    #####
    return new_x, new_y

num_its = 30 # number of gradient steps
eta = 0.1 # learning rate
N = 20 # number of samples for MC estimation
sigma = 0.3 # the magnitude of the Gaussian noise
```

```

x, y = -0.3, 0.5 # initial parameter values

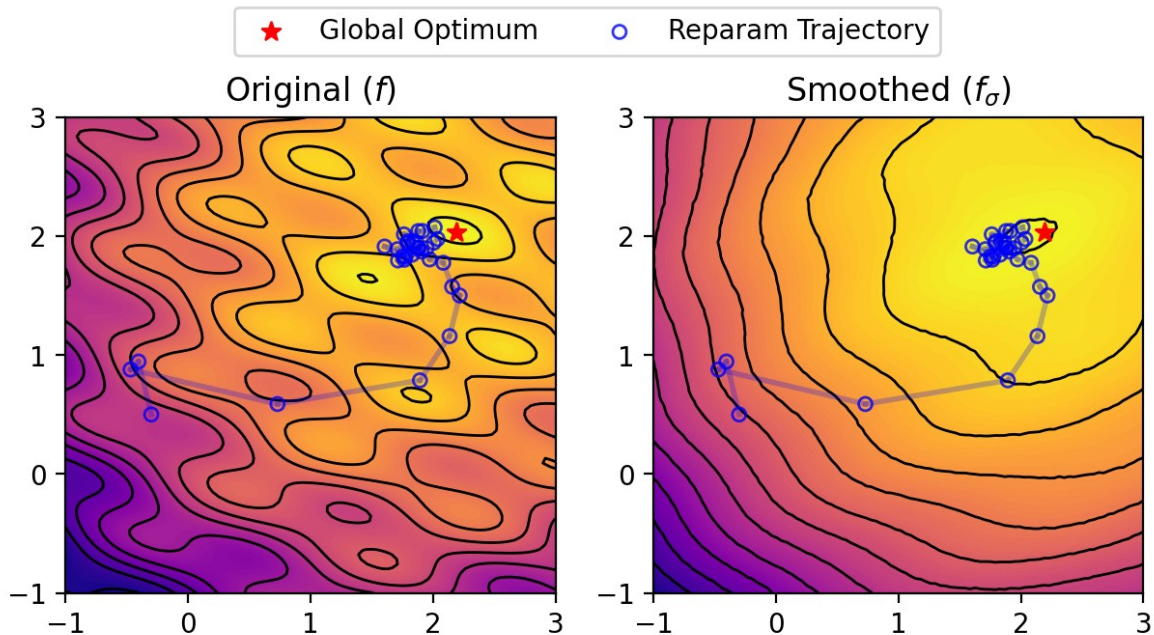
# history of parameter values
xs = [x]
ys = [y]

for i in range(num_its):
    x, y = pg_grad_step(x, y, N, eta, sigma)
    xs.append(x)
    ys.append(y)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(7.2, 3.2), dpi=200)
visualizer.draw(ax1, {f"Reparam Trajectory": {"color": "blue", "x":
xs, "y": ys}}, legend=False)
visualizer.draw(ax2, {f"Reparam Trajectory": {"color": "blue", "x":
xs, "y": ys}}, new_zs=zs_dict[sigma], legend=False)

ax1.set_title("Original ($f$)")
ax2.set_title("Smoothed ($f_\sigma$)")
handles, labels = ax2.get_legend_handles_labels()
fig.legend(handles, labels, loc='upper center', ncol=2,
bbox_to_anchor=(0.5, 1.08))
pass

```



```

get_interactive(reparam_grad_step, "Reparam Trajectory",
has_delta=False)

{"model_id": "2d7cfa04358e4f3fb36ddd0b7a76be06", "version_major": 2, "version_minor": 0}

```

(h) Compare the trajectory of reparam gradient vs. policy gradient. What do you observe? Is one more straight than the other one? Why do you think that is the case?

(your answer) Policy gradient pushes (x,y) toward regions with higher f . -----

Part 4: Objective functions with discontinuities

In the previous part, we introduced three optimization techniques to find the global optimum of a smoothed function. In this part, we are going to explore a more extreme objective function that contains discontinuities, $f_q(x, y)$. f_q is obtained by quantizing the objective function that we were using in the previous parts. Can we use what we have learned so far to optimize this objective function?

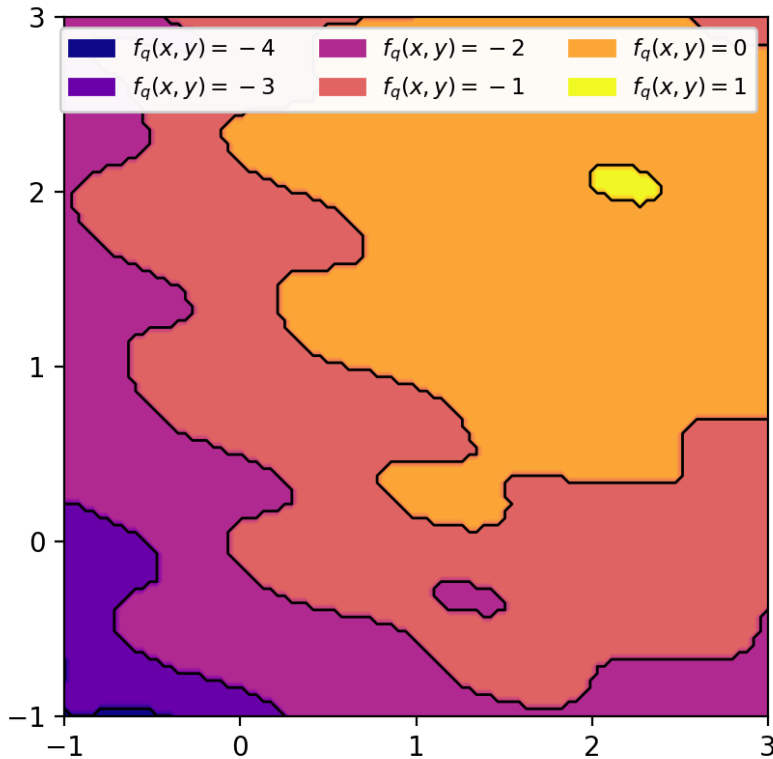
Let's first visualize what the function looks like!

```
def quantize(s):
    return np.floor(s / 4.5 + 0.64)

def qf(x, y):
    return quantize(np.sin(2.5 * (x - 2 * y)) + np.cos(4.0 * (x + 2 *
y))) - ((x - 2.0) ** 2 + (y - 2.0) ** 2))

q_visualizer = Visualizer(qf, n_contour_lines=5, quantized_values=[-4,
-3, -2, -1, 0, 1])

fig, ax = plt.subplots(1, 1, figsize=(4.70, 4.70), dpi=200)
q_visualizer.draw(ax, legend=False)
```



From the visualization, the function only outputs values from $\{-4, -3, -2, -1, 0, 1\}$ with $f_q(x, y)=1$ being the maximum value (located top right, yellow region).

(i) How well do you expect naive gradient descent algorithm to perform on this quantized objective function? Explain Why.

(your answer) Policy gradient pushes (x, y) toward regions with higher f . **(j) Can you use reparameterization gradient for this function? Explain why. What about finite-difference?**

(your answer) Policy gradient pushes (x, y) toward regions with higher f . -----

Your task below is to write code to use a suitable technique that you learned above to reach the maximum region of this quantized function. We provide some helper code below to get you started. Depending on the technique you use, you may or may not need δ in the function input. You should be able to reach the maximum region under 30 gradient steps and $N \leq 50$ with high probability (>85%).

```
def q_grad_step(x, y, N, eta, sigma, delta):
    tx = np.random.normal(x, sigma, size=(N,)) # tilde x samples
    ty = np.random.normal(y, sigma, size=(N,)) # tilde y samples

    #####
    #####
    # TODO: Implement the policy gradient estimate of the gradient of
```

```

the      #
    # smoothed function f. (Hint: make sure to use qf instead of f as
used      #
    # previously
#
    # Estimate qf_sigma(x, y)
    f_xy = np.mean([qf(np.random.normal(x, sigma), np.random.normal(y,
sigma)) for _ in range(N)])
    # Estimate qf_sigma(x + delta, y)
    f_xd = np.mean([qf(np.random.normal(x + delta, sigma),
np.random.normal(y, sigma)) for _ in range(N)])
    # Estimate qf_sigma(x, y + delta)
    f_yd = np.mean([qf(np.random.normal(x, sigma), np.random.normal(y +
delta, sigma)) for _ in range(N)])

    # Finite difference gradients
    df_dx = (f_xd - f_xy) / delta
    df_dy = (f_yd - f_xy) / delta

    # Gradient ascent step
    new_x = x + eta * df_dx
    new_y = y + eta * df_dy
#####

#####

#####
#                                     END OF YOUR CODE
#

#####
#####
    return new_x, new_y

num_its = 30      # number of gradient steps
eta = 0.5        # learning rate
N = 50           # number of samples for MC estimation
sigma = 0.3      # the magnitude of the Gaussian noise
delta = 0.25     # finite difference magnitude
x, y = -0.3, 0.5 # initial parameter values

# history of parameter values
xs = [x]
ys = [y]

#####
#####
# TODO: Implement the for loop that takes policy gradient steps. Make
sure #
# to append your intermediate x and y values into the list xs and ys

```

```

so that#
# the training trajectory can be visualized later (hint: you might
find the #
# code you have seen in the previous part helpful)
#
for i in range(num_its):
    x, y = q_grad_step(x, y, N, eta, sigma, delta)
    xs.append(x)
    ys.append(y)
#####
#####
#####
#####
#                                     END OF YOUR CODE
#
#####
#####

fig, ax = plt.subplots(1, 1, figsize=(4.70, 4.70), dpi=200)
q_visualizer.draw(ax, {f"PG Trajectory": {"color": "blue", "x": xs,
"y": ys}})

```

